
INDIAN INSTITUTE OF TECHNOLOGY,
GUWAHATI

PROJECT REPORT

on

**Modelling and Trajectory Tracking
Controller Design for Two
Wheeled Mobile Robots with
Non Holonomic Constraints**

Name	Aritra Biswas
Roll No.	254102503
Department	Electronics and Electrical Engineering
Institution	Indian Institute Of Technology, Guwahati

2026

Abstract

Trajectory tracking is one of the fundamental control problems in autonomous mobile robotics. The objective is to enable a mobile robot to follow a predefined time-varying trajectory while maintaining the required position and orientation.

In this project, a differential-drive TurtleBot3 is considered for trajectory tracking using Robot Operating System 2 (ROS 2) and MATLAB/Simulink. The robot's actual position and orientation are obtained through odometry information communicated over the ROS 2 `/odom` topic. The quaternion representation of orientation is processed to obtain the yaw angle.

A circular reference trajectory is generated, and the desired position and orientation are compared with the measured robot state to obtain the tracking errors. These errors are transformed from the global coordinate frame into the robot coordinate frame. A nonlinear feedback-feedforward kinematic controller is then used to generate the commanded linear and angular velocities.

The generated commands are subjected to velocity limitations and subsequently passed through a dynamic velocity control stage. The complete system forms a closed-loop control architecture in which the actual robot state is continuously fed back to the controller.

The project demonstrates the integration of ROS 2 communication, MATLAB/Simulink modelling, robot kinematics, trajectory generation, coordinate transformation, nonlinear control, and dynamic velocity control for mobile-robot trajectory tracking.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 ROS 2 and Simulink Integration	2
1.3 Motivation	3
1.4 Problem Statement	3
1.5 Need for Trajectory Tracking	5
1.6 Proposed Approach	5
1.6.1 Step 1: Obtain Robot State	6
1.6.2 Step 2: Obtain Robot Orientation	6
1.6.3 Step 3: Generate Reference Trajectory	6
1.6.4 Step 4: Calculate Tracking Errors	7
1.6.5 Step 5: Transform Errors	7
1.6.6 Step 6: Apply Kinematic Control	7
1.6.7 Step 7: Apply Velocity Limits	8
1.6.8 Step 8: Dynamic Velocity Control	8
1.7 Project Methodology	8
1.8 Scope of the Project	9
1.9 Significance of the Project	10
2 Objectives	12
2.1 Introduction	12
2.2 Main Objective	12
2.3 Specific Objectives	13
2.3.1 Establish ROS 2–Simulink Communication	14

2.3.2	Acquire Robot Odometry	14
2.3.3	Obtain Robot Orientation	15
2.4	Generate the Desired Trajectory	15
2.5	Synchronize the Initial Reference With the Robot	16
2.6	Calculate Position and Orientation Tracking Errors	17
2.7	Transform the Tracking Error Into the Robot Frame	17
2.8	Develop the Nonlinear Kinematic Controller	18
2.9	Implement Feedback and Feedforward Control	19
2.9.1	Feedback Control	19
2.9.2	Feedforward Control	19
2.10	Control Lateral Tracking Error	20
2.11	Apply Velocity Constraints	20
2.12	Develop the Dynamic Velocity Controller	21
2.13	Implement the Complete Closed-Loop System	22
2.14	Evaluate Trajectory-Tracking Performance	22
2.14.1	Position Tracking	23
2.14.2	Orientation Tracking	23
2.14.3	Tracking Errors	23
2.14.4	Velocity Response	23
2.14.5	Overall Trajectory	24
2.15	Performance Evaluation Parameters	24
2.15.1	Maximum Tracking Error	24
2.15.2	Root Mean Square Error	24
2.16	Overall Objective	25
3	System Overview	26
3.1	Introduction	26
3.2	Overall System Architecture	27
3.3	TurtleBot3	28
3.4	Differential-Drive Configuration	28
3.5	ROS 2 Framework	29
3.6	ROS 2 Odometry Communication	30
3.7	ROS 2 Publisher and Subscriber	31

3.7.1	Publisher	31
3.7.2	Subscriber	31
3.8	Odometry State Processing	31
3.9	Quaternion Orientation	32
3.10	Reference Trajectory Generation	33
3.11	Actual and Desired State Comparison	33
3.12	Global and Robot Coordinate Frames	34
3.12.1	Global Frame	34
3.12.2	Robot Frame	34
3.13	Coordinate Transformation	35
3.14	Kinematic Controller	36
3.15	Linear Velocity Control	36
3.16	Angular Velocity Control	37
3.16.1	Heading Correction	37
3.16.2	Lateral Correction	37
3.16.3	Feedforward Term	37
3.17	Velocity Saturation	38
3.18	Dynamic Velocity Controller	38
3.19	Dynamic Controller Operation	39
3.20	Command Generation and Publication	39
3.21	ROS 2 Command Publisher	40
3.22	Complete Closed-Loop Architecture	41
3.23	Working Principle of the Complete System	41
3.23.1	Step 1 – Robot Motion	42
3.23.2	Step 2 – State Measurement	42
3.23.3	Step 3 – ROS 2 Communication	42
3.23.4	Step 4 – State Processing	42
3.23.5	Step 5 – Yaw Calculation	42
3.23.6	Step 6 – Reference Generation	42
3.23.7	Step 7 – Error Calculation	42
3.23.8	Step 8 – Coordinate Transformation	42
3.23.9	Step 9 – Kinematic Control	43

3.23.10 Step 10 – Saturation	43
3.23.11 Step 11 – Dynamic Control	43
3.23.12 Step 12 – ROS 2 Publication	43
3.23.13 Step 13 – Robot Motion	43
3.23.14 Step 14 – Feedback	43
3.24 Summary	43
4 ROS 2 Communication and Data Acquisition	46
4.1 Introduction	46
4.2 Role of ROS 2 in the Proposed System	47
4.3 ROS 2 Communication Architecture	47
4.4 ROS 2 Subscriber	48
4.5 Odometry Data	49
4.6 Structure of Quaternion Data	49
4.7 Quaternion-to-Yaw Conversion	50
4.8 Importance of Yaw Angle	51
4.9 Extraction of Actual Robot State	51
4.10 Reference and Actual State	52
4.11 Role of ROS 2 in the Closed-Loop System	52
4.11.1 Stage 1 – Measurement	53
4.11.2 Stage 2 – Communication	53
4.11.3 Stage 3 – Processing	53
4.11.4 Stage 4 – Control	53
4.11.5 Stage 5 – Command Communication	53
4.11.6 Stage 6 – Robot Motion	53
4.11.7 Stage 7 – Feedback	53
4.12 Controller Output	53
4.13 Dynamic Controller and ROS 2 Interface	54
4.14 Command Message Formation	55
4.15 /cmd_vel Topic	55
4.16 ROS 2 Feedback Path	56
4.17 Complete ROS 2 Data Flow	56
4.18 Closed-Loop Operation	57

4.19	Importance of ROS 2 Communication in the Project	58
4.20	Summary	59
5	Robot Modelling	61
5.1	Introduction	61
5.2	TurtleBot3 as a Differential-Drive Robot	62
5.3	Robot Coordinate System	62
5.3.1	Global Coordinate Frame	63
5.3.2	Robot Coordinate Frame	63
5.4	Robot Pose	63
5.5	Control Inputs	64
5.6	Kinematic Model of TurtleBot3	65
5.7	Interpretation of the Kinematic Equations	65
5.8	State-Space Representation	66
5.9	Nonholonomic Constraint	67
5.10	Wheel-Speed Relationship	67
5.11	Straight-Line Motion	68
5.12	Rotational Motion	69
5.13	Circular Motion	69
5.14	Desired Robot Orientation	70
5.15	Actual Robot State From Odometry	71
5.16	Desired and Actual State Comparison	71
5.17	Global-to-Robot Coordinate Transformation	72
5.18	Physical Interpretation of Robot-Frame Errors	73
5.18.1	Forward Error	73
5.18.2	Lateral Error	73
5.18.3	Heading Error	73
5.19	Velocity Constraints	73
5.20	Dynamic Model	74
5.21	Kinematic and Dynamic Control Structure	75
5.21.1	Outer Kinematic Control	75
5.21.2	Inner Dynamic Velocity Control	75
5.22	Mathematical Model Used in the Controller	75

5.22.1	Robot State	76
5.22.2	Control Input	76
5.22.3	Kinematic Model	76
5.22.4	Tracking Error	76
5.22.5	Robot-Frame Error	76
5.23	Relationship Between Robot Model and Controller	77
5.24	Summary	77
6	Odometry and Orientation Processing	79
6.1	Introduction	79
6.2	Odometry	80
6.3	ROS 2 /odom Feedback	80
6.4	Odometry Message Processing	81
6.5	Position Extraction	81
6.6	Orientation Representation	82
6.7	Need for Quaternion-to-Yaw Conversion	82
6.8	Quaternion-to-Euler Conversion	83
6.9	Yaw Angle	83
6.10	Relationship Between Orientation and Robot Motion	84
6.11	Actual Robot State	84
6.12	Desired State	85
6.13	Initial State Synchronization	86
6.14	Position Tracking Error	86
6.15	Orientation Tracking Error	87
6.16	Global-Frame Error	88
6.17	Global-to-Robot Frame Transformation	88
6.18	Physical Meaning of the Transformed Errors	89
6.18.1	Forward/Backward Error	89
6.18.2	Lateral Error	89
6.18.3	Heading Error	89
6.19	Complete State-Processing Chain	89
6.20	Importance of Accurate State Processing	90
6.21	Relationship With the Kinematic Controller	91

6.22 Summary	92
7 Reference Trajectory Generation	94
7.1 Introduction	94
7.2 Purpose of Reference Trajectory	95
7.3 Selection of Circular Trajectory	95
7.4 Mathematical Representation of Circular Trajectory	96
7.5 Desired Orientation	97
7.6 Desired Linear Velocity	97
7.7 Reference Trajectory Parameters	99
7.8 Initial Conditions	99
7.9 Importance of Initial Synchronization	100
7.10 MATLAB Function Implementation	101
7.11 Reference Trajectory and Actual Trajectory	101
7.12 Calculation of Position Errors	102
7.13 Calculation of Orientation Error	102
7.14 Reference Velocity as Feedforward Information	103
7.15 Reference Trajectory in Simulink	104
7.16 Relationship Between Trajectory Generation and Controller	104
7.17 Circular Trajectory Motion	105
7.18 Trajectory Generation Algorithm	106
7.19 Reference Generation and Closed-Loop Tracking	107
7.20 Summary of Reference Trajectory	108
7.21 Summary	109
8 Tracking Error Calculation	111
8.1 Introduction	111
8.2 Purpose of Tracking-Error Calculation	112
8.3 Desired Robot State	112
8.4 Actual Robot State	113
8.5 General Error Vector	114
8.6 X-Position Error	114
8.7 Y-Position Error	115

8.8	Orientation Error	116
8.9	Complete Tracking-Error Vector	116
8.10	Why Three Errors Are Required	117
8.11	Example of Error Calculation	117
8.12	Position Error Magnitude	119
8.13	Orientation Error Magnitude	120
8.14	Error Calculation for Circular Trajectory	120
8.15	Error Calculation in Simulink	121
8.16	From Global Error to Robot-Frame Error	121
8.17	Why Error Transformation Is Necessary	122
8.18	Error Signals Used by the Controller	123
8.19	Error and Linear Velocity Control	123
8.20	Error and Angular Velocity Control	124
8.21	Desired Tracking Condition	125
8.22	Error Convergence	125
8.23	Error Monitoring	126
8.24	Performance Measures Based on Error	127
8.25	Error Calculation Algorithm	127
8.26	Complete Error-Processing Structure	129
8.27	Summary	130
9	Global-to-Robot Frame Transformation	132
9.1	Introduction	132
9.2	Need for Coordinate Transformation	132
9.3	Global Coordinate Frame	133
9.4	Robot Coordinate Frame	134
9.5	Position Error in the Global Frame	134
9.6	Rotation Matrix	135
9.7	Derivation of Forward Error	135
9.8	Derivation of Lateral Error	136
9.9	Orientation Error	137
9.10	Complete Transformation Equation	137
9.11	Physical Interpretation	138

9.11.1 Forward Error	138
9.11.2 Lateral Error	138
9.11.3 Orientation Error	138
9.12 Special Case: Robot Facing the Global X -Axis	138
9.13 Special Case: Robot Rotated by 90°	139
9.14 Example of Coordinate Transformation	140
9.15 Coordinate Transformation in Simulink	141
9.16 Signal Flow of the Transformation Block	142
9.17 Relationship With Trajectory Tracking	143
9.18 Relationship With the Kinematic Controller	143
9.19 Effect of Forward Error	144
9.20 Effect of Lateral Error	144
9.21 Effect of Orientation Error	145
9.22 Combined Error-Control Relationship	145
9.23 Importance of Correct Sign Convention	146
9.24 Transformation and Error Convergence	146
9.25 Complete Mathematical Flow	147
9.26 Overall Signal Flow	148
9.27 Summary	149
10 Nonlinear Kinematic Controller	151
10.1 Introduction	151
10.2 Purpose of the Kinematic Controller	152
10.3 Inputs to the Controller	153
10.4 Outputs of the Controller	154
10.5 Feedback and Feedforward Control	154
10.6 Linear Velocity Controller	155
10.7 Forward-Error Feedback Term	155
10.8 Feedforward Linear Velocity Term	156
10.9 Complete Linear Velocity Equation	157
10.10 Angular Velocity Controller	157
10.11 Heading-Error Correction	158
10.12 Lateral-Error Correction	158

10.13	Desired Angular Velocity Feedforward	159
10.14	Complete Angular Velocity Equation	159
10.15	Interpretation of the Controller	160
10.16	Controller Operation When Errors Are Zero	160
10.17	Controller Operation With Position Error	161
10.18	Controller Operation With Orientation Error	162
10.19	Role of Nonlinearity	162
10.20	Controller Gains	163
10.21	Controller Block Inputs and Outputs	163
10.22	Velocity Saturation After the Controller	164
10.23	Complete Kinematic-Control Structure	164
10.24	Relationship With Dynamic Controller	165
10.25	Controller Algorithm	166
10.26	Mathematical Summary of the Controller	167
10.27	Overall Control-Law Flow	167
10.28	Expected Controller Behaviour	169
10.29	Importance of the Kinematic Controller	170
10.30	Summary	170
11	Velocity Saturation	172
11.1	Need for Saturation	172
11.2	Linear Velocity Limit	173
11.3	Angular Velocity Limit	174
11.4	Saturation Function	175
11.5	Effect of Saturation	176
11.5.1	Safety	176
11.5.2	Actuator Limitations	177
11.5.3	Prevention of Unrealistic Commands	177
11.5.4	Effect on Tracking Performance	178
11.6	Saturation During the Circular Trajectory	178
11.7	Linear Velocity Operating Margin	179
11.8	Angular Velocity Operating Margin	180
11.9	Comparison of Velocity Margins	180

11.10Importance of Saturation in the Proposed Controller	181
11.11Effect of Saturation on Transient Response	181
11.12Relationship Between Saturation and Observed Trajectory	182
11.13Practical Significance	183
11.14Summary	183
12 Dynamic Velocity Controller	185
12.1 Introduction	185
12.2 Need for Dynamic Velocity Control	186
12.3 Dynamic Controller Inputs	187
12.4 Linear Velocity Error	187
12.5 Angular Velocity Error	188
12.6 Complete Velocity Error Vector	189
12.7 Dynamic Control Structure	189
12.8 Relationship Between Kinematic and Dynamic Controllers	190
12.9 Inner and Outer Control Loops	191
12.9.1 Outer Loop	191
12.9.2 Inner Loop	191
12.10Dynamic Model of Linear Motion	192
12.11Dynamic Model of Rotational Motion	192
12.12Linear Velocity Control	193
12.13Angular Velocity Control	193
12.14Combined Dynamic Control	193
12.15Velocity Tracking	194
12.16Interaction With the Robot Model	195
12.17Importance of Feedback	195
12.18Velocity Saturation	196
12.19Why Velocity Saturation Is Required	197
12.20Dynamic Controller and Circular Trajectory	197
12.21Complete Dynamic Control Architecture	198
12.22Overall Closed-Loop System	199
12.23Algorithm for Dynamic Velocity Control	201
12.24Expected Dynamic Controller Response	202

12.25	Importance of the Dynamic Controller in Trajectory Tracking	203
12.26	Controller Performance Parameters	203
12.26.1	Linear Velocity Tracking Error	203
12.26.2	Angular Velocity Tracking Error	204
12.26.3	Maximum Linear Velocity Error	204
12.26.4	Maximum Angular Velocity Error	204
12.26.5	Linear Velocity RMSE	204
12.26.6	Angular Velocity RMSE	204
12.27	Relation Between Velocity Error and Trajectory Error	204
12.28	Summary	205
13	Simulation Setup	207
13.1	Simulation Environment	207
13.2	Robot Parameters	207
13.3	Controller Parameters	208
13.4	Trajectory Parameters	208
13.5	Velocity Limits	209
13.6	Simulation Configuration Summary	210
13.7	Summary	210
14	Simulation and Implementation	212
14.1	Introduction	212
14.2	Simulation Objective	213
14.3	Simulation Environment	214
14.4	Overall Simulink Model	214
14.5	Reference Trajectory Configuration	216
14.6	Initial Condition	217
14.7	Odometry Feedback	217
14.8	Tracking-Error Calculation	218
14.9	Global-to-Robot Transformation	218
14.10	Nonlinear Kinematic Controller	219
14.11	Velocity Saturation	220
14.12	Dynamic Velocity Control	220

14.13 Robot Motion Model	221
14.14 Closed-Loop Simulation	221
14.15 Simulation Time	223
14.16 Signals to Be Observed	223
14.17 Desired Versus Actual Trajectory	224
14.18 Position Error Results	225
14.19 Orientation Error Results	226
14.20 Velocity Tracking Results	226
14.21 Position Tracking Performance	227
14.22 RMSE Analysis	227
14.23 Velocity RMSE	228
14.24 Expected Simulation Behaviour	228
14.25 Effect of Controller Gains	229
14.26 Simulation Procedure	229
14.27 Complete Simulation Flowchart	231
14.28 Implementation of ROS 2 Communication	232
14.29 Implementation of Feedback Loop	233
14.30 Simulation Results to Be Included	233
14.31 Performance Evaluation Table	236
14.32 Discussion of Simulation Results	236
14.33 Overall Simulation Architecture	237
14.34 Summary	239
15 Simulation Results and Performance Analysis	241
15.1 Introduction	241
15.2 Simulation Parameters	242
15.3 Theoretical Circular Trajectory	243
15.4 Circumference of the Reference Trajectory	244
15.5 Time Required for One Complete Revolution	244
15.6 Verification of Desired Linear Velocity	245
15.7 Simulation X - Y Trajectory	246
15.8 Reference Trajectory Analysis	247
15.9 Analysis of the Simulated Trajectory	248

15.10	Maximum Relative Radial Deviation	248
15.11	Approximate Maximum and Minimum Actual Coordinates	250
15.12	Approximate Deviation in the X -Direction	251
15.13	Approximate Deviation in the Negative X -Direction	251
15.14	Approximate Deviation in the Positive Y -Direction	252
15.15	Approximate Deviation in the Negative Y -Direction	253
15.16	Summary of Graph-Based Deviations	254
15.17	Approximate Maximum Radial Error	254
15.18	Interpretation of the Maximum Error	255
15.19	Analysis of the Initial Response	256
15.20	Upper-Region Tracking Behaviour	256
15.21	Lower-Region Tracking Behaviour	257
15.22	Side-Region Tracking Behaviour	257
15.23	Overall Trajectory Performance	258
15.24	Kinematic Controller Contribution	258
15.25	Dynamic Controller Contribution	259
15.26	Velocity Saturation Analysis	260
15.27	Angular Velocity Margin	261
15.28	Reference Trajectory Accuracy	261
15.29	Approximate Trajectory Envelope	262
15.30	Percentage Increase in Trajectory Envelope	263
15.31	Performance Summary From the X - Y Graph	263
15.32	Results Table	265
15.33	Important Note Regarding Error Calculation	265
15.34	Discussion of Simulation Performance	266
15.35	Interpretation of the Result	267
15.36	Overall Control-System Assessment	267
15.37	Conclusion of Simulation Analysis	267
15.38	Final Summary	269
16	Discussion	271
16.1	Introduction	271
16.2	Trajectory Tracking Performance	272

16.3 Position Tracking	272
16.4 Orientation Tracking	274
16.5 Error Convergence	275
16.6 Controller Performance	276
16.7 Contribution of Feedback Terms	276
16.8 Contribution of Feedforward Terms	277
16.9 Coordinate Transformation	278
16.10 Why Robot-Frame Error Improves Control Action	278
16.11 Effect of Coordinate Transformation on Circular Tracking	279
16.12 Velocity Saturation	279
16.13 Effect of Velocity Saturation on the Response	280
16.14 Dynamic Controller	281
16.15 Effect of the Inner Velocity Loop	282
16.16 Interaction Between the Outer and Inner Loops	282
16.16.1 Outer Loop	282
16.16.2 Inner Loop	283
16.17 Relationship Between the Controller and the Observed Result	283
16.18 Overall Discussion of the Simulation	284
16.19 Strengths of the Proposed Control Structure	284
16.19.1 Closed-Loop Correction	284
16.19.2 Robot-Frame Error Processing	284
16.19.3 Combined Feedback and Feedforward Control	285
16.19.4 Velocity Constraints	285
16.19.5 Inner Dynamic Control	285
16.20 Limitations Observed in the Simulation	285
16.21 Scope for Controller Improvement	285
16.22 Overall Performance Assessment	286
16.23 Conclusion	287
17 Advantages and Limitations	289
17.1 Introduction	289
17.2 Advantages	289
17.2.1 Closed-Loop Trajectory Tracking	289

17.2.2	ROS 2 Integration	290
17.2.3	Simulink-Based Implementation	290
17.2.4	Nonlinear Controller	291
17.2.5	Feedback and Feedforward Control	291
17.2.6	Robot-Frame Error Correction	292
17.2.7	Position and Orientation Control	292
17.2.8	Velocity Saturation	293
17.2.9	Modular Control Architecture	294
17.2.10	Possibility of Real-Time Implementation	294
17.3	Limitations	294
17.3.1	Odometry Errors	294
17.3.2	Sensor Noise	295
17.3.3	ROS 2 Communication Delay	295
17.3.4	Model Uncertainty	296
17.3.5	Controller-Gain Dependence	296
17.3.6	Actuator Limitations	297
17.3.7	Velocity Saturation	298
17.3.8	Simulation-to-Real-World Differences	298
17.4	Relationship Between Advantages and Limitations	299
17.5	Advantages and Limitations Summary	300
17.6	Overall Assessment	300
17.7	Conclusion	301
18	Future Scope	302
18.1	Introduction	302
18.2	Physical Robot Implementation	302
18.3	Obstacle Avoidance	303
18.4	More Complex Trajectories	303
18.4.1	Figure-Eight Trajectory	304
18.4.2	S-Shaped Trajectory	304
18.4.3	Spiral Trajectory	304
18.4.4	Waypoint Trajectory	304
18.5	Sensor Fusion	304

18.5.1	Odometry	305
18.5.2	IMU	305
18.5.3	LiDAR	305
18.5.4	Camera	305
18.6	Adaptive Control	305
18.7	Multi-Robot Coordination	306
18.8	Robust Control	307
18.8.1	Disturbances	307
18.8.2	Noise	307
18.8.3	Model Uncertainty	307
18.8.4	Communication Delays	308
18.9	Robustness Against Disturbances	308
18.10	Robustness Against Noise	308
18.11	Robustness Against Model Uncertainty	309
18.12	Communication Delay Analysis	309
18.13	Improvement of Trajectory Accuracy	310
18.14	Real-Time Optimization	311
18.15	Integration With Autonomous Navigation	311
18.16	Future Scope Summary	311
18.17	Conclusion	312
19	Conclusion	314
19.1	Introduction	314
19.2	Summary of the Complete Work	314
19.2.1	TurtleBot3 Trajectory Tracking	314
19.2.2	ROS 2 Communication	315
19.2.3	Odometry-Based State Acquisition	315
19.2.4	Quaternion-to-Yaw Conversion	315
19.2.5	Circular Reference Trajectory Generation	315
19.2.6	Tracking-Error Calculation	316
19.2.7	Transformation Into the Robot Frame	316
19.2.8	Nonlinear Kinematic Controller	317
19.2.9	Dynamic Velocity Controller	317

19.2.10 Velocity Saturation	318
19.2.11 Simulink Implementation	318
19.3 Simulation Results	319
19.4 Overall Performance	319
19.5 Main Contributions of the Project	320
19.6 Final Conclusion	321
20 References	323
20.1 TurtleBot3 Documentation	323

Introduction

1.1 Background

The rapid development of robotics, automation, and intelligent control systems has significantly increased the use of autonomous mobile robots in research and industrial applications. Mobile robots are capable of moving through an environment and performing tasks without continuous human intervention. For such robots, **trajectory tracking** is one of the fundamental control problems. Trajectory tracking refers to the ability of a robot to follow a predefined path while maintaining the desired position and orientation over time.

Accurate trajectory tracking is essential in applications such as autonomous navigation, warehouse automation, surveillance, transportation, inspection, and service robotics. During trajectory tracking, the robot continuously compares its actual state with the desired state and generates appropriate control commands to minimize the resulting error. Therefore, the performance of the trajectory-tracking controller directly affects the accuracy, stability, and reliability of the robotic system.

In this project, **TurtleBot3** is considered as the mobile robotic platform for implementing and studying trajectory tracking. TurtleBot3 is a compact differential-drive mobile robot that is widely used for robotics research, education, simulation, and experimentation. Its differential-drive configuration allows the robot to move forward and backward and change its orientation by controlling the motion of its wheels.

The motion of the robot can be described using its position and orientation. The position of the robot in a two-dimensional plane is represented by x and y , while its orientation is represented by the yaw angle ϕ . Hence, the robot pose can be expressed as

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix}. \quad (1.1)$$

Robot Pose:

The state of the planar mobile robot is represented by the position coordinates x , y and the orientation ϕ .

For effective trajectory tracking, these three quantities must be continuously monitored and compared with their desired values.

1.2 ROS 2 and Simulink Integration

A major component of the proposed system is the integration of **Robot Operating System 2 (ROS 2)** with **MATLAB/Simulink**.

ROS 2 provides a communication framework that allows different components of a robotic system to exchange information. In the present system, the robot publishes its state information through ROS 2, while Simulink receives this information and performs the required control calculations.

The robot's odometry information is communicated through the ROS 2 `/odom` topic. The odometry contains information about the robot's position and orientation. The ROS 2 Subscriber block in Simulink receives this information and makes it available to the controller.

The communication process can be represented as

$$\boxed{\text{TurtleBot3/Gazebo} \rightarrow \text{ROS 2} \rightarrow \text{/odom} \rightarrow \text{Simulink}} \quad (1.2)$$

The controller then processes the received state information and generates the required velocity commands, which are subsequently sent back toward the robot through the ROS 2 communication system.

Closed-Loop Communication:

The robot continuously publishes its state through ROS 2. Simulink receives this information, calculates the required control commands, and sends the resulting commands back to the robot.

This creates a **closed-loop control system**, where the actual robot motion

is continuously fed back to the controller.

1.3 Motivation

The motivation for this project comes from the requirement to accurately control the motion of a mobile robot along a predefined trajectory.

If a robot is commanded to follow a trajectory without feedback, any difference between the expected and actual robot motion can accumulate over time. Such deviations may occur because of modelling errors, actuator limitations, disturbances, initial-condition differences, or changes in robot orientation.

Therefore, a feedback-based controller is required. The basic principle is

$$\boxed{\text{Desired State} - \text{Actual State} = \text{Tracking Error}} \quad (1.3)$$

The controller uses this error to determine the appropriate corrective action.

For a mobile robot, the problem is more complicated because position and orientation are coupled. For example, if the desired position lies to the left of the robot, the robot cannot simply move directly sideways. Instead, it must modify its orientation and forward motion to reach the desired position.

Therefore, the control system must consider:

- longitudinal position error,
- lateral position error,
- orientation error,
- desired linear velocity, and
- desired angular velocity.

The project therefore investigates a nonlinear feedback-feedforward control approach in which these quantities are processed systematically to generate the required robot commands.

1.4 Problem Statement

The main problem addressed in this project is the **trajectory tracking of a differential-drive TurtleBot3 using ROS 2 and MATLAB/Simulink**.

The robot is required to follow a predefined circular trajectory. The actual position and orientation of the robot are obtained from odometry. These actual states are compared with the corresponding desired states generated by the reference trajectory.

The tracking errors are then transformed from the global coordinate frame into the robot's local coordinate frame. This transformation allows the controller to determine whether the desired trajectory is ahead, behind, or laterally displaced relative to the robot.

The controller then generates

$$v_c \tag{1.4}$$

and

$$\omega_c, \tag{1.5}$$

where:

- v_c is the commanded linear velocity,
- ω_c is the commanded angular velocity.

These commands are passed through the subsequent control stage and ultimately used to control the robot's motion.

The overall problem can therefore be expressed as

$$\boxed{\text{Track the desired trajectory while minimizing } e_x, e_y, e_\phi} \tag{1.6}$$

where

$$e_x = x_d - x, \tag{1.7}$$

$$e_y = y_d - y, \tag{1.8}$$

and

$$e_\phi = \phi_d - \phi. \tag{1.9}$$

The objective is to make these errors sufficiently small so that the actual robot trajectory closely follows the desired trajectory.

1.5 Need for Trajectory Tracking

Trajectory tracking is different from simply moving a robot from one point to another.

In point-to-point control, the primary objective is to reach a specified final position. In trajectory tracking, however, the robot must continuously follow a **time-dependent reference**.

For example, in this project the robot is required to follow a circular path. Therefore, the desired position changes continuously:

$$x_d = x_d(t), \quad (1.10)$$

$$y_d = y_d(t), \quad (1.11)$$

and the desired orientation also changes:

$$\phi_d = \phi_d(t). \quad (1.12)$$

The controller must therefore continuously update the velocity commands according to the instantaneous tracking error.

A successful trajectory-tracking system should satisfy the following requirements:

1. The robot should remain close to the desired trajectory.
2. Position errors should be minimized.
3. Orientation error should be minimized.
4. The robot should respond smoothly to changes in the reference.
5. The commanded velocities should remain within the permissible limits.
6. The controller should remain stable during operation.

1.6 Proposed Approach

The proposed system uses a combination of **ROS 2, MATLAB/Simulink, odometry feedback, trajectory generation, coordinate transformation, nonlinear kinematic control, and dynamic velocity control**.

The complete methodology consists of several sequential stages.

1.6.1 Step 1: Obtain Robot State

The robot publishes odometry information through ROS 2.

The required position information is obtained as

$$x_{\text{real}}, \quad y_{\text{real}}. \quad (1.13)$$

1.6.2 Step 2: Obtain Robot Orientation

The robot orientation is received in quaternion form:

$$q = [q_x, q_y, q_z, q_w]. \quad (1.14)$$

The quaternion is converted into yaw using the quaternion-to-Euler conversion block. The yaw angle is represented by

$$\phi. \quad (1.15)$$

The project material specifically identifies quaternion-to-yaw conversion as part of the proposed system.

1.6.3 Step 3: Generate Reference Trajectory

A circular trajectory is generated according to

$$x_d = R \cos(\omega t), \quad (1.16)$$

$$y_d = R \sin(\omega t), \quad (1.17)$$

$$\phi_d = \omega t. \quad (1.18)$$

The project uses

$$R = 0.4 \text{ m} \quad (1.19)$$

and

$$\omega = 0.5 \text{ rad/s}. \quad (1.20)$$

1.6.4 Step 4: Calculate Tracking Errors

The desired and actual states are compared:

$$e_x = x_d - x, \quad (1.21)$$

$$e_y = y_d - y, \quad (1.22)$$

$$e_\phi = \phi_d - \phi. \quad (1.23)$$

1.6.5 Step 5: Transform Errors

The global-frame position errors are transformed into the robot coordinate frame:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (1.24)$$

and

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (1.25)$$

The orientation error is retained as

$$\phi_{r,\text{dif}} = e_\phi. \quad (1.26)$$

This transformation enables the controller to work with errors relative to the robot's own orientation.

1.6.6 Step 6: Apply Kinematic Control

The transformed errors are supplied to the nonlinear kinematic controller.

The commanded linear velocity is

$$v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \quad (1.27)$$

and the commanded angular velocity is

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d. \quad (1.28)$$

The controller therefore combines feedback and feedforward components.

1.6.7 Step 7: Apply Velocity Limits

The generated commands are limited to the specified operating range:

$$-0.22 \leq v_c \leq 0.22 \quad (1.29)$$

and

$$-2.5 \leq \omega_c \leq 2.5. \quad (1.30)$$

This prevents the controller from producing commands beyond the assumed robot operating limits.

1.6.8 Step 8: Dynamic Velocity Control

The commanded linear and angular velocities are passed to the dynamic controller.

The dynamic controller considers:

$$v_c, \quad \omega_c, \quad \dot{v}_c, \quad \dot{\omega}_c \quad (1.31)$$

and generates the corresponding actuation-related quantities.

The dynamic controller uses the gains

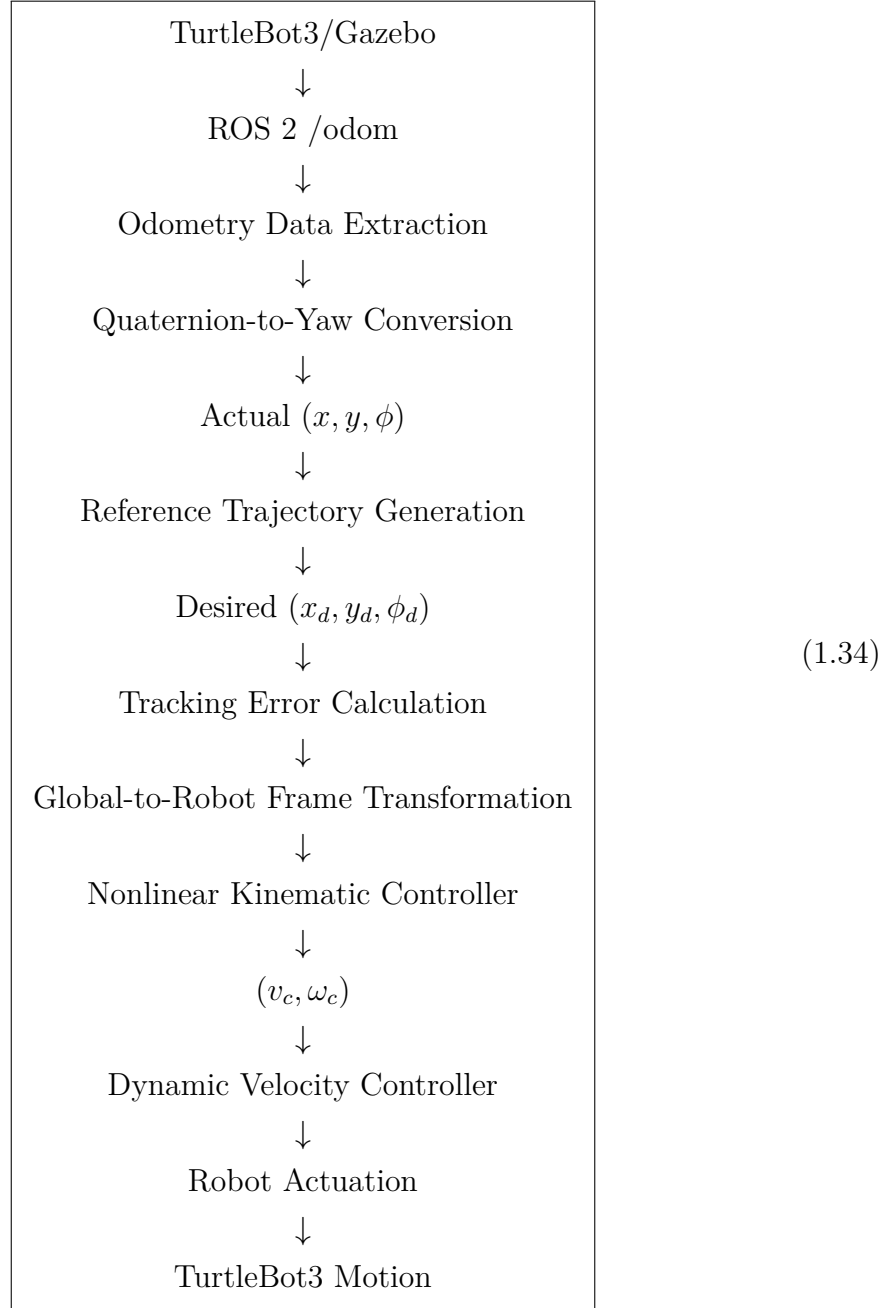
$$K_a = 25 \quad (1.32)$$

and

$$K_b = 8. \quad (1.33)$$

1.7 Project Methodology

The complete methodology followed in the project can be summarized as:



This architecture forms a closed-loop system because the robot's actual state is continuously measured and returned to the controller.

1.8 Scope of the Project

The scope of this project is focused on the development and simulation of a trajectory-tracking control system for TurtleBot3 using ROS 2 and MATLAB/Simulink.

The work includes:

- TurtleBot3 modelling and motion control.

- ROS 2 communication.
- Odometry acquisition.
- Position-state extraction.
- Quaternion orientation processing.
- Yaw-angle calculation.
- Circular reference trajectory generation.
- Position and orientation error calculation.
- Coordinate transformation.
- Nonlinear kinematic control.
- Dynamic velocity control.
- Velocity saturation.
- Simulink implementation.
- Simulation-based performance evaluation.

The project primarily considers **planar motion** and a circular reference trajectory. More advanced functions such as obstacle avoidance, autonomous path planning, and complex multi-robot coordination are outside the present scope unless they are specifically implemented as extensions.

1.9 Significance of the Project

The project demonstrates how a modern robotics software framework and a model-based control environment can be combined to develop a complete robotic control system.

ROS 2 provides the communication infrastructure required for exchanging information between the robot and the control system, while Simulink provides a convenient environment for modelling, controller implementation, and simulation.

The project is also significant from a control perspective because it demonstrates the complete process of converting robot-state information into a meaningful control action:

$$\boxed{\text{Measured State} \rightarrow \text{Error} \rightarrow \text{Control Law} \rightarrow \text{Velocity Command}} \quad (1.35)$$

The use of robot-frame errors, feedback terms, feedforward terms, and dynamic velocity control allows the system to address both positional and orientation requirements during trajectory tracking.

Objectives

2.1 Introduction

The primary objective of this project is to develop and implement a **trajectory-tracking control system for TurtleBot3 using ROS 2 and MATLAB/Simulink**. The developed system is intended to enable the robot to follow a predefined trajectory accurately by continuously comparing its actual position and orientation with the desired trajectory and generating appropriate control commands.

The project combines **robot-state feedback, reference trajectory generation, nonlinear kinematic control, dynamic velocity control, and ROS 2 communication** into a single closed-loop control architecture.

The overall objective can be represented as:

$$\text{Desired Trajectory} \rightarrow \text{Tracking Error} \rightarrow \text{Controller} \rightarrow \text{Velocity Command} \rightarrow \text{TurtleBot3} \quad (2.1)$$

The actual robot state is continuously fed back to the controller through the ROS 2 odometry system, thereby completing the closed-loop control process.

Overall Objective:

To develop a complete closed-loop trajectory-tracking system in which TurtleBot3 receives a desired trajectory, calculates the tracking error, generates corrective velocity commands, and continuously uses its actual state as feedback.

2.2 Main Objective

The main objective of the project is:

To design and implement a ROS 2 and MATLAB/Simulink-based nonlinear trajectory-tracking control system for TurtleBot3 that enables the robot to accurately follow a predefined circular trajectory using position and orientation feedback.

The system should obtain the actual robot state, generate a desired trajectory, calculate the tracking errors, transform these errors into the robot coordinate frame, and use them to generate appropriate linear and angular velocity commands.

The complete objective can be expressed as:

$$\boxed{(x(t), y(t), \phi(t)) \rightarrow (x_d(t), y_d(t), \phi_d(t))} \quad (2.2)$$

such that

$$x(t) \rightarrow x_d(t), \quad (2.3)$$

$$y(t) \rightarrow y_d(t), \quad (2.4)$$

and

$$\phi(t) \rightarrow \phi_d(t). \quad (2.5)$$

Therefore, the controller aims to minimize:

$$e_x = x_d - x, \quad (2.6)$$

$$e_y = y_d - y, \quad (2.7)$$

$$e_\phi = \phi_d - \phi. \quad (2.8)$$

2.3 Specific Objectives

The project is divided into several specific objectives to achieve the overall trajectory-tracking goal.

2.3.1 Establish ROS 2–Simulink Communication

The first objective is to establish communication between the TurtleBot3 robotic system and MATLAB/Simulink through **ROS 2**.

The communication system allows robot-state information to be transferred from the robot to Simulink, while the calculated control commands can be transmitted back to the robot.

The basic communication structure is:

$$\boxed{\text{TurtleBot3/Gazebo} \rightarrow \text{ROS 2} \rightarrow \text{Simulink}} \quad (2.9)$$

The control-command path is:

$$\boxed{\text{Simulink} \rightarrow \text{ROS 2} \rightarrow \text{TurtleBot3}} \quad (2.10)$$

The project material identifies ROS 2–Simulink communication as the first major objective of the system.

2.3.2 Acquire Robot Odometry

The second objective is to obtain the actual state of TurtleBot3 using odometry information.

The robot publishes its state through the ROS 2 `/odom` topic. The subscriber block in Simulink receives this information and extracts the required variables.

The primary position variables are:

$$x_{\text{real}} \quad (2.11)$$

and

$$y_{\text{real}}. \quad (2.12)$$

These represent the actual position of the robot in the global coordinate system.

The odometry information is essential because the controller needs the actual robot state to determine how far the robot has deviated from the desired trajectory.

Thus,

$$\boxed{\text{Odometry} \rightarrow \text{Actual Robot State} \rightarrow \text{Feedback}} \quad (2.13)$$

2.3.3 Obtain Robot Orientation

In addition to position, the controller requires the robot's orientation.

The orientation information received through ROS 2 is represented using a quaternion:

$$q = [q_x, q_y, q_z, q_w]. \quad (2.14)$$

The project therefore includes a quaternion-processing stage to obtain the robot's heading.

The quaternion representation is converted into Euler-angle information using the QUAT2ZYX block. For the planar TurtleBot3 trajectory-tracking problem, the required orientation variable is the **yaw angle**:

$$\phi. \quad (2.15)$$

Therefore, the orientation-processing chain is:

$$\boxed{[q_x, q_y, q_z, q_w] \rightarrow \text{QUAT2ZYX} \rightarrow \phi} \quad (2.16)$$

The supplied material identifies quaternion-to-yaw conversion as one of the main objectives of the project.

2.4 Generate the Desired Trajectory

Another major objective is to generate a reference trajectory for TurtleBot3.

For the present project, a **circular trajectory** is selected.

The desired trajectory is defined by:

$$x_d = R \cos(\omega t) \quad (2.17)$$

$$y_d = R \sin(\omega t) \quad (2.18)$$

$$\phi_d = \omega t. \quad (2.19)$$

The trajectory parameters used in the supplied project material are:

$$R = 0.4 \text{ m} \quad (2.20)$$

and

$$\omega = 0.5 \text{ rad/s.} \quad (2.21)$$

The objective is therefore not simply to move the robot toward a fixed point, but to continuously follow a reference whose position and orientation vary with time.

Time-Varying Reference $\rightarrow$ Continuous Trajectory Tracking

(2.22)

2.5 Synchronize the Initial Reference With the Robot

The desired trajectory should begin consistently with the robot's actual initial state.

Therefore, at the beginning of the simulation,

$$x_d(0) = x_{\text{real}}(0), \quad (2.23)$$

$$y_d(0) = y_{\text{real}}(0), \quad (2.24)$$

and

$$\phi_d(0) = \phi_{\text{real}}(0). \quad (2.25)$$

This ensures that the reference trajectory begins from the robot's initial location and orientation.

The supplied material describes this initial-condition approach for the trajectory generator.

Initial Condition Synchronization:

The reference trajectory is initialized using the actual robot state so that the desired and actual states are consistently aligned at the beginning of the simulation.

2.6 Calculate Position and Orientation Tracking Errors

Once the desired and actual states are available, the next objective is to calculate the tracking errors.

The position errors are defined as:

$$e_x = x_d - x \quad (2.26)$$

and

$$e_y = y_d - y. \quad (2.27)$$

The orientation error is:

$$e_\phi = \phi_d - \phi. \quad (2.28)$$

Therefore,

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (2.29)$$

represents the difference between the desired robot state and the actual robot state.

The supplied material defines the corresponding differences as x_{dif} , y_{dif} , and ϕ_{dif} .

The objective is to minimize these errors during the entire trajectory.

2.7 Transform the Tracking Error Into the Robot Frame

The position errors obtained above are initially expressed in the global coordinate frame. However, the controller needs to know the error relative to the robot's current orientation.

Therefore, another objective is to transform the global-frame errors into the robot/body frame.

The transformed errors are:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (2.30)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (2.31)$$

and

$$\phi_{r,\text{dif}} = e_\phi. \quad (2.32)$$

Here,

- $x_{r,\text{dif}}$ represents the forward/backward error,
- $y_{r,\text{dif}}$ represents the lateral error,
- $\phi_{r,\text{dif}}$ represents the heading error.

This transformation allows the controller to determine whether the desired trajectory lies ahead, behind, or laterally away from the robot.

The project material explicitly identifies global-to-robot frame transformation as part of the trajectory-tracking methodology.

2.8 Develop the Nonlinear Kinematic Controller

The next objective is to design the nonlinear kinematic controller responsible for generating the desired robot velocities.

The controller receives:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}, \quad v_d, \quad \omega_d. \quad (2.33)$$

It generates:

$$v_c \quad (2.34)$$

and

$$\omega_c. \quad (2.35)$$

The commanded linear velocity is:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (2.36)$$

while the commanded angular velocity is:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (2.37)$$

The supplied project material identifies these controller equations and their respective feedback and feedforward components.

2.9 Implement Feedback and Feedforward Control

The controller is designed using both **feedback** and **feedforward** actions.

2.9.1 Feedback Control

The feedback terms respond to the current tracking errors:

$$k_x x_{r,\text{dif}} \quad (2.38)$$

and

$$k_\phi \sin(\phi_{r,\text{dif}}). \quad (2.39)$$

These terms correct deviations between the actual and desired robot states.

2.9.2 Feedforward Control

The feedforward terms are:

$$v_d \cos(\phi_{r,\text{dif}}) \quad (2.40)$$

and

$$\omega_d. \quad (2.41)$$

These terms provide information about the desired motion directly to the controller.

Therefore, the objective is to combine:

$$\boxed{\text{Feedback Correction} + \text{Feedforward Motion}} \quad (2.42)$$

to improve trajectory-tracking performance.

2.10 Control Lateral Tracking Error

A specific objective of the angular-velocity controller is to correct lateral trajectory error.

The lateral correction term is:

$$45v_d y_{r,\text{dif}}. \quad (2.43)$$

If the robot moves away from the desired path in the lateral direction, this term produces an appropriate angular correction to bring the robot back toward the trajectory.

Thus,

$$\boxed{\text{Lateral Error} \rightarrow \text{Angular Correction} \rightarrow \text{Trajectory Recovery}} \quad (2.44)$$

The supplied material explains that this term provides stronger lateral correction as the robot's desired velocity increases.

2.11 Apply Velocity Constraints

The controller should not produce arbitrary velocity commands.

Therefore, another objective is to apply saturation limits to the commanded linear and angular velocities.

The linear velocity is restricted to:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (2.45)$$

and the angular velocity is restricted to:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (2.46)$$

These limits ensure that the generated commands remain within the specified

operating range of the robot.

$$\boxed{\text{Controller Output} \rightarrow \text{Saturation} \rightarrow \text{Permissible Velocity}} \quad (2.47)$$

2.12 Develop the Dynamic Velocity Controller

The next objective is to implement the dynamic velocity-control layer.

The kinematic controller produces:

$$v_c, \quad \omega_c. \quad (2.48)$$

These quantities are then supplied to the dynamic controller along with their derivatives:

$$\dot{v}_c, \quad \dot{\omega}_c. \quad (2.49)$$

The dynamic controller generates the corresponding force and torque-related commands.

The controller uses the gains:

$$K_a = 25 \quad (2.50)$$

and

$$K_b = 8. \quad (2.51)$$

The supplied material identifies this as the inner-loop dynamic velocity controller.

The objective of this stage is to ensure that the robot's actual velocity response follows the desired velocity commands generated by the kinematic controller.

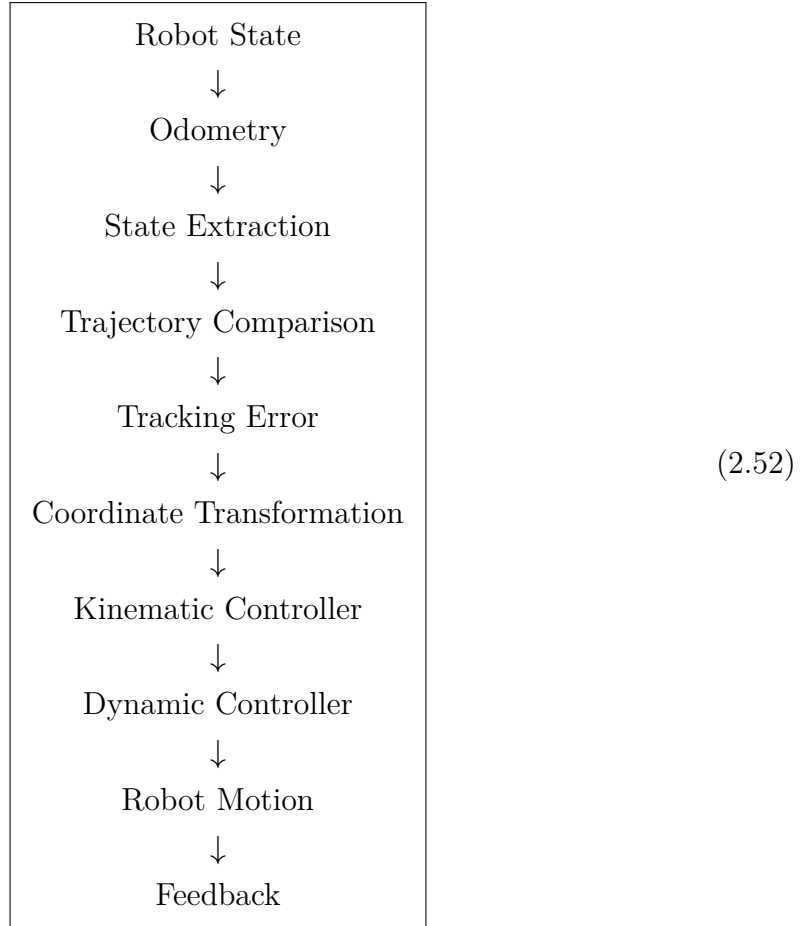
Dynamic Control Objective:

The dynamic velocity controller acts as an inner control layer that processes the commanded velocities and their derivatives to produce the corresponding actuation-related commands.

2.13 Implement the Complete Closed-Loop System

All the individual components must operate together as one closed-loop control system.

The complete objective is therefore:



This feedback loop continuously updates the control command based on the robot's actual motion.

Closed-Loop Objective:

The actual robot state is continuously measured and returned to the controller so that corrective actions can be generated throughout the trajectory.

2.14 Evaluate Trajectory-Tracking Performance

The final objective is to evaluate the performance of the complete system.

The following quantities should be investigated.

2.14.1 Position Tracking

The actual and desired x positions are compared using:

$$x(t) \quad \text{vs} \quad x_d(t) \quad (2.53)$$

Similarly, the y positions are compared using:

$$y(t) \quad \text{vs} \quad y_d(t) \quad (2.54)$$

2.14.2 Orientation Tracking

The actual and desired orientations are compared using:

$$\phi(t) \quad \text{vs} \quad \phi_d(t) \quad (2.55)$$

2.14.3 Tracking Errors

The position and orientation tracking errors are evaluated using:

$$e_x(t) \quad (2.56)$$

$$e_y(t) \quad (2.57)$$

and

$$e_\phi(t). \quad (2.58)$$

2.14.4 Velocity Response

The commanded linear velocity is evaluated using:

$$v_c(t) \quad (2.59)$$

and the commanded angular velocity is evaluated using:

$$\omega_c(t). \quad (2.60)$$

2.14.5 Overall Trajectory

The actual robot trajectory is compared with the desired trajectory:

$$(x(t), y(t)) \quad (2.61)$$

with

$$(x_d(t), y_d(t)). \quad (2.62)$$

The purpose is to determine whether the robot follows the desired circular trajectory with acceptable accuracy.

2.15 Performance Evaluation Parameters

To quantitatively evaluate the system, the following performance measures can be included in the results chapter.

2.15.1 Maximum Tracking Error

The maximum tracking error can be expressed as:

$$e_{\max} = \max |e(t)|. \quad (2.63)$$

2.15.2 Root Mean Square Error

For the x -position:

$$RMSE_x = \sqrt{\frac{1}{N} \sum_{k=1}^N (x_d[k] - x[k])^2} \quad (2.64)$$

Similarly,

$$RMSE_y = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_d[k] - y[k])^2} \quad (2.65)$$

and

$$RMSE_{\phi} = \sqrt{\frac{1}{N} \sum_{k=1}^N (\phi_d[k] - \phi[k])^2}. \quad (2.66)$$

These measures provide an objective assessment of the trajectory-tracking accuracy.

2.16 Overall Objective

The overall objective of the project can finally be represented as:

$$\boxed{\text{Accurate TurtleBot3 Trajectory Tracking}} \quad (2.67)$$

through the integration of

$$\boxed{\begin{array}{l} \text{ROS 2 + Odometry + Simulink + Trajectory Generation} \\ \text{+ Error Transformation + Nonlinear Kinematic Control + Dynamic Velocity Control} \end{array}} \quad (2.68)$$

with the ultimate goal of achieving

$$\boxed{e_x \rightarrow 0, \quad e_y \rightarrow 0, \quad e_{\phi} \rightarrow 0} \quad (2.69)$$

while the robot follows the desired circular trajectory.

Final Objective

Accurately track the desired circular trajectory by integrating ROS 2 communication, odometry feedback, trajectory generation, coordinate transformation, nonlinear kinematic control, velocity saturation, and dynamic velocity control into a complete closed-loop TurtleBot3 control system.

This chapter therefore establishes the **specific technical objectives** that will be addressed in the subsequent chapters on robot modelling, ROS 2 communication, trajectory generation, controller design, Simulink implementation, and simulation results.

System Overview

3.1 Introduction

The proposed project develops a **trajectory-tracking control system for TurtleBot3 using ROS 2 and MATLAB/Simulink**. The purpose of the system is to enable TurtleBot3 to follow a predefined trajectory while continuously correcting its position and orientation errors.

The complete system is implemented as a **closed-loop control system**. The actual state of the robot is obtained through odometry and supplied to Simulink. Simulink processes this information, compares it with the desired trajectory, calculates the tracking errors, and generates appropriate linear and angular velocity commands. These commands are then sent back to TurtleBot3 through ROS 2.

The major functions of the system include ROS 2–Simulink communication, odometry-based state acquisition, quaternion-to-yaw conversion, circular trajectory generation, tracking-error calculation, nonlinear kinematic control, dynamic velocity control, and velocity-command publication.

The overall system can be represented as:

$$\boxed{\text{TurtleBot3} \rightarrow \text{ROS 2} \rightarrow \text{Simulink} \rightarrow \text{Controller} \rightarrow \text{ROS 2} \rightarrow \text{TurtleBot3}} \quad (3.1)$$

System Objective:

The complete system is designed to continuously measure the robot state, compare it with the desired trajectory, calculate the tracking error, and generate corrective velocity commands.

3.2 Overall System Architecture

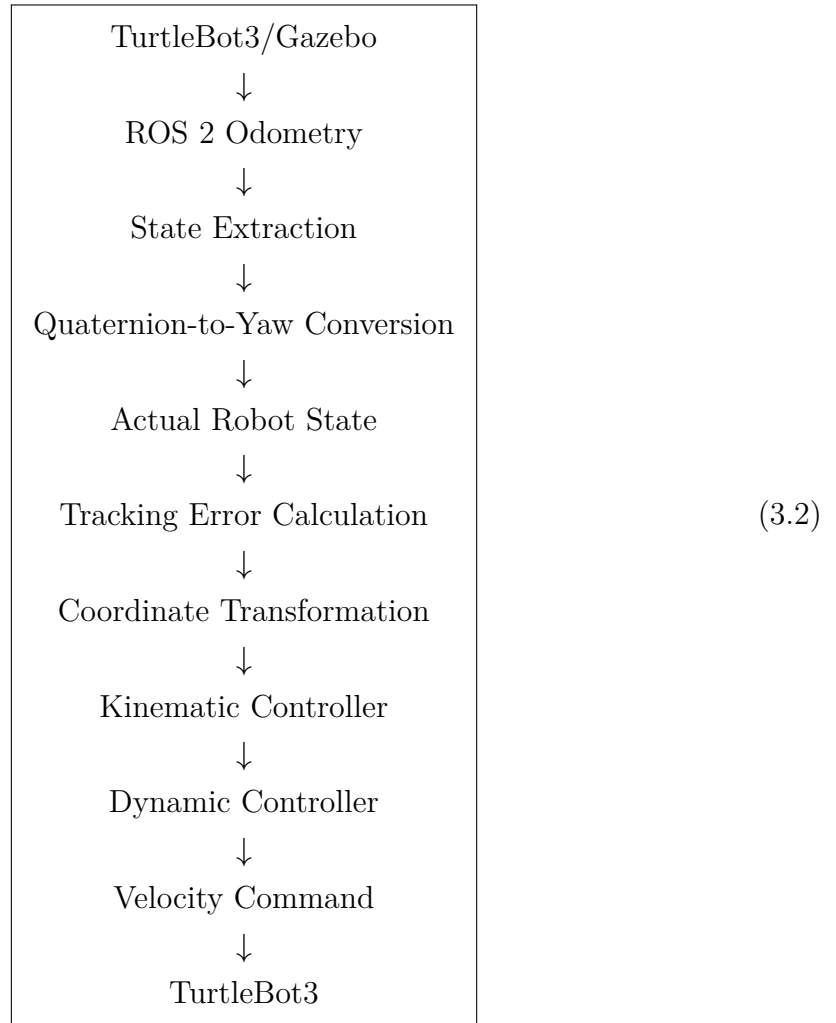
The complete architecture consists of two major paths:

1. **Feedback path** – obtains the actual robot state.
2. **Control path** – calculates and sends the required commands.

The feedback path begins with TurtleBot3. The robot's odometry information is transmitted through ROS 2 and received by the Simulink model.

The control path then processes the received information and generates the required commands.

The overall process is:



This architecture allows the controller to continuously observe the robot's motion and modify the control input according to the instantaneous tracking error.

3.3 TurtleBot3

TurtleBot3 is the mobile robotic platform used in this project. It is a differential-drive mobile robot capable of planar motion.

The robot's motion can be described using three primary state variables:

$$x, \quad y, \quad \phi \quad (3.3)$$

where:

- x represents the robot's position along the global X -axis,
- y represents the robot's position along the global Y -axis,
- ϕ represents the robot's orientation or heading angle.

Therefore, the robot pose is written as

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix}. \quad (3.4)$$

The controller attempts to make the actual robot pose follow a desired pose:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix}. \quad (3.5)$$

Thus, the fundamental objective is:

$$\boxed{\mathbf{q}(t) \rightarrow \mathbf{q}_d(t)} \quad (3.6)$$

during the complete trajectory.

3.4 Differential-Drive Configuration

TurtleBot3 uses a differential-drive mechanism. The motion of the robot is controlled by the relative motion of its wheels.

The robot has two independently controlled wheels:

- Left wheel

- Right wheel

By controlling the velocities of these wheels, the robot can:

- Move forward
- Move backward
- Turn left
- Turn right
- Rotate about its vertical axis

For planar motion, the robot's kinematic model can be represented as:

$$\dot{x} = v \cos \phi \quad (3.7)$$

$$\dot{y} = v \sin \phi \quad (3.8)$$

$$\dot{\phi} = \omega \quad (3.9)$$

where:

- v = linear velocity,
- ω = angular velocity.

These equations establish the relationship between the velocity commands and the robot's position and orientation.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \phi & 0 \\ \sin \phi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.10)$$

3.5 ROS 2 Framework

Robot Operating System 2 (ROS 2) is used as the communication framework between the robot and the Simulink controller.

ROS 2 allows different components of the robotic system to exchange information through communication mechanisms such as:

- Nodes
- Topics
- Publishers
- Subscribers
- Messages

In the present project, ROS 2 acts as the communication layer connecting TurtleBot3 and MATLAB/Simulink.

The basic structure is:

$$\boxed{\text{Robot} \rightarrow \text{ROS 2} \rightarrow \text{Simulink}} \quad (3.11)$$

for feedback and

$$\boxed{\text{Simulink} \rightarrow \text{ROS 2} \rightarrow \text{Robot}} \quad (3.12)$$

for control commands.

3.6 ROS 2 Odometry Communication

The robot state is obtained through the ROS 2 odometry system.

The odometry information contains the robot's position and orientation information. In the Simulink implementation, the received message is processed to obtain the variables required by the controller.

The important position variables are:

$$x_{\text{real}} \quad (3.13)$$

and

$$y_{\text{real}}. \quad (3.14)$$

The orientation information is provided in quaternion form.

The subscriber output contains the actual robot position x_{real} , y_{real} , and quaternion components q_x, q_y, q_z, q_w .

Odometry Feedback:

Odometry provides the actual robot state required by the controller for tracking-error calculation and closed-loop feedback.

3.7 ROS 2 Publisher and Subscriber

The communication system contains two important components.

3.7.1 Publisher

The publisher transmits information through a ROS 2 topic.

For the robot-state feedback, the robot/Gazebo system publishes odometry information.

For the control side, the final velocity command is published to the appropriate command topic.

3.7.2 Subscriber

The Simulink ROS 2 Subscriber receives the robot's odometry information.

The subscriber makes the information available to the Simulink control system.

Therefore:

$$\boxed{\text{Robot State} \xrightarrow{\text{ROS 2}} \text{Simulink Subscriber}} \quad (3.15)$$

The project material identifies ROS 2 communication and odometry acquisition as fundamental parts of the implementation.

3.8 Odometry State Processing

After the odometry message is received, the information is separated into individual signals.

The relevant signals include:

$$q_x, \quad q_y, \quad q_z, \quad q_w \quad (3.16)$$

and

$$x_{\text{real}}, \quad y_{\text{real}}. \quad (3.17)$$

The position information can directly be used for trajectory-error calculation.

However, the orientation information requires further processing because it is provided in quaternion form.

Therefore, the orientation-processing path is:

$$\boxed{q_x, q_y, q_z, q_w \rightarrow \text{QUAT2ZYX} \rightarrow \phi} \quad (3.18)$$

3.9 Quaternion Orientation

The orientation of the robot is represented using a quaternion:

$$q = [q_x, q_y, q_z, q_w]. \quad (3.19)$$

The four components are:

$$q_x = \text{x component} \quad (3.20)$$

$$q_y = \text{y component} \quad (3.21)$$

$$q_z = \text{z component} \quad (3.22)$$

$$q_w = \text{scalar component}. \quad (3.23)$$

The project uses the QUAT2ZYX block to convert this quaternion representation into Euler-angle information.

The ZYX convention represents the orientation using:

$$[\text{yaw}, \text{pitch}, \text{roll}]. \quad (3.24)$$

For the planar motion considered in this project, yaw is the important orientation variable.

Thus,

$$\boxed{\phi = \text{yaw}} \quad (3.25)$$

is used by the trajectory-tracking controller.

3.10 Reference Trajectory Generation

The system generates a desired circular trajectory.

The desired trajectory is represented by:

$$x_d(t), \quad y_d(t), \quad \phi_d(t). \quad (3.26)$$

For a circular trajectory:

$$x_d = R \cos(\omega t) \quad (3.27)$$

$$y_d = R \sin(\omega t) \quad (3.28)$$

and

$$\phi_d = \omega t. \quad (3.29)$$

The reference trajectory provides the desired position and orientation that the robot must follow.

The trajectory parameters used in the project are:

$$R = 0.4 \text{ m} \quad (3.30)$$

and

$$\omega = 0.5 \text{ rad/s}. \quad (3.31)$$

3.11 Actual and Desired State Comparison

The actual state obtained from odometry is continuously compared with the desired state generated by the trajectory generator.

The actual state is:

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix} \quad (3.32)$$

while the desired state is:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix}. \quad (3.33)$$

The differences are calculated as:

$$e_x = x_d - x_{\text{real}} \quad (3.34)$$

$$e_y = y_d - y_{\text{real}} \quad (3.35)$$

and

$$e_\phi = \phi_d - \phi. \quad (3.36)$$

These errors indicate how far the actual robot state is from the desired state.

3.12 Global and Robot Coordinate Frames

Two coordinate systems are used in the control architecture.

3.12.1 Global Frame

The global frame is fixed and provides the reference for the robot's overall position.

The robot position is represented by:

$$(x, y). \quad (3.37)$$

3.12.2 Robot Frame

The robot frame moves and rotates with the robot.

In this frame:

- x_r represents the forward direction,
- y_r represents the lateral direction.

The controller uses the robot-frame errors because they have a direct physical interpretation for robot motion.

For example,

$$x_{r,\text{dif}} > 0 \quad (3.38)$$

indicates that the desired position is in front of the robot, while a positive or negative $y_{r,\text{dif}}$ indicates lateral displacement.

3.13 Coordinate Transformation

The global-frame position errors are transformed into the robot frame using the robot's current orientation.

The transformed errors are:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (3.39)$$

and

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (3.40)$$

The orientation error remains:

$$\phi_{r,\text{dif}} = \phi_d - \phi. \quad (3.41)$$

Thus:

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \end{bmatrix} = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} e_x \\ e_y \end{bmatrix} \quad (3.42)$$

This transformation provides the controller with errors relative to the current robot orientation.

3.14 Kinematic Controller

The transformed errors are supplied to the nonlinear kinematic controller.

The controller inputs include:

$$v_d \tag{3.43}$$

$$\omega_d \tag{3.44}$$

$$x_{r,\text{dif}} \tag{3.45}$$

$$y_{r,\text{dif}} \tag{3.46}$$

and

$$\phi_{r,\text{dif}}. \tag{3.47}$$

The controller produces:

$$v_c \tag{3.48}$$

and

$$\omega_c. \tag{3.49}$$

3.15 Linear Velocity Control

The linear velocity command is generated using:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \tag{3.50}$$

The first term,

$$k_x x_{r,\text{dif}}, \tag{3.51}$$

is the feedback correction term.

The second term,

$$v_d \cos(\phi_{r,\text{dif}}), \quad (3.52)$$

is the feedforward trajectory-following term with an alignment factor.

The project material explicitly identifies these feedback and feedforward components.

3.16 Angular Velocity Control

The angular velocity command is generated using:

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d \quad (3.53)$$

The three terms have different functions.

3.16.1 Heading Correction

The heading correction term is:

$$k_\phi \sin(\phi_{r,\text{dif}}) \quad (3.54)$$

which corrects the robot's orientation.

3.16.2 Lateral Correction

The lateral correction term is:

$$45v_d y_{r,\text{dif}} \quad (3.55)$$

which corrects lateral displacement from the trajectory.

3.16.3 Feedforward Term

The feedforward term is:

$$\omega_d \quad (3.56)$$

which provides the desired angular velocity.

The lateral term produces left or right steering according to the sign of $y_{r,dif}$, with the correction scaled by v_d .

3.17 Velocity Saturation

The calculated velocity commands are passed through saturation limits.

For linear velocity:

$$\boxed{-0.22 \leq v_c \leq 0.22} \quad (3.57)$$

For angular velocity:

$$\boxed{-2.5 \leq \omega_c \leq 2.5} \quad (3.58)$$

The saturation blocks prevent the controller from generating velocity commands outside the specified robot operating range.

The complete saturation process can be represented as:

$$\boxed{\text{Controller Output} \rightarrow \text{Saturation} \rightarrow \text{Permissible Velocity}} \quad (3.59)$$

3.18 Dynamic Velocity Controller

After the kinematic controller produces v_c and ω_c , the system uses a second control stage for velocity control.

This stage is represented by **Controller 2** in the Simulink model.

The actual velocity information is compared with the commanded velocity:

$$v_c \quad (3.60)$$

and

$$\omega_c. \quad (3.61)$$

The resulting velocity errors are supplied to the dynamic controller.

This creates an inner velocity-control loop.

Inner Velocity-Control Loop:

The dynamic controller compares the commanded velocity with the actual velocity response and generates the required corrective actuation.

3.19 Dynamic Controller Operation

The dynamic controller processes the velocity commands and their associated errors to generate the required actuation quantities.

The controller therefore provides an additional control layer between the desired trajectory and the physical robot motion.

The complete control structure becomes:

$$\text{Trajectory} \rightarrow \text{Kinematic Controller} \rightarrow (v_c, \omega_c) \rightarrow \text{Dynamic Controller} \rightarrow \text{Robot Motion} \quad (3.62)$$

The project material characterizes this stage as a dynamic nonlinear feedback-feedforward controller.

3.20 Command Generation and Publication

Once the control commands are obtained, they must be converted into the appropriate ROS 2 message format.

The command message uses the standard velocity fields:

$$\text{Linear.x} \quad (3.63)$$

and

$$\text{Angular.z.} \quad (3.64)$$

The required assignment is:

$$\text{Linear.x} = v_c \quad (3.65)$$

and

$$\boxed{\text{Angular.z} = \omega_c} \quad (3.66)$$

The Bus Assignment stage takes v_c and ω_c and assigns them to `linear.x` and `angular.z`.

3.21 ROS 2 Command Publisher

After the command message is constructed, it is sent through the ROS 2 command topic.

The command publisher sends the message to:

$$\boxed{\text{/cmd_vel}} \quad (3.67)$$

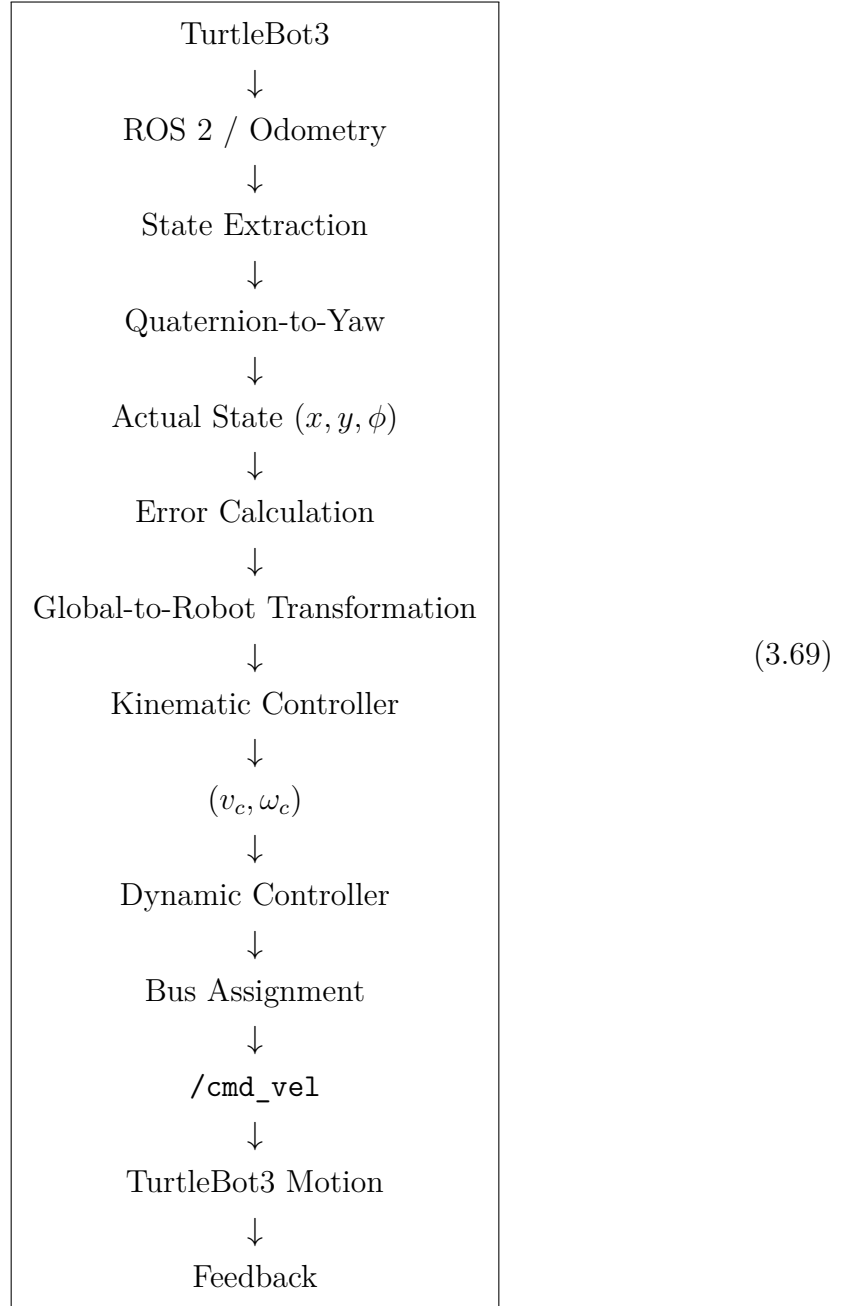
TurtleBot3 listens to this topic and uses the received linear and angular velocity commands to control its motion.

Therefore, the final command path is:

$$\boxed{(v_c, \omega_c) \rightarrow \text{Bus Assignment} \rightarrow \text{/cmd_vel} \rightarrow \text{TurtleBot3}} \quad (3.68)$$

3.22 Complete Closed-Loop Architecture

Combining all the individual stages gives the complete control architecture:



This architecture allows the robot's actual motion to continuously influence the control input.

3.23 Working Principle of the Complete System

The operation of the proposed system can be summarized in the following sequence.

3.23.1 Step 1 – Robot Motion

TurtleBot3 moves according to the applied velocity commands.

3.23.2 Step 2 – State Measurement

The robot's current position and orientation are obtained through odometry.

3.23.3 Step 3 – ROS 2 Communication

The odometry information is transferred to Simulink through ROS 2.

3.23.4 Step 4 – State Processing

The position x_{real} , y_{real} and quaternion orientation are extracted.

3.23.5 Step 5 – Yaw Calculation

The quaternion is converted into the yaw angle:

$$\phi. \quad (3.70)$$

3.23.6 Step 6 – Reference Generation

The desired circular trajectory is generated:

$$x_d, \quad y_d, \quad \phi_d. \quad (3.71)$$

3.23.7 Step 7 – Error Calculation

The desired and actual states are compared:

$$e_x, \quad e_y, \quad e_\phi. \quad (3.72)$$

3.23.8 Step 8 – Coordinate Transformation

The position errors are transformed into the robot frame:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}. \quad (3.73)$$

3.23.9 Step 9 – Kinematic Control

The controller generates:

$$v_c, \quad \omega_c. \quad (3.74)$$

3.23.10 Step 10 – Saturation

The velocity commands are restricted to the specified limits.

3.23.11 Step 11 – Dynamic Control

The velocity-control stage processes the commanded and actual velocities.

3.23.12 Step 12 – ROS 2 Publication

The final commands are assigned to:

$$\text{linear.x} \quad (3.75)$$

and

$$\text{angular.z}. \quad (3.76)$$

3.23.13 Step 13 – Robot Motion

The command is published through `/cmd_vel`, and TurtleBot3 responds accordingly.

3.23.14 Step 14 – Feedback

The new robot state is again measured and returned to the controller.

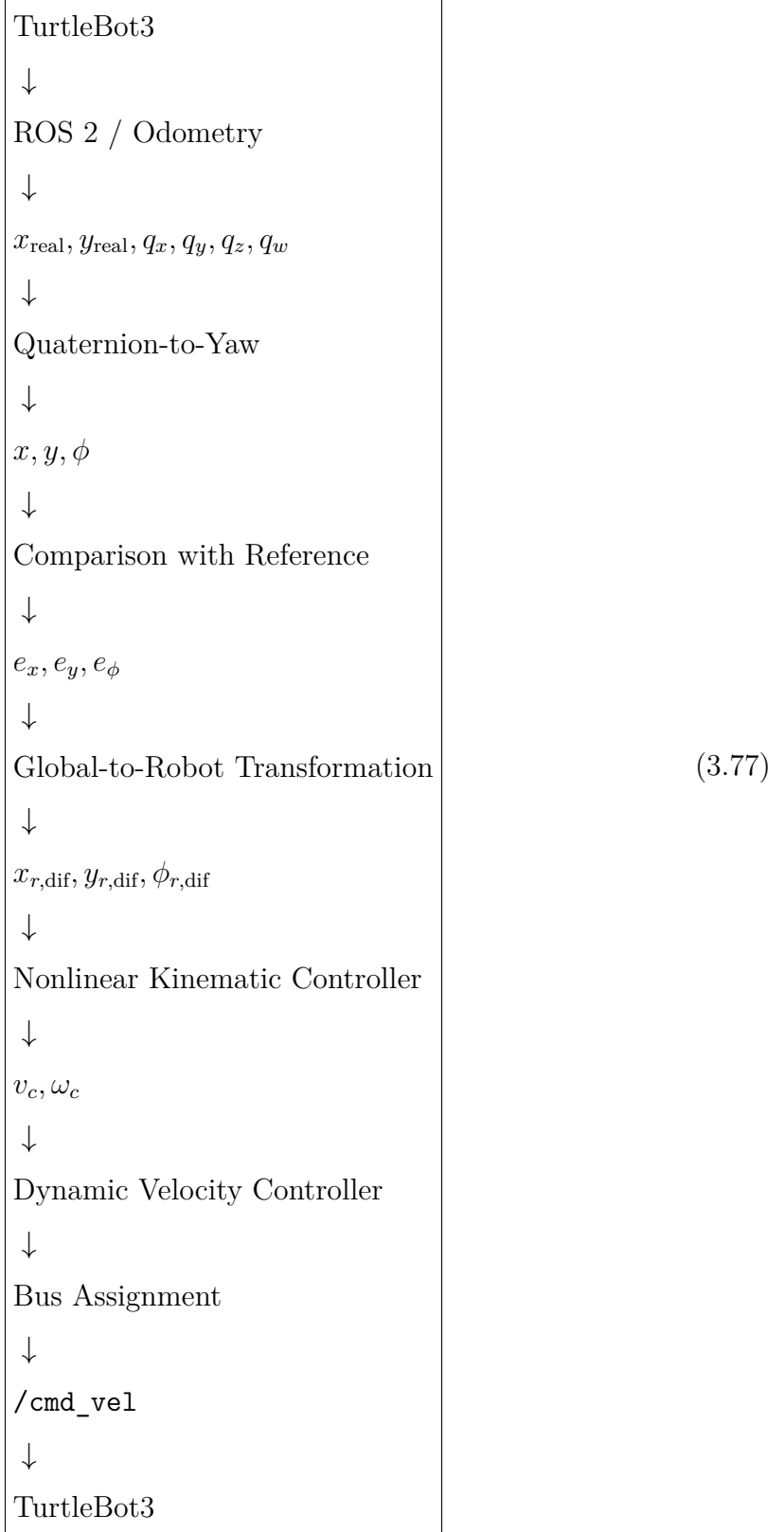
Therefore, the entire process repeats continuously.

3.24 Summary

The proposed system integrates **TurtleBot3**, **ROS 2**, **MATLAB/Simulink**, **odometry feedback**, **trajectory generation**, **coordinate transformation**, **nonlinear**

kinematic control, dynamic velocity control, and ROS 2 command publication into a single closed-loop architecture.

The main signal flow can be summarized as:



Thus, the system provides a complete feedback-based framework for trajectory tracking. The nonlinear feedback-feedforward trajectory-tracking controller is implemented using ROS 2 and MATLAB/Simulink, with the objective of minimizing position and orientation errors while keeping motion within the robot's specified physical limits.

ROS 2 Communication and Data Acquisition

4.1 Introduction

Robot Operating System 2 (ROS 2) forms the communication layer between the TurtleBot3 system and the MATLAB/Simulink control model. In the proposed trajectory-tracking system, ROS 2 is responsible for transferring the robot's measured state information to Simulink and transmitting the calculated velocity commands from Simulink back to the robot.

The complete communication process can be represented as:

$$\boxed{\text{TurtleBot3} \rightarrow \text{ROS 2} \rightarrow \text{Simulink} \rightarrow \text{Controller} \rightarrow \text{ROS 2} \rightarrow \text{TurtleBot3}} \quad (4.1)$$

The project specifically requires communication between TurtleBot3 and Simulink using ROS 2, acquisition of position and orientation through odometry, and publication of velocity commands for robot motion.

This chapter explains the ROS 2 communication architecture, odometry data acquisition, state extraction, orientation processing, and command publication used in the proposed system.

Role of ROS 2:

ROS 2 provides the communication interface between the TurtleBot3 robotic system and the MATLAB/Simulink control model. It carries feedback information from the robot to the controller and transmits calculated velocity commands back to the robot.

4.2 Role of ROS 2 in the Proposed System

ROS 2 acts as the interface between the robotic platform and the control algorithm implemented in Simulink.

The controller requires real-time information about the robot's current state. This information cannot be generated by the controller itself; it must be obtained from the robot or simulation environment.

Similarly, after the controller calculates the required velocity, the command must be transferred to the robot.

Therefore, ROS 2 performs two important functions:

1. **Feedback communication**
2. **Control-command communication**

The feedback path is:

$$\boxed{\text{TurtleBot3} \rightarrow \text{Odometry} \rightarrow \text{ROS 2} \rightarrow \text{Simulink}} \quad (4.2)$$

The command path is:

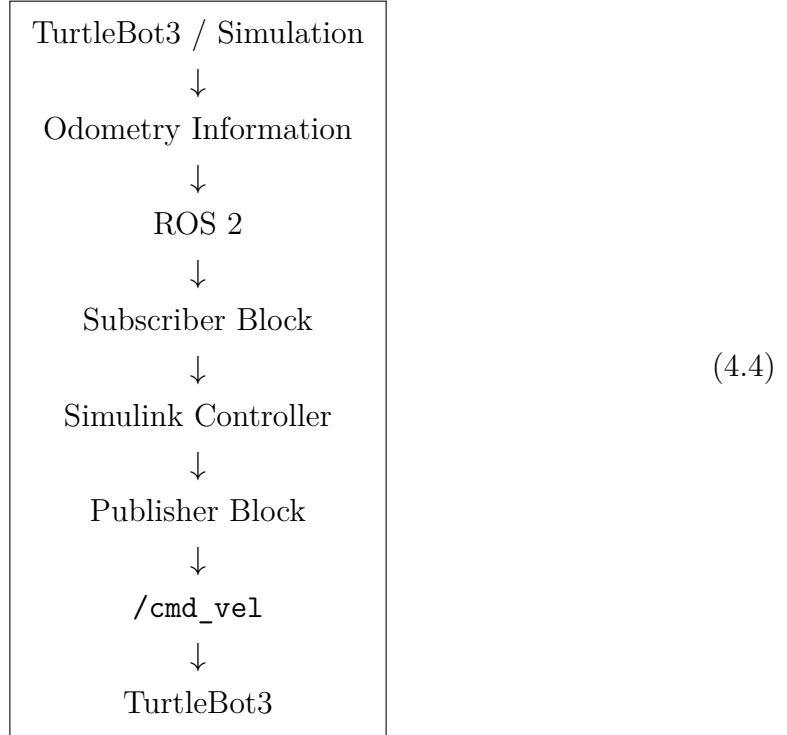
$$\boxed{\text{Simulink} \rightarrow \text{ROS 2} \rightarrow \text{/cmd_vel} \rightarrow \text{TurtleBot3}} \quad (4.3)$$

This communication structure makes it possible to implement the controller as a closed-loop system.

4.3 ROS 2 Communication Architecture

The communication architecture consists of the robot, ROS 2 communication topics, Simulink subscriber and publisher blocks, and the trajectory-tracking controller.

The overall structure is:



The subscriber is used for receiving the robot's state, whereas the publisher is used for transmitting the calculated velocity command.

Communication Architecture:

The feedback side obtains robot-state information through odometry, while the control side sends the calculated velocity command through the ROS 2 command interface.

4.4 ROS 2 Subscriber

The ROS 2 Subscriber is the first major block on the feedback side of the system.

Its function is to receive the odometry message containing information about the robot's current state.

The received information includes:

$$x_{\text{real}}, \quad y_{\text{real}} \quad (4.5)$$

for position and

$$q_x, \quad q_y, \quad q_z, \quad q_w \quad (4.6)$$

for orientation.

The subscriber therefore provides the basic feedback information required by the trajectory-tracking controller.

Subscriber Output:

The ROS 2 Subscriber provides the actual robot position and quaternion orientation information required for subsequent state processing and trajectory tracking.

4.5 Odometry Data

Odometry is used to obtain information about the current motion and pose of the robot.

For the proposed trajectory-tracking system, the important information extracted from odometry is:

$$\boxed{x_{\text{real}}, \quad y_{\text{real}}, \quad \phi} \quad (4.7)$$

where:

- x_{real} = actual x -coordinate,
- y_{real} = actual y -coordinate,
- ϕ = actual robot orientation/yaw.

The position information can be directly used for calculating the difference between the desired and actual trajectory.

The orientation information requires an additional conversion because it is received in quaternion form.

4.6 Structure of Quaternion Data

The robot orientation is received as a quaternion containing four components:

$$q = [q_x, q_y, q_z, q_w]. \quad (4.8)$$

The components are:

$$q_x = \text{x component} \quad (4.9)$$

$$q_y = \text{y component} \quad (4.10)$$

$$q_z = \text{z component} \quad (4.11)$$

$$q_w = \text{scalar component.} \quad (4.12)$$

A quaternion is useful for representing three-dimensional orientation, but the trajectory-tracking controller used in this project requires the robot's planar heading angle.

Therefore, the quaternion must be converted into an appropriate Euler-angle representation.

4.7 Quaternion-to-Yaw Conversion

The Simulink model uses the **QUAT2ZYX** block for converting the quaternion orientation into ZYX Euler-angle representation.

The ZYX representation provides the orientation in terms of:

$$[\text{yaw}, \text{pitch}, \text{roll}]. \quad (4.13)$$

According to the project material, the required angle for the controller is the rotation about the z -axis, i.e. the **yaw angle**.

Thus, the orientation-processing chain can be represented as:

$$\boxed{[q_x, q_y, q_z, q_w] \rightarrow \text{QUAT2ZYX} \rightarrow \text{Yaw} \rightarrow \phi} \quad (4.14)$$

The resulting angle ϕ represents the robot's heading in the plane.

Yaw Extraction:

For planar TurtleBot3 motion, the yaw angle is the required orientation variable used by the trajectory-tracking controller.

4.8 Importance of Yaw Angle

For a differential-drive mobile robot moving on a two-dimensional plane, the robot's orientation with respect to the global coordinate frame is represented by the yaw angle.

The robot state is therefore represented as:

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix}. \quad (4.15)$$

This state is required by the trajectory-tracking controller because the controller needs to determine not only how far the robot is from the desired position, but also how its current heading differs from the desired heading.

The yaw angle is also required for converting global-coordinate tracking errors into robot-frame tracking errors.

4.9 Extraction of Actual Robot State

After receiving the odometry message, the required state variables are separated from the received data.

The actual position is:

$$x = x_{\text{real}} \quad (4.16)$$

$$y = y_{\text{real}}. \quad (4.17)$$

The actual orientation is:

$$\phi = \text{Yaw}. \quad (4.18)$$

Therefore:

$$\boxed{\mathbf{q}_{\text{actual}} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix}} \quad (4.19)$$

This actual state is continuously supplied to the trajectory-tracking section.

4.10 Reference and Actual State

The actual state obtained from ROS 2 is compared with the desired state generated by the trajectory generator.

The desired state is:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix}. \quad (4.20)$$

The actual state is:

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix}. \quad (4.21)$$

The tracking error is calculated using:

$$e_x = x_d - x_{\text{real}} \quad (4.22)$$

$$e_y = y_d - y_{\text{real}} \quad (4.23)$$

and

$$e_\phi = \phi_d - \phi. \quad (4.24)$$

Thus:

$$\boxed{\mathbf{e} = \mathbf{q}_d - \mathbf{q}} \quad (4.25)$$

The calculated error becomes the input to the subsequent control algorithm.

4.11 Role of ROS 2 in the Closed-Loop System

ROS 2 is not itself responsible for calculating the trajectory-tracking control law. Instead, it provides the communication path required by the controller.

The control process is divided into the following stages:

4.11.1 Stage 1 – Measurement

The robot generates its actual state.

4.11.2 Stage 2 – Communication

The actual state is transferred through ROS 2.

4.11.3 Stage 3 – Processing

Simulink extracts position and orientation information.

4.11.4 Stage 4 – Control

The trajectory-tracking controller calculates the required velocity.

4.11.5 Stage 5 – Command Communication

The calculated velocity is transferred through ROS 2.

4.11.6 Stage 6 – Robot Motion

TurtleBot3 responds to the velocity command.

4.11.7 Stage 7 – Feedback

The new robot state is again obtained through odometry.

Therefore:

$$\boxed{\text{Measurement} \rightarrow \text{Communication} \rightarrow \text{Control} \rightarrow \text{Command} \rightarrow \text{Motion} \rightarrow \text{Feedback}}$$

(4.26)

4.12 Controller Output

The nonlinear kinematic controller generates two primary command signals:

$$v_c \tag{4.27}$$

and

$$\omega_c. \quad (4.28)$$

Here,

$$v_c = \text{commanded linear velocity} \quad (4.29)$$

and

$$\omega_c = \text{commanded angular velocity}. \quad (4.30)$$

These signals are subsequently processed by the dynamic velocity-control stage.

Before being sent to the robot, the commands are restricted to the specified limits:

$$-0.22 \leq v_c \leq 0.22 \quad (4.31)$$

and

$$-2.5 \leq \omega_c \leq 2.5. \quad (4.32)$$

$$\boxed{\begin{array}{l} -0.22 \leq v_c \leq 0.22 \\ -2.5 \leq \omega_c \leq 2.5 \end{array}} \quad (4.33)$$

4.13 Dynamic Controller and ROS 2 Interface

The velocity commands from the kinematic controller are supplied to the dynamic controller.

The actual velocity is compared with the commanded velocity:

$$v_c \quad (4.34)$$

and

$$\omega_c. \quad (4.35)$$

The resulting errors are used by the dynamic controller.

This provides an additional control layer before the final command is sent to TurtleBot3.

Dynamic Control Layer:

The dynamic controller provides an additional velocity-control stage between the kinematic controller and the physical robot.

4.14 Command Message Formation

After the control algorithm produces the required velocity commands, the commands must be placed into the appropriate ROS 2 velocity-message fields.

The project uses a **Bus Assignment** stage for this purpose.

The two main assignments are:

$$\boxed{\text{Linear.x} = v_c} \quad (4.36)$$

and

$$\boxed{\text{Angular.z} = \omega_c} \quad (4.37)$$

The command message therefore contains the required linear and angular motion information for TurtleBot3.

4.15 /cmd_vel Topic

The generated velocity message is sent to the ROS 2 command topic:

$$\boxed{\text{/cmd_vel}} \quad (4.38)$$

The publisher receives the command message generated by the Simulink control system and transmits it to /cmd_vel.

TurtleBot3 listens to this topic and uses the received velocity information to control its movement.

The complete command path is therefore:

$$\boxed{v_c, \omega_c \rightarrow \text{Bus Assignment} \rightarrow \text{ROS 2 Publisher} \rightarrow \text{/cmd_vel} \rightarrow \text{TurtleBot3}} \quad (4.39)$$

4.16 ROS 2 Feedback Path

The feedback path operates in the opposite direction.

The robot provides odometry information, which is received by the Simulink subscriber.

The feedback path is:

$$\boxed{\text{TurtleBot3} \rightarrow \text{Odometry} \rightarrow \text{ROS 2} \rightarrow \text{Subscriber} \rightarrow \text{Simulink}} \quad (4.40)$$

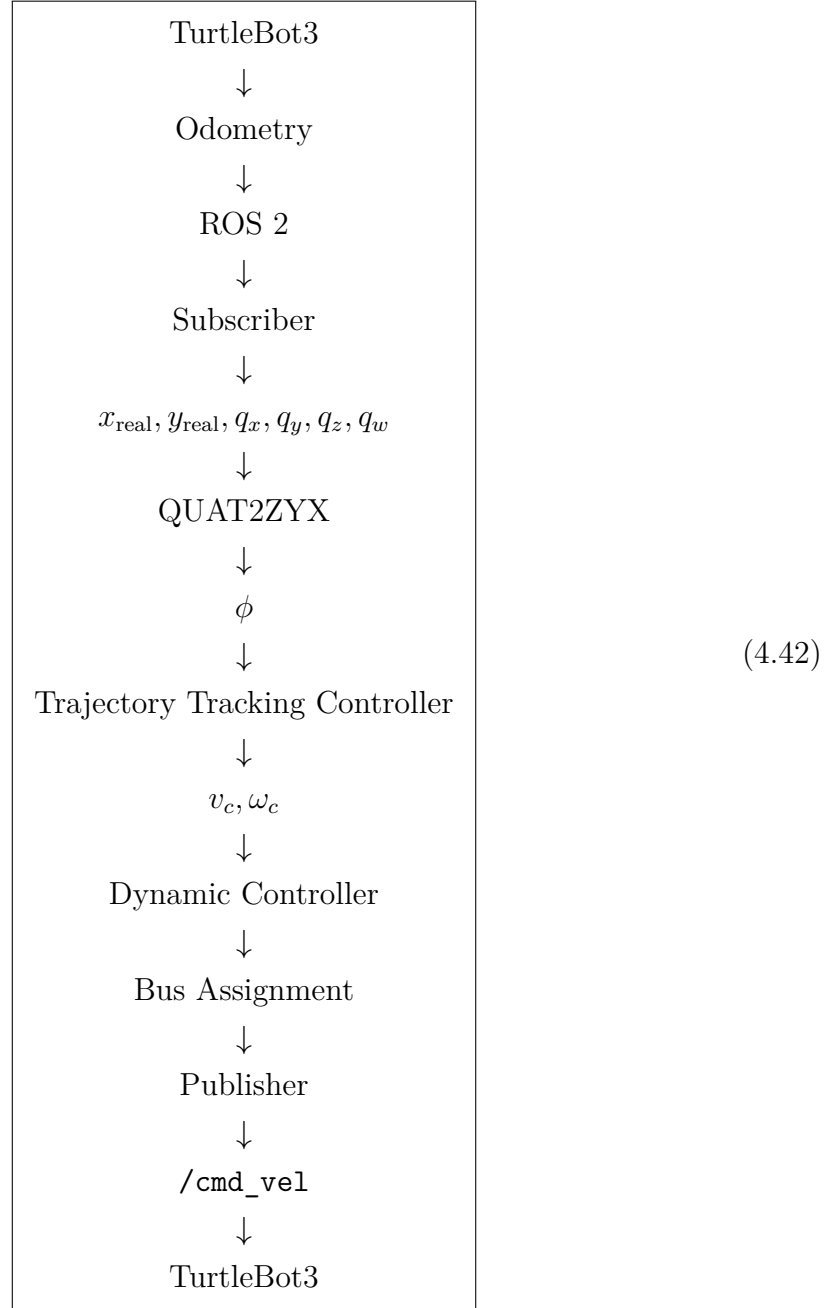
The received data contains:

$$x_{\text{real}}, \quad y_{\text{real}}, \quad q_x, \quad q_y, \quad q_z, \quad q_w. \quad (4.41)$$

The quaternion information is then converted into the required yaw angle.

4.17 Complete ROS 2 Data Flow

The complete communication process can be summarized as follows:



This forms the communication backbone of the entire trajectory-tracking system.

4.18 Closed-Loop Operation

The ROS 2 communication system allows the controller to operate continuously rather than as a one-time calculation.

At every control cycle:

1. TurtleBot3 provides its current state.

2. ROS 2 transfers the state information to Simulink.
3. Simulink extracts the actual position and orientation.
4. The desired trajectory is generated.
5. The actual and desired states are compared.
6. Tracking errors are calculated.
7. The errors are transformed into the robot frame.
8. The kinematic controller generates v_c and ω_c .
9. The dynamic controller processes the velocity commands.
10. The commands are assigned to the appropriate ROS 2 message fields.
11. The command is published through `/cmd_vel`.
12. TurtleBot3 responds to the command.
13. The resulting robot state is measured again.

Thus, the loop continues:

$$\boxed{\text{Feedback} \rightarrow \text{Control} \rightarrow \text{Command} \rightarrow \text{Motion} \rightarrow \text{Feedback}} \quad (4.43)$$

4.19 Importance of ROS 2 Communication in the Project

ROS 2 communication is essential because it connects the mathematical controller implemented in Simulink with the robotic system.

Without the feedback channel, the controller would not know the actual robot position and orientation.

Without the command channel, the calculated control output could not be applied to the robot.

Therefore, the two communication paths are equally important:

$$\boxed{\text{Feedback Path} = \text{Robot} \rightarrow \text{Simulink}} \quad (4.44)$$

and

$$\boxed{\text{Control Path} = \text{Simulink} \rightarrow \text{Robot}} \quad (4.45)$$

Together, they form the communication structure required for closed-loop trajectory tracking.

4.20 Summary

This chapter presented the ROS 2 communication and data-acquisition structure used in the TurtleBot3 trajectory-tracking system.

The ROS 2 Subscriber receives odometry information containing the robot's actual position and quaternion orientation. The position components x_{real} and y_{real} are extracted directly, while the quaternion components q_x, q_y, q_z, q_w are processed using the QUAT2ZYX block to obtain the yaw angle required by the controller.

The controller then generates the commanded linear and angular velocities:

$$v_c, \quad \omega_c. \quad (4.46)$$

These commands are assigned to:

$$\text{Linear.x} = v_c \quad (4.47)$$

and

$$\text{Angular.z} = \omega_c, \quad (4.48)$$

and are published through `/cmd_vel` for TurtleBot3 motion control.

Hence, the complete ROS 2 communication process is:

$$\boxed{\text{Odometry} \rightarrow \text{ROS 2} \rightarrow \text{Simulink} \rightarrow \text{Controller} \rightarrow \text{ROS 2} \rightarrow \text{/cmd_vel} \rightarrow \text{TurtleBot3}} \quad (4.49)$$

Chapter 4 Conclusion:

The ROS 2 communication architecture provides the necessary feedback and command interfaces for the nonlinear trajectory-tracking controller. Odometry information is transferred to Simulink for state estimation and control, while the generated velocity commands are published through `/cmd_vel` to control TurtleBot3.

Robot Modelling

5.1 Introduction

The mathematical modelling of the mobile robot is an important part of the trajectory-tracking control system. A mathematical model provides a relationship between the robot's motion variables and the control inputs applied to the robot.

In this project, **TurtleBot3** is considered as a differential-drive mobile robot operating on a two-dimensional plane. Its motion is described using three main state variables: the x -position, y -position, and orientation angle.

The robot state can be represented as

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (5.1)$$

where:

- x is the robot position along the global X -axis,
- y is the robot position along the global Y -axis,
- ϕ is the robot heading or yaw angle.

The control inputs used for the trajectory-tracking system are the linear velocity v and angular velocity ω :

$$\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}. \quad (5.2)$$

The objective of the controller is to determine appropriate values of v and ω such that the robot follows the desired trajectory.

5.2 TurtleBot3 as a Differential-Drive Robot

TurtleBot3 uses a differential-drive mechanism in which the left and right wheels are independently controlled.

The motion of the robot is determined by the velocities of these two wheels.

Let

$$v_R = \text{right-wheel linear velocity} \quad (5.3)$$

and

$$v_L = \text{left-wheel linear velocity}. \quad (5.4)$$

The robot's forward linear velocity depends on the combined motion of the two wheels, while its angular velocity depends on the difference between their velocities.

If the two wheels rotate at the same velocity,

$$v_R = v_L, \quad (5.5)$$

the robot moves approximately in a straight line.

If

$$v_R > v_L, \quad (5.6)$$

the robot turns toward one direction, whereas if

$$v_L > v_R, \quad (5.7)$$

the robot turns toward the opposite direction.

Thus, independent control of the two wheels allows TurtleBot3 to perform both translational and rotational motion.

5.3 Robot Coordinate System

Two coordinate frames are important for the trajectory-tracking problem:

1. Global/world coordinate frame
2. Robot/body coordinate frame

5.3.1 Global Coordinate Frame

The global coordinate frame is fixed with respect to the environment.

The robot position is represented by

$$(x, y). \quad (5.8)$$

The desired trajectory is also initially defined with respect to this coordinate system.

5.3.2 Robot Coordinate Frame

The robot coordinate frame moves and rotates along with TurtleBot3.

The x_r -axis is normally associated with the forward direction of the robot, while the y_r -axis represents the lateral direction.

This frame becomes particularly important when the tracking error is transformed from the global frame into the robot frame.

5.4 Robot Pose

The pose of TurtleBot3 consists of its position and orientation.

The position is

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix} \quad (5.9)$$

and the orientation is

$$\phi. \quad (5.10)$$

Therefore, the complete pose is

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (5.11)$$

The corresponding desired pose is

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix} \quad (5.12)$$

The trajectory-tracking problem is therefore the problem of making

$$\mathbf{q} \rightarrow \mathbf{q}_d. \quad (5.13)$$

5.5 Control Inputs

The two primary control inputs for the differential-drive robot are

$$v \quad (5.14)$$

and

$$\omega. \quad (5.15)$$

Here,

$$v = \text{linear velocity} \quad (5.16)$$

and

$$\omega = \text{angular velocity}. \quad (5.17)$$

The linear velocity controls the forward motion of the robot, while the angular velocity controls its rotational motion.

For the controller developed in this project, the corresponding commanded quantities are represented as

$$v_c \quad (5.18)$$

and

$$\omega_c. \quad (5.19)$$

The kinematic controller calculates these quantities based on the tracking error.

5.6 Kinematic Model of TurtleBot3

The kinematic model describes the relationship between the robot's velocity and the rate of change of its position and orientation.

For a differential-drive robot moving on a planar surface, the kinematic equations are

$$\dot{x} = v \cos \phi \quad (5.20)$$

$$\dot{y} = v \sin \phi \quad (5.21)$$

$$\dot{\phi} = \omega \quad (5.22)$$

These equations form the fundamental mathematical model used for the trajectory-tracking controller.

The first equation indicates that the x -direction velocity depends on the robot's linear velocity and its orientation.

The second equation indicates that the y -direction velocity also depends on the robot's orientation.

The third equation shows that the rate of change of the robot heading is directly determined by the angular velocity.

5.7 Interpretation of the Kinematic Equations

The equation

$$\dot{x} = v \cos \phi \quad (5.23)$$

means that only the component of the robot's forward velocity along the global X -axis contributes to x -direction motion.

Similarly,

$$\dot{y} = v \sin \phi \quad (5.24)$$

means that the component of forward velocity along the global Y -axis determines the robot's y -direction motion.

Finally,

$$\dot{\phi} = \omega \quad (5.25)$$

means that a positive angular velocity changes the robot heading in one direction, while a negative angular velocity changes it in the opposite direction.

Together, these equations describe the planar motion of TurtleBot3.

5.8 State-Space Representation

The kinematic model can be written in matrix form.

The robot state is

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix}. \quad (5.26)$$

The state derivative is

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix}. \quad (5.27)$$

Therefore,

$$\dot{\mathbf{q}} = \begin{bmatrix} \cos \phi & 0 \\ \sin \phi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (5.28)$$

or,

$$\dot{\mathbf{q}} = G(\phi)\mathbf{u} \quad (5.29)$$

where

$$G(\phi) = \begin{bmatrix} \cos \phi & 0 \\ \sin \phi & 0 \\ 0 & 1 \end{bmatrix} \quad (5.30)$$

and

$$\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}. \quad (5.31)$$

This representation clearly shows how the two control inputs influence the three robot states.

5.9 Nonholonomic Constraint

A differential-drive robot cannot move instantaneously sideways.

Unlike a holonomic robot, TurtleBot3 cannot independently command a velocity in the lateral direction.

The robot's motion is constrained to its forward direction and rotation.

This characteristic is important in trajectory tracking because a lateral tracking error cannot be corrected by simply commanding a sideways velocity.

Instead, the robot must:

1. Change its orientation.
2. Move forward.
3. Reduce the lateral error.

This is one of the reasons why the tracking error is transformed into the robot coordinate frame before being supplied to the controller.

5.10 Wheel-Speed Relationship

The linear and angular velocity of a differential-drive robot are related to the left- and right-wheel velocities.

Let

$$r = \text{wheel radius} \quad (5.32)$$

and

$$L = \text{distance between the two wheels.} \quad (5.33)$$

If the wheel angular velocities are

$$\omega_R \tag{5.34}$$

and

$$\omega_L, \tag{5.35}$$

then the corresponding wheel linear velocities are

$$v_R = r\omega_R \tag{5.36}$$

and

$$v_L = r\omega_L. \tag{5.37}$$

The robot linear velocity can then be expressed as

$$\boxed{v = \frac{v_R + v_L}{2}} \tag{5.38}$$

and its angular velocity as

$$\boxed{\omega = \frac{v_R - v_L}{L}} \tag{5.39}$$

for the standard differential-drive convention.

These relationships demonstrate how the robot's overall linear and angular motion can be produced by controlling the two wheels.

5.11 Straight-Line Motion

If the wheel velocities are equal,

$$v_R = v_L, \tag{5.40}$$

then

$$\omega = 0. \tag{5.41}$$

Therefore,

$$\dot{\phi} = 0. \quad (5.42)$$

The robot maintains its current orientation and moves along a straight path.

This condition is useful for understanding the relationship between wheel control and robot motion.

5.12 Rotational Motion

If the two wheels move at different velocities, then

$$v_R \neq v_L. \quad (5.43)$$

Consequently,

$$\omega \neq 0. \quad (5.44)$$

The robot changes its orientation according to

$$\dot{\phi} = \omega. \quad (5.45)$$

If the wheels rotate with equal magnitude but opposite directions, the robot can rotate approximately about its center.

This ability to generate angular motion is essential for following curved trajectories.

5.13 Circular Motion

The reference trajectory used in the project is circular.

For circular motion, the desired trajectory is given by

$$x_d = R \cos(\omega_d t) \quad (5.46)$$

$$y_d = R \sin(\omega_d t). \quad (5.47)$$

The corresponding desired linear velocity is

$$v_d = R\omega_d. \quad (5.48)$$

For the parameters used in the project,

$$R = 0.4 \text{ m} \quad (5.49)$$

and

$$\omega_d = 0.5 \text{ rad/s}. \quad (5.50)$$

Therefore,

$$v_d = R\omega_d \quad (5.51)$$

$$v_d = (0.4)(0.5) \quad (5.52)$$

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (5.53)$$

This desired velocity is used as part of the feedforward information supplied to the kinematic controller.

5.14 Desired Robot Orientation

During circular trajectory tracking, the desired robot orientation changes continuously.

The desired orientation is represented by

$$\phi_d. \quad (5.54)$$

The supplied project material uses

$$\boxed{\phi_d = \omega_d t} \quad (5.55)$$

for the desired orientation trajectory.

The desired angular velocity is therefore

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (5.56)$$

for the specified trajectory.

5.15 Actual Robot State From Odometry

The mathematical model requires the actual state of the robot for feedback control.

This information is obtained from the ROS 2 odometry system.

The actual state consists of

$$x_{\text{real}} \quad (5.57)$$

$$y_{\text{real}} \quad (5.58)$$

and

$$\phi. \quad (5.59)$$

Thus,

$$\mathbf{q}_{\text{actual}} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix} \quad (5.60)$$

The x - and y -coordinates are obtained from odometry, while the orientation ϕ is obtained by converting the received quaternion into yaw.

5.16 Desired and Actual State Comparison

The desired state is

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix}. \quad (5.61)$$

The actual state is

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix}. \quad (5.62)$$

The difference between these two states gives the tracking error:

$$\mathbf{e} = \mathbf{q}_d - \mathbf{q}. \quad (5.63)$$

Therefore,

$$e_x = x_d - x_{\text{real}} \quad (5.64)$$

$$e_y = y_d - y_{\text{real}} \quad (5.65)$$

and

$$e_\phi = \phi_d - \phi. \quad (5.66)$$

These errors are subsequently transformed into the robot frame for controller implementation.

5.17 Global-to-Robot Coordinate Transformation

Because TurtleBot3 is continuously changing its orientation, the global-frame errors cannot always directly represent how the desired trajectory appears to the robot.

Therefore, the position error is transformed using the current robot orientation.

The transformation is

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \end{bmatrix} = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} e_x \\ e_y \end{bmatrix}. \quad (5.67)$$

Hence,

$$\boxed{x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi} \quad (5.68)$$

and

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (5.69)$$

The orientation error is

$$\boxed{\phi_{r,\text{dif}} = \phi_d - \phi} \quad (5.70)$$

The project material identifies this transformation as the conversion of the global-frame tracking error into the robot's local coordinate frame.

5.18 Physical Interpretation of Robot-Frame Errors

The transformed errors have a direct physical meaning.

5.18.1 Forward Error

$$x_{r,\text{dif}} \quad (5.71)$$

represents the error along the robot's forward/backward direction.

5.18.2 Lateral Error

$$y_{r,\text{dif}} \quad (5.72)$$

represents the displacement of the desired trajectory to the left or right of the robot.

5.18.3 Heading Error

$$\phi_{r,\text{dif}} \quad (5.73)$$

represents the difference between the desired and actual orientation.

Thus, the error vector can be represented as

$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{bmatrix} \quad (5.74)$$

This error vector forms the basis for the nonlinear kinematic controller.

5.19 Velocity Constraints

The robot cannot operate with unlimited velocity.

Therefore, the control commands are constrained according to the specified limits.

For the linear velocity,

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (5.75)$$

For the angular velocity,

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (5.76)$$

These saturation limits are part of the implemented control architecture.

The saturation ensures that the commands generated by the controller remain within the specified operating range.

5.20 Dynamic Model

The kinematic model describes the geometric motion of the robot but does not explicitly describe the forces and torques responsible for producing that motion.

The dynamic model considers the relationship between velocity changes and the corresponding force or torque required to produce them.

For the simplified planar motion considered in the project, the linear dynamic relationship can be represented conceptually as

$$F = m\dot{v} \quad (5.77)$$

where:

- F is the resultant force,
- m is the robot mass,
- $\dot{v}$ is the linear acceleration.

Similarly, the rotational dynamics can be represented as

$$\tau = J\dot{\omega} \quad (5.78)$$

where:

- τ is the applied torque,

- J is the rotational inertia,
- $\dot{\omega}$ is the angular acceleration.

The exact dynamic equations used in the Simulink implementation should be presented in the controller-design chapter according to the equations implemented in the model.

5.21 Kinematic and Dynamic Control Structure

The project uses a two-stage control structure.

5.21.1 Outer Kinematic Control

The kinematic controller receives the trajectory-tracking errors and desired velocities.

It generates

$$v_c, \quad \omega_c. \quad (5.79)$$

5.21.2 Inner Dynamic Velocity Control

The dynamic controller then works with the commanded velocities and actual velocity response.

Therefore, the control structure is

Trajectory Error $\rightarrow$ Kinematic Controller $\rightarrow (v_c, \omega_c) \rightarrow$ Dynamic Controller $\rightarrow$ Robot
--

(5.80)

The project material describes the dynamic controller as the second controller stage and identifies the comparison between commanded and actual velocities before the dynamic controller.

5.22 Mathematical Model Used in the Controller

The complete mathematical representation used for trajectory tracking can therefore be summarized as follows.

5.22.1 Robot State

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (5.81)$$

5.22.2 Control Input

$$\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (5.82)$$

5.22.3 Kinematic Model

$$\dot{x} = v \cos \phi \quad (5.83)$$

$$\dot{y} = v \sin \phi \quad (5.84)$$

$$\dot{\phi} = \omega \quad (5.85)$$

5.22.4 Tracking Error

$$e_x = x_d - x \quad (5.86)$$

$$e_y = y_d - y \quad (5.87)$$

$$e_\phi = \phi_d - \phi. \quad (5.88)$$

5.22.5 Robot-Frame Error

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (5.89)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (5.90)$$

This mathematical model provides the foundation for the nonlinear trajectory-tracking controller discussed in the next chapters.

5.23 Relationship Between Robot Model and Controller

The robot model and controller are directly connected.

The robot model describes

$$(v, \omega) \rightarrow (x, y, \phi). \quad (5.91)$$

The controller performs the reverse control operation:

$$(x_d, y_d, \phi_d) - (x, y, \phi) \rightarrow (v_c, \omega_c). \quad (5.92)$$

Thus,

Robot Model + Tracking Error + Control Law = Closed-Loop Trajectory Tracking
--

(5.93)

The controller continuously determines the required velocity based on the difference between the desired and actual states.

5.24 Summary

This chapter presented the mathematical modelling of TurtleBot3 used for the trajectory-tracking system.

TurtleBot3 is considered as a differential-drive mobile robot whose planar state is described by

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix}. \quad (5.94)$$

Its principal control inputs are the linear and angular velocities:

$$v, \quad \omega. \quad (5.95)$$

The kinematic model is given by

$$\dot{x} = v \cos \phi, \quad (5.96)$$

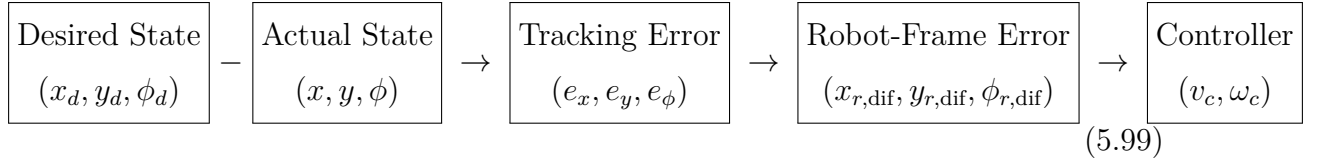
$$\dot{y} = v \sin \phi, \quad (5.97)$$

$$\dot{\phi} = \omega. \quad (5.98)$$

The model demonstrates the coupling between the robot's orientation and its translational motion.

The actual state is obtained through odometry, while the desired state is generated from the reference trajectory. Their difference produces the tracking error, which is transformed into the robot coordinate frame before being supplied to the nonlinear kinematic controller.

The complete modelling and control relationship can be summarized as:



This robot model provides the mathematical foundation for the **nonlinear kinematic controller and dynamic velocity controller** developed in the following chapters.

Odometry and Orientation Processing

6.1 Introduction

Accurate knowledge of the robot's current position and orientation is essential for trajectory tracking. The controller must continuously compare the actual state of TurtleBot3 with the desired state generated by the reference trajectory. In the proposed system, this actual-state information is obtained through the **ROS 2 odometry feedback**.

The odometry information received by the Simulink model contains the robot's position and orientation. The position information is extracted directly, whereas the orientation is received in quaternion form and must be converted into the yaw angle required by the trajectory-tracking controller.

The complete state-processing sequence is:

$$\text{ROS 2 /odom} \rightarrow \text{Odometry Message} \rightarrow \text{State Extraction} \rightarrow \text{Quaternion Processing} \rightarrow \text{Yaw} \quad (6.1)$$

The resulting actual state is represented by:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (6.2)$$

where x and y represent the robot's position and ϕ represents its yaw angle.

6.2 Odometry

Odometry is the process of obtaining information about the robot's position and orientation based on its motion.

In this project, odometry is used as the primary feedback source for the trajectory-tracking controller. The TurtleBot3 system provides odometry information through ROS 2, which is received by the Simulink model.

The important state quantities required by the controller are:

$$x_{\text{real}} \quad (6.3)$$

$$y_{\text{real}} \quad (6.4)$$

and

$$\phi. \quad (6.5)$$

Thus, the odometry-processing system provides:

$$\boxed{\text{Actual Position} + \text{Actual Orientation}} \quad (6.6)$$

to the controller.

6.3 ROS 2 /odom Feedback

The robot-state information is transferred from the robot/simulation environment to Simulink using ROS 2.

The feedback path is:

$$\boxed{\text{TurtleBot3} \rightarrow \text{/odom} \rightarrow \text{ROS 2 Subscriber} \rightarrow \text{Simulink}} \quad (6.7)$$

The ROS 2 Subscriber block receives the odometry message and provides the required state information to the rest of the Simulink model.

The /odom feedback path and the extraction of actual position and quaternion orientation form an important part of the implementation.

6.4 Odometry Message Processing

Once the odometry message reaches the Simulink model, the required information is separated from the received message.

The position information is represented by:

$$x_{\text{real}}, \quad y_{\text{real}}. \quad (6.8)$$

The orientation information is represented using four quaternion components:

$$q_x, \quad q_y, \quad q_z, \quad q_w. \quad (6.9)$$

Therefore, the important information extracted from the odometry message can be represented as:

$$\boxed{\{x_{\text{real}}, y_{\text{real}}, q_x, q_y, q_z, q_w\}} \quad (6.10)$$

The project material explicitly identifies these variables as the relevant subscriber outputs used for the controller.

6.5 Position Extraction

The robot's position is required for calculating the difference between the actual and desired trajectory.

The extracted coordinates are:

$$x = x_{\text{real}} \quad (6.11)$$

and

$$y = y_{\text{real}}. \quad (6.12)$$

Thus, the actual position vector is:

$$\mathbf{p} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \end{bmatrix} \quad (6.13)$$

These signals are passed to the trajectory-error calculation stage.

The position information therefore follows the path:

$$\boxed{/odom \rightarrow \text{Subscriber} \rightarrow x_{\text{real}}, y_{\text{real}} \rightarrow \text{Error Calculation}} \quad (6.14)$$

6.6 Orientation Representation

The orientation of TurtleBot3 is received in quaternion form.

The quaternion is written as:

$$\boxed{q = [q_x, q_y, q_z, q_w]} \quad (6.15)$$

where:

- q_x is the x -component,
- q_y is the y -component,
- q_z is the z -component,
- q_w is the scalar component.

The quaternion provides a representation of the robot's three-dimensional orientation.

The quaternion components obtained from the subscriber are supplied to the quaternion-to-Euler conversion stage.

6.7 Need for Quaternion-to-Yaw Conversion

Although quaternion representation is useful for describing three-dimensional orientation, the trajectory-tracking controller used in this project requires the robot's heading angle.

TurtleBot3 is considered to be moving primarily in a two-dimensional plane. Therefore, the important orientation quantity is the rotation about the vertical z -axis.

This angle is the **yaw angle**, represented by:

$$\phi. \quad (6.16)$$

The quaternion is converted into Euler angles and yaw is the required quantity for the controller.

Therefore:

$$\boxed{\text{Quaternion} \rightarrow \text{Euler Angles} \rightarrow \text{Yaw}} \quad (6.17)$$

6.8 Quaternion-to-Euler Conversion

The Simulink model uses the **QUAT2ZYX** block to convert the received quaternion into ZYX Euler-angle representation.

The conversion can be represented as:

$$[q_x, q_y, q_z, q_w] \rightarrow [\text{Yaw}, \text{Pitch}, \text{Roll}]. \quad (6.18)$$

The ZYX convention represents the sequence of rotations about the Z , Y , and X axes.

For the planar mobile-robot problem, the required output is:

$$\boxed{\phi = \text{Yaw}} \quad (6.19)$$

while pitch and roll are not the primary variables used in the trajectory-tracking controller.

6.9 Yaw Angle

The yaw angle represents the orientation of TurtleBot3 with respect to the global coordinate frame.

It is denoted by:

$$\phi. \quad (6.20)$$

The complete actual robot state can therefore be written as:

$$\boxed{\mathbf{q}_{\text{actual}} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix}} \quad (6.21)$$

This state is used by the controller to determine the robot's deviation from the desired trajectory.

The yaw angle is particularly important because the robot's direction of motion depends on its orientation.

6.10 Relationship Between Orientation and Robot Motion

The kinematic model of TurtleBot3 is:

$$\dot{x} = v \cos \phi \quad (6.22)$$

$$\dot{y} = v \sin \phi \quad (6.23)$$

$$\dot{\phi} = \omega. \quad (6.24)$$

These equations show that the robot's orientation directly affects its motion in the global coordinate system.

For example, the x -direction velocity is:

$$\dot{x} = v \cos \phi. \quad (6.25)$$

Similarly, the y -direction velocity is:

$$\dot{y} = v \sin \phi. \quad (6.26)$$

Therefore, accurate measurement of ϕ is essential for determining the robot's actual direction of motion and for transforming the tracking error into the robot coordinate frame.

6.11 Actual Robot State

After processing the odometry message, the actual robot state is available as:

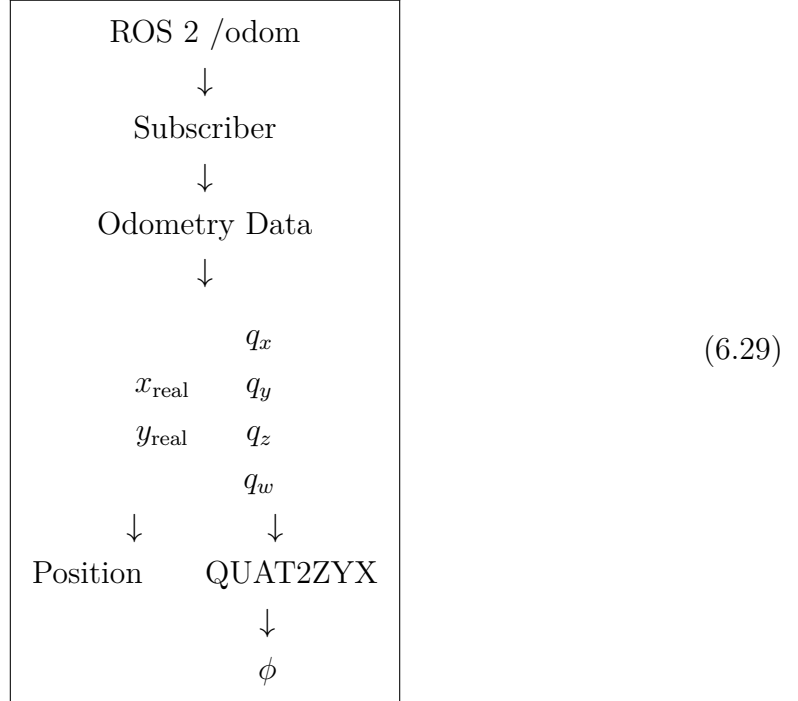
$$x_{\text{real}}, \quad y_{\text{real}}, \quad \phi. \quad (6.27)$$

The state vector is:

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix} \quad (6.28)$$

This state forms the feedback input to the trajectory-tracking controller.

The complete state-processing path is:



6.12 Desired State

The actual state obtained from odometry is compared with the desired state generated by the trajectory generator.

The desired state is:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix} \quad (6.30)$$

where:

$$x_d = R \cos(\omega_d t) \quad (6.31)$$

$$y_d = R \sin(\omega_d t) \quad (6.32)$$

and

$$\phi_d = \omega_d t. \quad (6.33)$$

The trajectory parameters used in the project are:

$$R = 0.4 \text{ m} \quad (6.34)$$

and

$$\omega_d = 0.5 \text{ rad/s}. \quad (6.35)$$

6.13 Initial State Synchronization

At the beginning of the trajectory, the reference is synchronized with the robot's current state.

Therefore:

$$x_d(0) = x_{\text{real}}(0) \quad (6.36)$$

$$y_d(0) = y_{\text{real}}(0) \quad (6.37)$$

and

$$\phi_d(0) = \phi(0). \quad (6.38)$$

This provides a consistent starting condition between the desired and actual states.

The initial-state synchronization forms part of the reference trajectory generation process.

6.14 Position Tracking Error

Once the actual and desired positions are available, the position errors are calculated.

The x -direction error is:

$$e_x = x_d - x_{\text{real}} \quad (6.39)$$

The y -direction error is:

$$e_y = y_d - y_{\text{real}} \quad (6.40)$$

These errors represent the difference between the desired and actual robot positions.

For example:

- $e_x > 0$: desired position lies in the positive X -direction relative to the actual position.
- $e_x < 0$: desired position lies in the negative X -direction.
- $e_y > 0$: desired position lies in the positive Y -direction.
- $e_y < 0$: desired position lies in the negative Y -direction.

6.15 Orientation Tracking Error

The desired orientation is compared with the actual yaw:

$$e_\phi = \phi_d - \phi \quad (6.41)$$

This error indicates the difference between the robot's desired and actual heading.

The three tracking errors can therefore be represented as:

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (6.42)$$

The position and orientation differences form the tracking-error signals supplied to the subsequent transformation and control stages.

6.16 Global-Frame Error

The errors e_x and e_y calculated above are initially represented in the global coordinate frame.

Therefore:

$$\mathbf{e}_p = \begin{bmatrix} e_x \\ e_y \end{bmatrix}. \quad (6.43)$$

However, the robot itself may be oriented at an arbitrary angle ϕ .

Consequently, the controller requires the error relative to the robot's own orientation.

This leads to the next processing stage: **global-to-robot frame transformation**.

6.17 Global-to-Robot Frame Transformation

The global position errors are transformed using the robot's current yaw angle.

The transformation is:

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \end{bmatrix} = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} e_x \\ e_y \end{bmatrix} \quad (6.44)$$

Therefore:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (6.45)$$

and

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (6.46)$$

The orientation error is:

$$\phi_{r,\text{dif}} = e_\phi \quad (6.47)$$

The transformation converts the global tracking errors into the robot's local coordinate frame.

6.18 Physical Meaning of the Transformed Errors

The robot-frame errors have a direct physical interpretation.

6.18.1 Forward/Backward Error

$$x_{r,\text{dif}} \quad (6.48)$$

represents the tracking error along the robot's forward direction.

6.18.2 Lateral Error

$$y_{r,\text{dif}} \quad (6.49)$$

represents the tracking error perpendicular to the robot's forward direction.

6.18.3 Heading Error

$$\phi_{r,\text{dif}} \quad (6.50)$$

represents the orientation difference between the desired and actual robot heading.

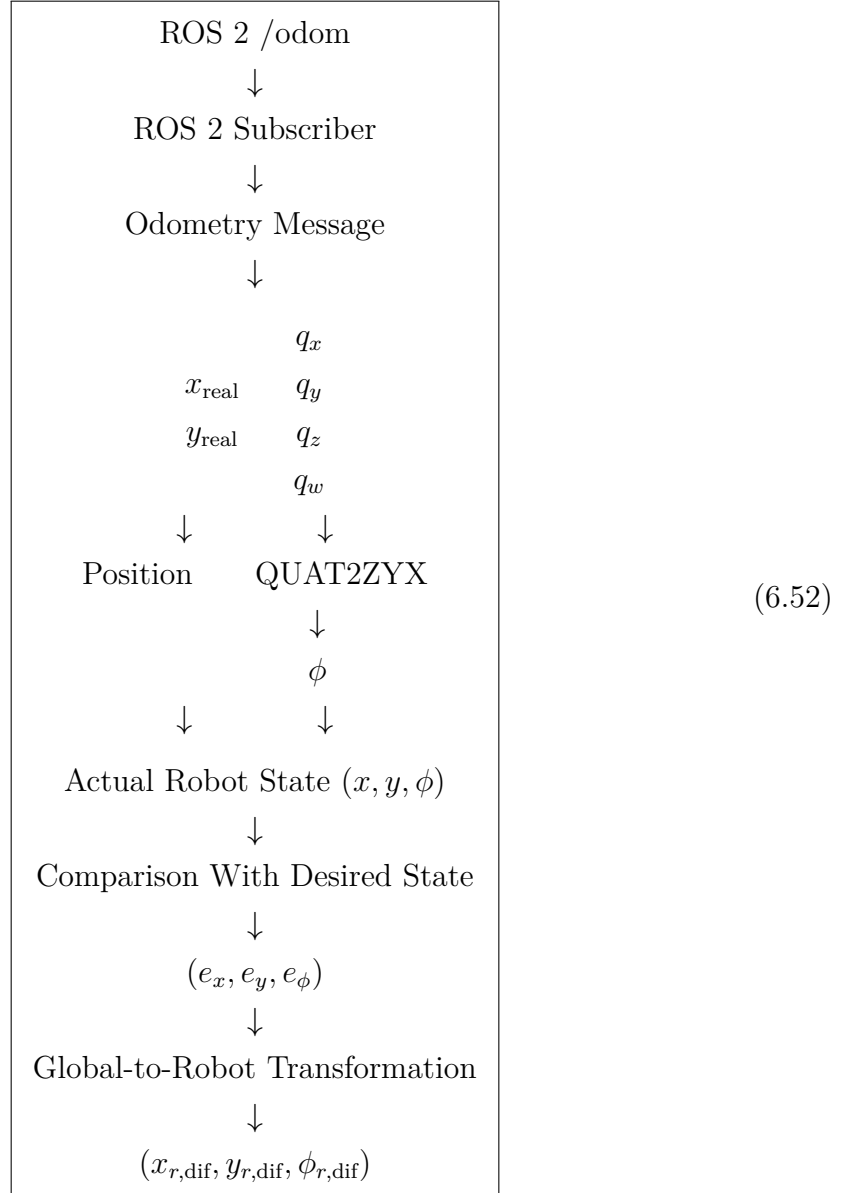
Therefore:

$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{bmatrix} \quad (6.51)$$

becomes the error representation used by the nonlinear trajectory-tracking controller.

6.19 Complete State-Processing Chain

The complete odometry and orientation processing system can be summarized as:



This processed error vector is then supplied to the nonlinear kinematic controller.

6.20 Importance of Accurate State Processing

Accurate state processing is essential because any error in the measured position or orientation directly affects the calculated tracking error.

For example, an error in the yaw angle affects the coordinate transformation:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (6.53)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (6.54)$$

Therefore, an incorrect value of ϕ can result in incorrect forward and lateral error values.

These incorrect errors may then produce inappropriate linear and angular velocity commands.

Thus, the complete control performance depends on the quality of the state information.

6.21 Relationship With the Kinematic Controller

The final outputs of the state-processing section are:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}. \quad (6.55)$$

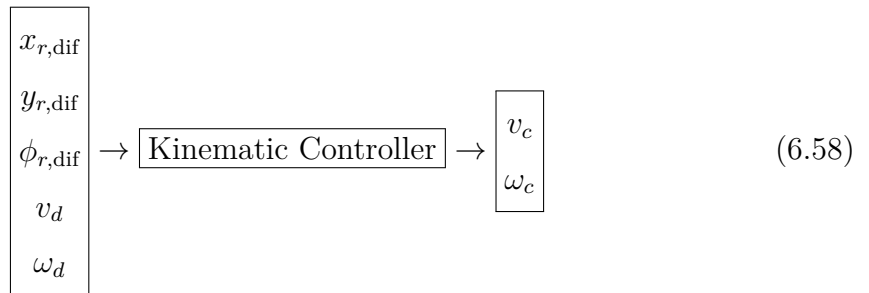
These are supplied to the nonlinear kinematic controller along with the desired velocity information:

$$v_d \quad (6.56)$$

and

$$\omega_d. \quad (6.57)$$

Therefore:



The controller then generates the commands required to reduce the tracking errors.

6.22 Summary

This chapter presented the odometry and orientation-processing stages of the TurtleBot3 trajectory-tracking system.

The robot's actual position and orientation are obtained through ROS 2 odometry. The position variables are:

$$x_{\text{real}}, \quad y_{\text{real}}, \quad (6.59)$$

while the orientation is received as the quaternion:

$$[q_x, q_y, q_z, q_w]. \quad (6.60)$$

The quaternion is processed using the **QUAT2ZYX** block to obtain the robot's yaw angle:

$$\phi. \quad (6.61)$$

The complete actual state is therefore:

$$\mathbf{q} = \begin{bmatrix} x_{\text{real}} \\ y_{\text{real}} \\ \phi \end{bmatrix} \quad (6.62)$$

This actual state is compared with the desired trajectory state:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix} \quad (6.63)$$

to calculate:

$$e_x = x_d - x_{\text{real}}, \quad (6.64)$$

$$e_y = y_d - y_{\text{real}}, \quad (6.65)$$

and

$$e_\phi = \phi_d - \phi. \quad (6.66)$$

The position errors are subsequently transformed into the robot coordinate frame:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (6.67)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (6.68)$$

The resulting robot-frame error signals are then supplied to the nonlinear kinematic controller.

Hence, the complete processing chain is:

$$\boxed{\text{ROS 2 /odom} \rightarrow \text{Position and Quaternion} \rightarrow \text{Yaw} \rightarrow \text{Actual State} \rightarrow \text{Tracking Error} \rightarrow \text{Robot-Fra}} \quad (6.69)$$

This state-processing stage provides the necessary feedback information for the trajectory-tracking controller developed in the subsequent chapter.

Reference Trajectory Generation

7.1 Introduction

Trajectory generation is an important stage in the proposed TurtleBot3 trajectory-tracking system. The purpose of the trajectory generator is to provide the controller with a continuously varying **desired position and orientation** that the robot is required to follow.

In the proposed system, a **circular trajectory** is selected as the reference path. The trajectory is generated as a function of time and provides the desired values of:

$$x_d(t), \quad y_d(t), \quad \phi_d(t). \quad (7.1)$$

The desired trajectory is then compared with the actual robot state obtained through ROS 2 odometry. The resulting differences are used to calculate the tracking errors.

The complete reference-generation process can be represented as:

$$\boxed{t \rightarrow \text{Trajectory Generator} \rightarrow (x_d, y_d, \phi_d) \rightarrow \text{Error Calculation}} \quad (7.2)$$

The circular trajectory parameters specified in the project material are:

$$R = 0.4 \text{ m} \quad (7.3)$$

and

$$\omega_d = 0.5 \text{ rad/s}. \quad (7.4)$$

The supplied material identifies circular trajectory generation, desired x - and y -positions, desired orientation, and the corresponding desired velocities as part of the trajectory-tracking implementation.

7.2 Purpose of Reference Trajectory

The reference trajectory defines the motion that TurtleBot3 is expected to perform.

A trajectory differs from a fixed target position because the desired state changes continuously with time.

The desired state is represented as:

$$\mathbf{q}_d(t) = \begin{bmatrix} x_d(t) \\ y_d(t) \\ \phi_d(t) \end{bmatrix} \quad (7.5)$$

The controller attempts to make the actual state

$$\mathbf{q}(t) = \begin{bmatrix} x(t) \\ y(t) \\ \phi(t) \end{bmatrix} \quad (7.6)$$

follow the desired state.

Therefore:

$$\mathbf{q}(t) \rightarrow \mathbf{q}_d(t) \quad (7.7)$$

during the entire motion.

7.3 Selection of Circular Trajectory

A circular trajectory is selected for evaluating the performance of the trajectory-tracking controller.

A circular path is useful because it continuously changes both the position and orientation of the robot. Consequently, the controller must simultaneously handle:

- x -position tracking,
- y -position tracking,

- heading tracking,
- linear velocity,
- angular velocity.

The desired circular path is defined using the radius R and angular velocity ω_d .

The trajectory parameters used in the project are:

$$\boxed{R = 0.4 \text{ m}} \quad (7.8)$$

and

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (7.9)$$

as specified in the project material.

7.4 Mathematical Representation of Circular Trajectory

The desired x -coordinate is generated using:

$$\boxed{x_d(t) = R \cos(\omega_d t)} \quad (7.10)$$

The desired y -coordinate is:

$$\boxed{y_d(t) = R \sin(\omega_d t)} \quad (7.11)$$

These equations describe a circular path with radius R .

The trajectory satisfies the circle equation:

$$x_d^2 + y_d^2 = R^2. \quad (7.12)$$

Substituting the trajectory equations:

$$x_d^2 + y_d^2 = R^2 \cos^2(\omega_d t) + R^2 \sin^2(\omega_d t) \quad (7.13)$$

$$= R^2 [\cos^2(\omega_d t) + \sin^2(\omega_d t)] \quad (7.14)$$

Using

$$\cos^2 \theta + \sin^2 \theta = 1, \quad (7.15)$$

we obtain:

$$\boxed{x_d^2 + y_d^2 = R^2} \quad (7.16)$$

which confirms that the generated reference is circular.

7.5 Desired Orientation

In addition to the desired position, the robot must maintain the required orientation while following the circular trajectory.

The desired orientation is represented by:

$$\boxed{\phi_d(t) = \omega_d t} \quad (7.17)$$

The orientation changes continuously as the robot moves around the circular path.

The desired angular velocity is therefore:

$$\boxed{\omega_d = \frac{d\phi_d}{dt}} \quad (7.18)$$

and hence:

$$\boxed{\omega_d = \omega_d} \quad (7.19)$$

where the same symbol represents the constant reference angular velocity.

For the project:

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (7.20)$$

7.6 Desired Linear Velocity

The desired linear velocity can be obtained by differentiating the desired trajectory.

Starting with:

$$x_d = R \cos(\omega_d t) \quad (7.21)$$

differentiate with respect to time:

$$\dot{x}_d = -R\omega_d \sin(\omega_d t). \quad (7.22)$$

Similarly,

$$y_d = R \sin(\omega_d t) \quad (7.23)$$

gives:

$$\dot{y}_d = R\omega_d \cos(\omega_d t). \quad (7.24)$$

The magnitude of the desired velocity is therefore:

$$v_d = \sqrt{\dot{x}_d^2 + \dot{y}_d^2}. \quad (7.25)$$

Substituting:

$$v_d = \sqrt{R^2\omega_d^2 \sin^2(\omega_d t) + R^2\omega_d^2 \cos^2(\omega_d t)}. \quad (7.26)$$

Taking the common terms:

$$v_d = R\omega_d \sqrt{\sin^2(\omega_d t) + \cos^2(\omega_d t)}. \quad (7.27)$$

Using:

$$\sin^2 \theta + \cos^2 \theta = 1, \quad (7.28)$$

we obtain:

$$\boxed{v_d = R\omega_d} \quad (7.29)$$

For the specified trajectory:

$$v_d = (0.4)(0.5) \quad (7.30)$$

Therefore:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (7.31)$$

Therefore, the reference trajectory provides:

$$\boxed{v_d = 0.2 \text{ m/s}, \quad \omega_d = 0.5 \text{ rad/s.}} \quad (7.32)$$

7.7 Reference Trajectory Parameters

The main trajectory parameters can be summarized as follows:

Table 7.1: Reference Trajectory Parameters

Parameter	Symbol	Value
Radius	R	0.4 m
Desired angular velocity	ω_d	0.5 rad/s
Desired linear velocity	v_d	0.2 m/s

These parameters determine the size and rate of traversal of the desired circular trajectory.

7.8 Initial Conditions

The initial conditions of the reference trajectory are important because the robot may not initially be located at the origin.

To avoid a large initial tracking error, the desired trajectory is synchronized with the robot's initial state.

Thus:

$$\boxed{x_d(0) = x_{\text{real}}(0)} \quad (7.33)$$

$$\boxed{y_d(0) = y_{\text{real}}(0)} \quad (7.34)$$

and

$$\boxed{\phi_d(0) = \phi(0)} \quad (7.35)$$

where $x_{\text{real}}(0)$, $y_{\text{real}}(0)$, and $\phi(0)$ represent the initial actual robot state.

The initialization of the reference uses the robot's current state at the beginning of the trajectory.

7.9 Importance of Initial Synchronization

If the desired trajectory were started from a fixed location that was significantly different from the robot's initial location, the initial tracking error could be large.

For example:

$$e_x(0) = x_d(0) - x(0) \quad (7.36)$$

and

$$e_y(0) = y_d(0) - y(0). \quad (7.37)$$

A large initial error could result in large control commands and a large transient response.

By setting:

$$x_d(0) = x(0) \quad (7.38)$$

and

$$y_d(0) = y(0), \quad (7.39)$$

the initial position errors become:

$$e_x(0) = 0 \quad (7.40)$$

and

$$e_y(0) = 0. \quad (7.41)$$

Similarly, setting:

$$\phi_d(0) = \phi(0) \quad (7.42)$$

gives:

$$e_\phi(0) = 0. \quad (7.43)$$

This creates a consistent starting condition for the trajectory-tracking controller.

7.10 MATLAB Function Implementation

The reference trajectory is implemented in the Simulink model using a MATLAB Function block.

The function receives the required state/time information and generates the desired trajectory variables.

The main outputs are:

$$x_d \quad (7.44)$$

$$y_d \quad (7.45)$$

and

$$\phi_d. \quad (7.46)$$

The trajectory generator therefore acts as the reference source for the complete control system.

The basic implementation can be represented as:

$$\boxed{\text{Time} \rightarrow \text{MATLAB Function} \rightarrow \begin{cases} x_d \\ y_d \\ \phi_d \end{cases}} \quad (7.47)$$

These outputs are then supplied to the error-calculation blocks.

7.11 Reference Trajectory and Actual Trajectory

The reference trajectory represents the ideal path, whereas the actual trajectory represents the motion obtained from the robot.

The desired trajectory is:

$$\mathbf{p}_d(t) = \begin{bmatrix} x_d(t) \\ y_d(t) \end{bmatrix}. \quad (7.48)$$

The actual trajectory is:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}. \quad (7.49)$$

The objective is to make:

$$\boxed{\mathbf{p}(t) \rightarrow \mathbf{p}_d(t)} \quad (7.50)$$

as closely as possible.

The difference between these trajectories provides the position tracking error.

7.12 Calculation of Position Errors

The x -direction tracking error is:

$$\boxed{e_x = x_d - x} \quad (7.51)$$

and the y -direction tracking error is:

$$\boxed{e_y = y_d - y} \quad (7.52)$$

These errors are calculated continuously during the simulation.

The position-error vector can therefore be written as:

$$\boxed{\mathbf{e}_p = \begin{bmatrix} e_x \\ e_y \end{bmatrix}} \quad (7.53)$$

7.13 Calculation of Orientation Error

The desired orientation is:

$$\phi_d \quad (7.54)$$

and the actual orientation obtained from odometry is:

$$\phi. \quad (7.55)$$

The orientation error is therefore:

$$e_\phi = \phi_d - \phi \quad (7.56)$$

This error indicates how much the robot's heading differs from the desired heading.

The complete error vector is:

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (7.57)$$

This error is then supplied to the coordinate-transformation stage.

7.14 Reference Velocity as Feedforward Information

The trajectory generator also provides the desired velocities:

$$v_d \quad (7.58)$$

and

$$\omega_d. \quad (7.59)$$

These quantities are important because the controller does not rely only on the tracking error.

The desired velocities provide information about the motion that the robot should ideally perform.

Therefore, the controller combines:

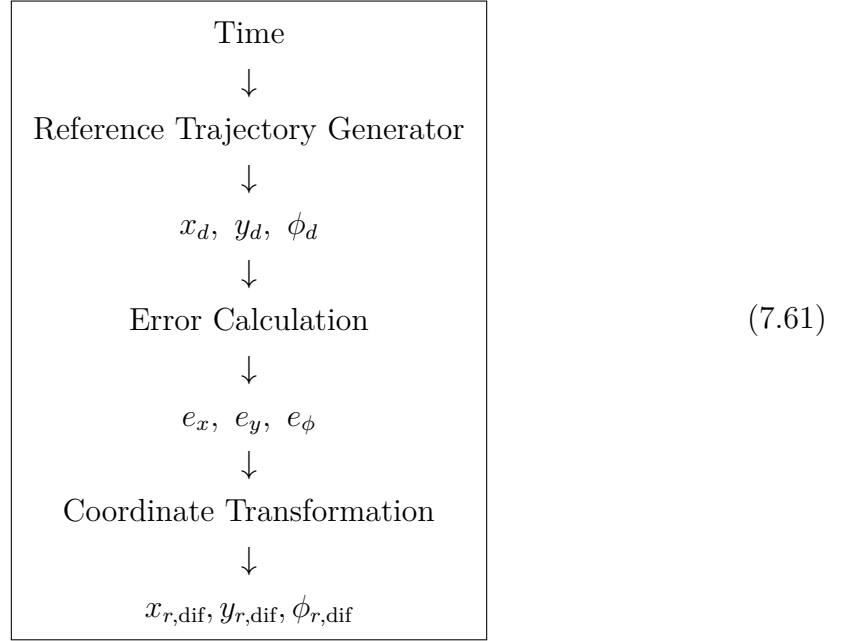
$$\boxed{\text{Tracking Error Feedback} + \text{Desired Trajectory Feedforward}} \quad (7.60)$$

This improves the controller's ability to follow a continuously changing trajectory.

7.15 Reference Trajectory in Simulink

In the complete Simulink model, the reference trajectory-generation block is connected to the controller through several intermediate stages.

The signal flow is:



The desired velocities are supplied to the controller along with the transformed tracking errors.

7.16 Relationship Between Trajectory Generation and Controller

The trajectory generator and controller have complementary functions.

The trajectory generator determines:

$$\boxed{\text{What the robot should do}} \quad (7.62)$$

while the controller determines:

$$\boxed{\text{What command should be applied to make it happen}} \quad (7.63)$$

Therefore:

$$\boxed{\text{Reference Trajectory} \rightarrow \text{Error Calculation} \rightarrow \text{Controller}} \quad (7.64)$$

The controller uses the reference information and the actual robot state to calculate:

$$v_c \quad (7.65)$$

and

$$\omega_c. \quad (7.66)$$

7.17 Circular Trajectory Motion

For the selected trajectory:

$$x_d = 0.4 \cos(0.5t) \quad (7.67)$$

and

$$y_d = 0.4 \sin(0.5t). \quad (7.68)$$

The robot therefore moves around a circle of radius:

$$0.4 \text{ m}. \quad (7.69)$$

The desired angular rate is:

$$0.5 \text{ rad/s}. \quad (7.70)$$

The corresponding desired linear velocity is:

$$0.2 \text{ m/s}. \quad (7.71)$$

The time required for one complete revolution can be obtained from:

$$T = \frac{2\pi}{\omega_d}. \quad (7.72)$$

Substituting:

$$T = \frac{2\pi}{0.5} \quad (7.73)$$

Therefore:

$$\boxed{T = 4\pi \approx 12.57 \text{ s}} \quad (7.74)$$

Thus, under the ideal reference conditions, one complete circular revolution takes approximately 12.57 seconds.

7.18 Trajectory Generation Algorithm

The reference trajectory generation process can be summarized as follows.

Step 1

Obtain the current robot state:

$$x(0), \quad y(0), \quad \phi(0). \quad (7.75)$$

Step 2

Set the initial desired state equal to the actual state:

$$x_d(0) = x(0) \quad (7.76)$$

$$y_d(0) = y(0) \quad (7.77)$$

$$\phi_d(0) = \phi(0). \quad (7.78)$$

Step 3

Define the circular trajectory parameters:

$$R = 0.4 \text{ m} \quad (7.79)$$

$$\omega_d = 0.5 \text{ rad/s.} \quad (7.80)$$

Step 4

Generate the desired position:

$$x_d = R \cos(\omega_d t) \quad (7.81)$$

$$y_d = R \sin(\omega_d t). \quad (7.82)$$

Step 5

Generate the desired orientation:

$$\phi_d = \omega_d t. \quad (7.83)$$

Step 6

Calculate the desired linear velocity:

$$v_d = R\omega_d. \quad (7.84)$$

Step 7

Provide the desired state and velocity to the tracking controller.

Thus:

$$\boxed{(x_d, y_d, \phi_d, v_d, \omega_d)} \quad (7.85)$$

forms the reference information for the controller.

7.19 Reference Generation and Closed-Loop Tracking

The reference trajectory does not directly control the robot.

Instead, it provides a target for the closed-loop controller.

The actual robot state is continuously measured:

$$(x, y, \phi). \quad (7.86)$$

The reference generator provides:

$$(x_d, y_d, \phi_d). \quad (7.87)$$

The error is then calculated:

$$\mathbf{e} = \mathbf{q}_d - \mathbf{q} \quad (7.88)$$

and used by the controller.

The resulting command changes the robot motion, which changes the next measured state.

Therefore:

$$\boxed{\text{Reference} \rightarrow \text{Error} \rightarrow \text{Control} \rightarrow \text{Robot Motion} \rightarrow \text{New Error}} \quad (7.89)$$

This process repeats continuously.

7.20 Summary of Reference Trajectory

The complete reference trajectory used in this project can be summarized by:

$$\boxed{x_d(t) = 0.4 \cos(0.5t)} \quad (7.90)$$

$$\boxed{y_d(t) = 0.4 \sin(0.5t)} \quad (7.91)$$

$$\boxed{\phi_d(t) = 0.5t} \quad (7.92)$$

with:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (7.93)$$

and

$$\boxed{\omega_d = 0.5 \text{ rad/s.}} \quad (7.94)$$

The desired trajectory is initialized according to the robot's initial state so that the trajectory-tracking process begins with consistent position and orientation conditions.

7.21 Summary

This chapter presented the generation of the reference trajectory used for TurtleBot3 trajectory tracking.

A circular trajectory was selected with:

$$R = 0.4 \text{ m} \quad (7.95)$$

and

$$\omega_d = 0.5 \text{ rad/s.} \quad (7.96)$$

The desired position is generated using:

$$x_d = R \cos(\omega_d t) \quad (7.97)$$

and

$$y_d = R \sin(\omega_d t). \quad (7.98)$$

The desired orientation is:

$$\phi_d = \omega_d t. \quad (7.99)$$

The desired linear velocity is obtained as:

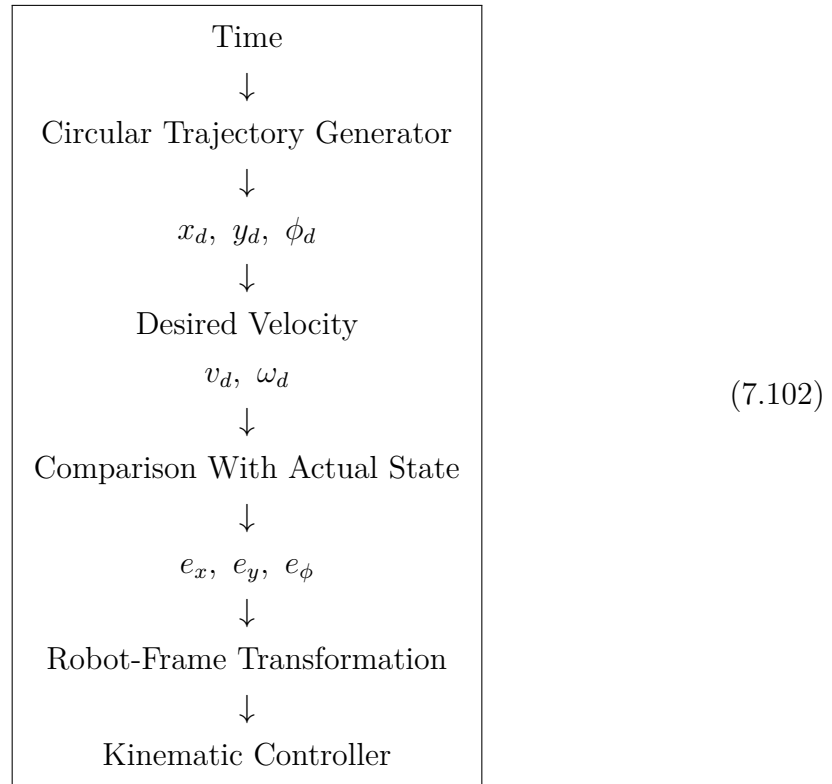
$$v_d = R\omega_d = 0.2 \text{ m/s.} \quad (7.100)$$

The desired angular velocity is:

$$\omega_d = 0.5 \text{ rad/s.} \quad (7.101)$$

The reference is synchronized with the initial robot state, ensuring that the desired and actual states start from consistent conditions.

The overall reference-generation process is:



Thus, the reference trajectory generator provides the **desired position, orientation, and velocity information** required by the subsequent tracking-error calculation and nonlinear controller.

Tracking Error Calculation

8.1 Introduction

Tracking-error calculation is one of the most important stages of the proposed TurtleBot3 trajectory-tracking system. The purpose of this stage is to determine the difference between the **desired trajectory** and the **actual trajectory** of the robot.

The desired trajectory is generated using the reference trajectory generator, while the actual robot position and orientation are obtained from ROS 2 odometry. These two sets of information are continuously compared to determine the instantaneous tracking error.

The basic principle is:

$$\boxed{\text{Tracking Error} = \text{Desired State} - \text{Actual State}} \quad (8.1)$$

For the TurtleBot3 considered in this project, three primary errors are calculated:

$$e_x, \quad e_y, \quad e_\phi \quad (8.2)$$

where:

- e_x = position error along the global X -axis,
- e_y = position error along the global Y -axis,
- e_ϕ = orientation or heading error.

The calculation of position and orientation differences is performed before transforming the errors into the robot coordinate frame.

8.2 Purpose of Tracking-Error Calculation

The main purpose of calculating the tracking error is to provide the controller with information about the deviation of the robot from the desired trajectory.

If the actual robot is exactly on the desired trajectory and has the correct orientation, then:

$$e_x = 0 \quad (8.3)$$

$$e_y = 0 \quad (8.4)$$

and

$$e_\phi = 0. \quad (8.5)$$

Therefore, the ideal tracking condition is:

$$\boxed{e_x \rightarrow 0, \quad e_y \rightarrow 0, \quad e_\phi \rightarrow 0} \quad (8.6)$$

The controller uses these errors to generate corrective velocity commands.

The overall process is:

$$\boxed{\text{Desired State} - \text{Actual State} \rightarrow \text{Tracking Error} \rightarrow \text{Controller}} \quad (8.7)$$

8.3 Desired Robot State

The desired state is obtained from the reference trajectory generator.

It consists of:

$$x_d, \quad y_d, \quad \phi_d. \quad (8.8)$$

Thus, the desired state vector is:

$$\boxed{\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix}} \quad (8.9)$$

For the circular trajectory used in the project:

$$x_d = R \cos(\omega_d t) \quad (8.10)$$

$$y_d = R \sin(\omega_d t) \quad (8.11)$$

and

$$\phi_d = \omega_d t. \quad (8.12)$$

The trajectory parameters are:

$$R = 0.4 \text{ m} \quad (8.13)$$

and

$$\omega_d = 0.5 \text{ rad/s}. \quad (8.14)$$

8.4 Actual Robot State

The actual robot state is obtained from the ROS 2 odometry feedback.

The actual position is:

$$x = x_{\text{real}} \quad (8.15)$$

and

$$y = y_{\text{real}}. \quad (8.16)$$

The actual orientation is obtained by converting the quaternion received from odometry into yaw:

$$\phi = \text{Yaw}. \quad (8.17)$$

Therefore, the actual state vector is:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (8.18)$$

This state is continuously updated as the robot moves.

8.5 General Error Vector

The difference between the desired and actual states can be represented by:

$$\mathbf{e} = \mathbf{q}_d - \mathbf{q}. \quad (8.19)$$

Therefore:

$$\mathbf{e} = \begin{bmatrix} x_d - x \\ y_d - y \\ \phi_d - \phi \end{bmatrix} \quad (8.20)$$

This can be expanded into the three individual error equations.

8.6 X-Position Error

The X -direction tracking error is defined as:

$$e_x = x_d - x \quad (8.21)$$

where:

- x_d is the desired X -position,
- x is the actual X -position.

If:

$$e_x > 0, \quad (8.22)$$

the desired position lies in the positive X -direction relative to the actual position.

If:

$$e_x < 0, \quad (8.23)$$

the desired position lies in the negative X -direction.

If:

$$e_x = 0, \quad (8.24)$$

the actual and desired X -coordinates are identical.

8.7 Y -Position Error

The Y -direction tracking error is:

$$\boxed{e_y = y_d - y} \quad (8.25)$$

where:

- y_d is the desired Y -position,
- y is the actual Y -position.

If:

$$e_y > 0, \quad (8.26)$$

the desired position is in the positive Y -direction relative to the actual position.

If:

$$e_y < 0, \quad (8.27)$$

the desired position is in the negative Y -direction.

If:

$$e_y = 0, \quad (8.28)$$

the actual and desired Y -coordinates are identical.

8.8 Orientation Error

The orientation error is calculated using the desired and actual heading angles:

$$e_\phi = \phi_d - \phi \quad (8.29)$$

where:

- ϕ_d = desired yaw,
- ϕ = actual yaw.

The orientation error indicates how much the robot must change its heading to align with the desired orientation.

If:

$$e_\phi > 0, \quad (8.30)$$

the robot needs to correct its orientation in one direction.

If:

$$e_\phi < 0, \quad (8.31)$$

the correction is required in the opposite direction.

If:

$$e_\phi = 0, \quad (8.32)$$

the actual and desired orientations are equal.

8.9 Complete Tracking-Error Vector

The three errors can be combined into a single error vector:

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (8.33)$$

or:

$$\mathbf{e} = \begin{bmatrix} x_d - x \\ y_d - y \\ \phi_d - \phi \end{bmatrix} \quad (8.34)$$

This vector represents the difference between the desired and actual robot states.

The controller does not simply need the magnitude of the total error. It needs to know the individual components so that the appropriate linear and angular corrections can be generated.

8.10 Why Three Errors Are Required

A mobile robot has both translational and rotational motion.

Therefore, tracking only the position would not be sufficient.

For example, two robots could have:

$$x = x_d \quad (8.35)$$

and

$$y = y_d, \quad (8.36)$$

while having different orientations:

$$\phi \neq \phi_d. \quad (8.37)$$

In this situation, the robot is at the correct position but is not correctly aligned with the desired trajectory.

Therefore, the controller must consider:

$$\boxed{\text{Position Error} + \text{Orientation Error}} \quad (8.38)$$

This is why e_x , e_y , and e_ϕ are all required.

8.11 Example of Error Calculation

Consider an instantaneous state where:

$$x_d = 0.30 \text{ m} \quad (8.39)$$

and

$$x = 0.25 \text{ m}. \quad (8.40)$$

Then:

$$e_x = x_d - x \quad (8.41)$$

$$e_x = 0.30 - 0.25 \quad (8.42)$$

Therefore:

$$\boxed{e_x = 0.05 \text{ m}} \quad (8.43)$$

Similarly, if:

$$y_d = 0.20 \text{ m} \quad (8.44)$$

and

$$y = 0.24 \text{ m}, \quad (8.45)$$

then:

$$e_y = 0.20 - 0.24 \quad (8.46)$$

Therefore:

$$\boxed{e_y = -0.04 \text{ m}} \quad (8.47)$$

If:

$$\phi_d = 0.50 \text{ rad} \quad (8.48)$$

and

$$\phi = 0.40 \text{ rad}, \quad (8.49)$$

then:

$$e_\phi = 0.50 - 0.40 \quad (8.50)$$

Therefore:

$$\boxed{e_\phi = 0.10 \text{ rad}} \quad (8.51)$$

Thus, the instantaneous error vector is:

$$\mathbf{e} = \begin{bmatrix} 0.05 \\ -0.04 \\ 0.10 \end{bmatrix} \quad (8.52)$$

The controller uses these deviations to determine the required corrective action.

8.12 Position Error Magnitude

The overall positional deviation can also be represented by the magnitude of the position error:

$$\boxed{e_p = \sqrt{e_x^2 + e_y^2}} \quad (8.53)$$

This provides a scalar measure of the distance between the actual and desired positions.

If:

$$e_x = 0 \quad (8.54)$$

and

$$e_y = 0, \quad (8.55)$$

then:

$$e_p = 0. \quad (8.56)$$

Thus, the robot is exactly at the desired position.

A smaller value of e_p indicates better positional tracking.

8.13 Orientation Error Magnitude

The magnitude of the orientation error can be represented as:

$$\boxed{e_{\phi}^{\text{mag}} = |e_{\phi}|} \quad (8.57)$$

A value close to zero indicates that the actual robot heading is close to the desired heading.

For performance evaluation, the orientation error can be plotted as a function of time:

$$e_{\phi}(t). \quad (8.58)$$

8.14 Error Calculation for Circular Trajectory

For the circular reference trajectory:

$$x_d = R \cos(\omega_d t) \quad (8.59)$$

$$y_d = R \sin(\omega_d t) \quad (8.60)$$

$$\phi_d = \omega_d t. \quad (8.61)$$

The corresponding tracking errors become:

$$\boxed{e_x = R \cos(\omega_d t) - x} \quad (8.62)$$

$$\boxed{e_y = R \sin(\omega_d t) - y} \quad (8.63)$$

and

$$\boxed{e_{\phi} = \omega_d t - \phi} \quad (8.64)$$

These equations are evaluated continuously during the simulation.

8.15 Error Calculation in Simulink

In the Simulink implementation, the desired and actual signals are supplied to subtraction blocks.

For the x -position:

$$x_d \rightarrow \boxed{+} \quad (8.65)$$

and

$$x \rightarrow \boxed{-} \quad (8.66)$$

produce:

$$e_x = x_d - x. \quad (8.67)$$

Similarly, the y -position signals produce:

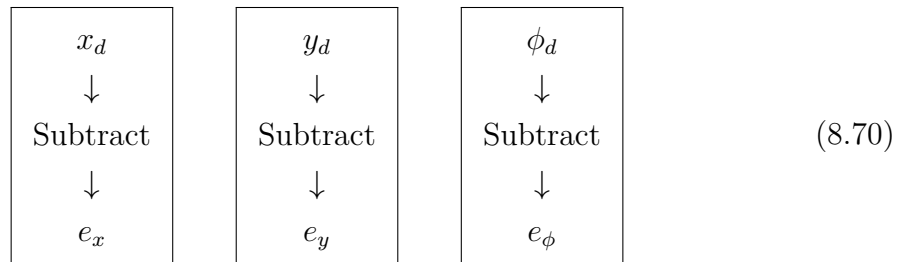
$$e_y = y_d - y. \quad (8.68)$$

The orientation signals produce:

$$e_\phi = \phi_d - \phi. \quad (8.69)$$

The resulting signals are then passed to the coordinate-transformation section.

The complete Simulink signal flow is:



8.16 From Global Error to Robot-Frame Error

The errors calculated above are initially expressed in the global coordinate frame.

However, TurtleBot3 cannot directly move laterally.

Therefore, the global position errors must be converted into errors relative to

the robot's current orientation.

The transformation is:

$$\boxed{x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi} \quad (8.71)$$

and

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (8.72)$$

The orientation error is:

$$\boxed{\phi_{r,\text{dif}} = e_\phi} \quad (8.73)$$

The global-to-robot transformation is the next stage after tracking-error calculation.

8.17 Why Error Transformation Is Necessary

Suppose the desired position is located to the left of the robot.

In the global frame, the error may appear as a combination of e_x and e_y .

However, the robot needs to know whether the desired point is:

- directly ahead,
- behind,
- to the left,
- or to the right.

The robot-frame transformation provides this information.

Therefore:

$$\boxed{(e_x, e_y) \rightarrow (x_{r,\text{dif}}, y_{r,\text{dif}})} \quad (8.74)$$

allows the controller to interpret the error according to the robot's current heading.

8.18 Error Signals Used by the Controller

After coordinate transformation, the controller receives:

$$x_{r,\text{dif}} \quad (8.75)$$

$$y_{r,\text{dif}} \quad (8.76)$$

and

$$\phi_{r,\text{dif}}. \quad (8.77)$$

It also receives the desired velocity signals:

$$v_d \quad (8.78)$$

and

$$\omega_d. \quad (8.79)$$

Therefore, the complete controller input is:

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \\ v_d \\ \omega_d \end{bmatrix} \quad (8.80)$$

These signals are then used by the nonlinear kinematic controller.

8.19 Error and Linear Velocity Control

The forward error $x_{r,\text{dif}}$ contributes to the commanded linear velocity.

The controller uses:

$$v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \quad (8.81)$$

The first term:

$$k_x x_{r,\text{dif}} \quad (8.82)$$

is a feedback correction based on the forward tracking error.

The second term:

$$v_d \cos(\phi_{r,\text{dif}}) \quad (8.83)$$

provides the feedforward contribution.

The controller therefore combines feedback and feedforward information.

8.20 Error and Angular Velocity Control

The lateral and orientation errors affect the angular velocity command.

The controller uses:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (8.84)$$

The orientation-error term:

$$k_\phi \sin(\phi_{r,\text{dif}}) \quad (8.85)$$

corrects the robot's heading.

The lateral-error term:

$$45v_d y_{r,\text{dif}} \quad (8.86)$$

corrects the robot's deviation from the desired path.

The feedforward term:

$$\omega_d \quad (8.87)$$

provides the desired angular motion.

Thus:

$$\boxed{\text{Tracking Error} \rightarrow \text{Velocity Correction}} \quad (8.88)$$

8.21 Desired Tracking Condition

The ideal tracking condition is:

$$x = x_d, \quad (8.89)$$

$$y = y_d, \quad (8.90)$$

and

$$\phi = \phi_d. \quad (8.91)$$

Substituting these conditions into the error equations:

$$e_x = x_d - x_d = 0, \quad (8.92)$$

$$e_y = y_d - y_d = 0, \quad (8.93)$$

and

$$e_\phi = \phi_d - \phi_d = 0. \quad (8.94)$$

Therefore:

$$\mathbf{e} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (8.95)$$

This represents perfect trajectory tracking.

8.22 Error Convergence

The controller is designed so that the tracking errors decrease as the robot follows the reference trajectory.

The desired response is:

$$e_x(t) \rightarrow 0 \quad (8.96)$$

$$e_y(t) \rightarrow 0 \quad (8.97)$$

and

$$e_\phi(t) \rightarrow 0. \quad (8.98)$$

A typical acceptable response should show:

1. An initial transient error.
2. Error reduction with time.
3. Small steady-state tracking error.
4. No sustained divergence.

The actual performance is evaluated using the simulation results presented in the later chapters.

8.23 Error Monitoring

The tracking errors should be monitored throughout the simulation.

Important signals to observe include:

$$e_x(t) \quad (8.99)$$

$$e_y(t) \quad (8.100)$$

$$e_\phi(t) \quad (8.101)$$

and the overall positional error:

$$e_p(t) = \sqrt{e_x^2(t) + e_y^2(t)}. \quad (8.102)$$

Plots of these signals can be used to determine whether the controller successfully reduces the tracking error.

For the final report, these plots should be included in the **Simulation Results** and **Performance Analysis** chapters.

8.24 Performance Measures Based on Error

Several quantitative measures can be calculated from the tracking-error signals.

Maximum Error

For a particular error signal:

$$e_{\max} = \max |e(t)| \quad (8.103)$$

This gives the largest deviation during the simulation.

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{k=1}^N |e[k]| \quad (8.104)$$

Root Mean Square Error

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^N e^2[k]} \quad (8.105)$$

These measures can be calculated separately for:

$$e_x, \quad e_y, \quad e_\phi. \quad (8.106)$$

They provide numerical measures of trajectory-tracking accuracy.

8.25 Error Calculation Algorithm

The complete tracking-error calculation procedure can be summarized as follows.

Step 1: Obtain Desired Position

Generate:

$$x_d, \quad y_d. \quad (8.107)$$

Step 2: Obtain Desired Orientation

Generate:

$$\phi_d. \quad (8.108)$$

Step 3: Obtain Actual Position

From odometry:

$$x, \quad y. \quad (8.109)$$

Step 4: Obtain Actual Orientation

Convert quaternion data into yaw:

$$\phi. \quad (8.110)$$

Step 5: Calculate Position Errors

$$e_x = x_d - x \quad (8.111)$$

$$e_y = y_d - y. \quad (8.112)$$

Step 6: Calculate Orientation Error

$$e_\phi = \phi_d - \phi. \quad (8.113)$$

Step 7: Form the Error Vector

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix}. \quad (8.114)$$

Step 8: Transform Position Errors

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (8.115)$$

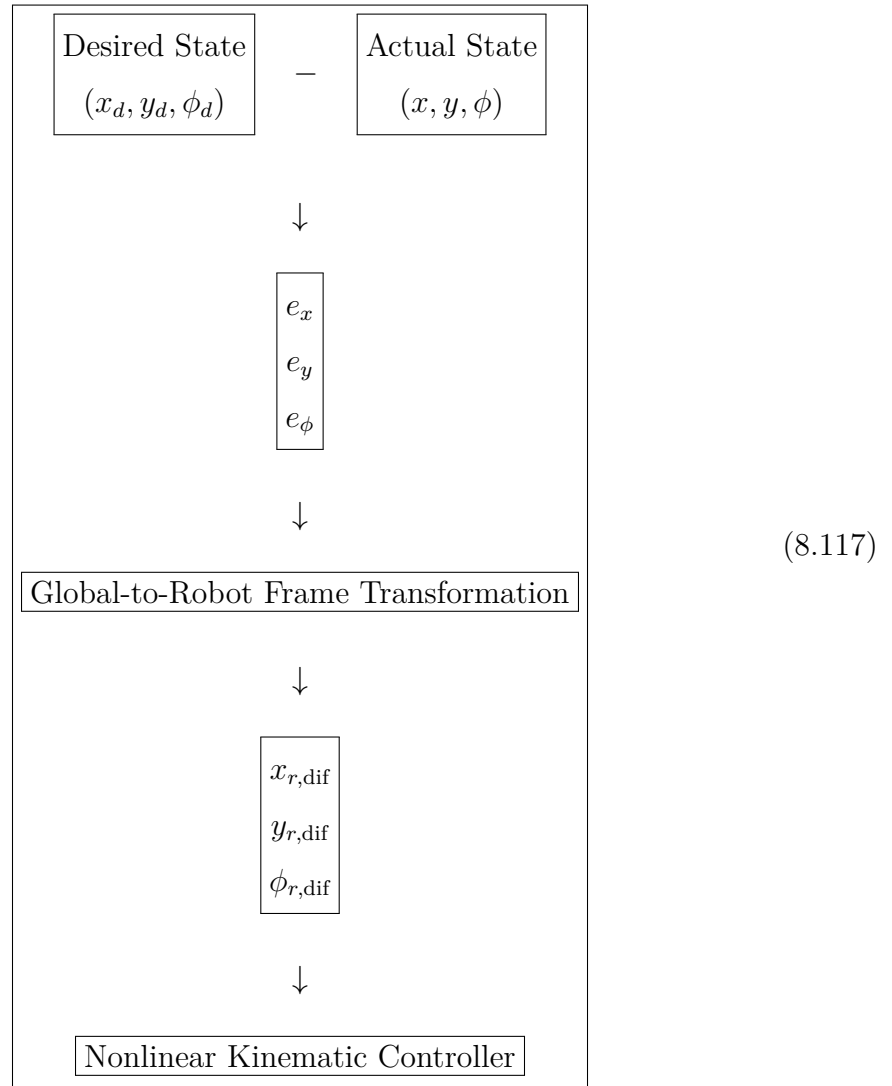
$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (8.116)$$

Step 9: Send Errors to Controller

The transformed errors are supplied to the nonlinear kinematic controller.

8.26 Complete Error-Processing Structure

The complete error-processing architecture can be represented as:



This structure provides the direct connection between the reference trajectory and the control law.

8.27 Summary

This chapter presented the calculation and processing of tracking errors for the TurtleBot3 trajectory-tracking system.

The desired state is:

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix} \quad (8.118)$$

while the actual state obtained from odometry is:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix}. \quad (8.119)$$

The tracking errors are calculated as:

$$e_x = x_d - x \quad (8.120)$$

$$e_y = y_d - y \quad (8.121)$$

and

$$e_\phi = \phi_d - \phi \quad (8.122)$$

giving the error vector:

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (8.123)$$

Since TurtleBot3 operates using its own body-fixed coordinate frame, the global position errors are transformed into robot-frame errors:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (8.124)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (8.125)$$

with:

$$\phi_{r,\text{dif}} = e_\phi \quad (8.126)$$

These transformed errors form the principal feedback inputs to the nonlinear kinematic controller.

The complete process is:

$$\boxed{\text{Desired State} \rightarrow \text{Comparison} \rightarrow \text{Tracking Error} \rightarrow \text{Robot-Frame Error} \rightarrow \text{Kinematic Controller}} \quad (8.127)$$

The ultimate objective is to achieve:

$$\boxed{e_x \rightarrow 0, \quad e_y \rightarrow 0, \quad e_\phi \rightarrow 0} \quad (8.128)$$

so that the actual TurtleBot3 trajectory closely follows the desired circular reference trajectory.

Global-to-Robot Frame Transformation

9.1 Introduction

In the proposed TurtleBot3 trajectory-tracking system, the position and orientation errors are initially calculated with respect to the **global coordinate frame**. However, TurtleBot3 is a differential-drive robot and its motion is defined relative to its own body-fixed coordinate frame.

Therefore, before applying the nonlinear kinematic controller, the global position errors must be transformed into the **robot coordinate frame**.

The purpose of this transformation is to express the tracking error in terms that are directly meaningful to the robot:

- error in the forward/backward direction,
- error in the lateral direction,
- error in orientation.

The transformation process is:

$$\boxed{(e_x, e_y, e_\phi) \rightarrow (x_{r,dif}, y_{r,dif}, \phi_{r,dif})} \quad (9.1)$$

This stage transforms the global-frame tracking errors into the robot's local coordinate frame before the nonlinear kinematic controller.

9.2 Need for Coordinate Transformation

The desired and actual positions are obtained in the global coordinate system.

The position errors are:

$$e_x = x_d - x \quad (9.2)$$

and

$$e_y = y_d - y. \quad (9.3)$$

These errors indicate the difference between the desired and actual positions with respect to the global X - and Y -axes.

However, the robot does not move independently in the global X - and Y -directions. A differential-drive robot moves primarily in its forward direction and changes direction through rotation.

Therefore, the controller needs to know:

$$\boxed{\text{How much of the position error is in front of the robot,}} \quad (9.4)$$

$$\boxed{\text{and how much is to the side of the robot?}} \quad (9.5)$$

This information is obtained through the global-to-robot coordinate transformation.

9.3 Global Coordinate Frame

The global coordinate frame is fixed with respect to the environment.

The robot's position is represented by:

$$\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix}. \quad (9.6)$$

The desired position is:

$$\mathbf{p}_d = \begin{bmatrix} x_d \\ y_d \end{bmatrix}. \quad (9.7)$$

The global position error is therefore:

$$\boxed{\mathbf{e}_p = \mathbf{p}_d - \mathbf{p}} \quad (9.8)$$

or:

$$\mathbf{e}_p = \begin{bmatrix} e_x \\ e_y \end{bmatrix} \quad (9.9)$$

where:

$$e_x = x_d - x \quad (9.10)$$

and

$$e_y = y_d - y. \quad (9.11)$$

9.4 Robot Coordinate Frame

The robot coordinate frame is attached to TurtleBot3 and moves and rotates with the robot.

The robot frame consists of:

- x_r : forward direction of the robot,
- y_r : lateral direction of the robot.

The robot's orientation relative to the global frame is represented by:

$$\phi. \quad (9.12)$$

Because the robot frame rotates as the robot changes its heading, the same physical error can have different x_r and y_r components depending on ϕ .

Therefore, the robot's current yaw angle must be included in the coordinate transformation.

9.5 Position Error in the Global Frame

The position error before transformation is:

$$\mathbf{e}_p = \begin{bmatrix} e_x \\ e_y \end{bmatrix} \quad (9.13)$$

where:

$$e_x = x_d - x \quad (9.14)$$

and

$$e_y = y_d - y. \quad (9.15)$$

This represents the displacement from the actual robot position to the desired position in the global coordinate system.

Although this information is sufficient to determine the distance between the two points, it does not directly indicate whether the desired position is ahead or to the side of the robot.

9.6 Rotation Matrix

To transform the error from the global frame to the robot frame, a two-dimensional rotation matrix is used.

The rotation matrix is:

$$R(\phi) = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix} \quad (9.16)$$

Therefore, the robot-frame position error is:

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \end{bmatrix} = R(\phi) \begin{bmatrix} e_x \\ e_y \end{bmatrix} \quad (9.17)$$

This produces the error components relative to the current orientation of TurtleBot3.

9.7 Derivation of Forward Error

The first row of the transformation matrix gives:

$$x_{r,\text{dif}} = \cos \phi e_x + \sin \phi e_y. \quad (9.18)$$

Therefore:

$$\boxed{x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi} \quad (9.19)$$

This quantity represents the component of the position error along the robot's forward direction.

If:

$$x_{r,\text{dif}} > 0, \quad (9.20)$$

the desired position has a component in front of the robot.

If:

$$x_{r,\text{dif}} < 0, \quad (9.21)$$

the desired position has a component behind the robot.

9.8 Derivation of Lateral Error

The second row of the transformation matrix gives:

$$y_{r,\text{dif}} = -\sin \phi e_x + \cos \phi e_y. \quad (9.22)$$

Rearranging:

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (9.23)$$

This quantity represents the component of the position error perpendicular to the robot's forward direction.

If:

$$y_{r,\text{dif}} > 0, \quad (9.24)$$

the desired position is on one side of the robot.

If:

$$y_{r,\text{dif}} < 0, \quad (9.25)$$

the desired position is on the opposite side.

The sign determines the direction of the required steering correction.

9.9 Orientation Error

The orientation error does not require the same position rotation matrix.

It is calculated directly as:

$$\phi_{r,\text{dif}} = \phi_d - \phi \quad (9.26)$$

where:

- ϕ_d = desired orientation,
- ϕ = actual orientation.

Thus, the complete robot-frame error vector becomes:

$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{bmatrix} \quad (9.27)$$

This is the error representation supplied to the nonlinear kinematic controller.

9.10 Complete Transformation Equation

Combining the position and orientation errors:

$$x_{r,\text{dif}} = (x_d - x) \cos \phi + (y_d - y) \sin \phi \quad (9.28)$$

$$y_{r,\text{dif}} = (y_d - y) \cos \phi - (x_d - x) \sin \phi \quad (9.29)$$

and:

$$\phi_{r,\text{dif}} = \phi_d - \phi \quad (9.30)$$

These equations form the mathematical basis of the coordinate-transformation stage.

9.11 Physical Interpretation

The transformation converts the global error into quantities that are directly related to robot motion.

9.11.1 Forward Error

$$x_{r,\text{dif}} \quad (9.31)$$

indicates how far the desired trajectory is from the robot along its forward direction.

9.11.2 Lateral Error

$$y_{r,\text{dif}} \quad (9.32)$$

indicates how far the desired trajectory is from the robot in the lateral direction.

9.11.3 Orientation Error

$$\phi_{r,\text{dif}} \quad (9.33)$$

indicates how much the robot's heading differs from the desired heading.

Therefore:

$x_{r,\text{dif}} \rightarrow \text{Forward Correction}$ $y_{r,\text{dif}} \rightarrow \text{Lateral/Steering Correction}$ $\phi_{r,\text{dif}} \rightarrow \text{Heading Correction}$	(9.34)
--	--------

9.12 Special Case: Robot Facing the Global X-Axis

Consider:

$$\phi = 0. \quad (9.35)$$

Then:

$$\cos 0 = 1 \quad (9.36)$$

and

$$\sin 0 = 0. \quad (9.37)$$

The forward error becomes:

$$x_{r,\text{dif}} = e_x(1) + e_y(0) \quad (9.38)$$

so:

$$\boxed{x_{r,\text{dif}} = e_x} \quad (9.39)$$

Similarly:

$$y_{r,\text{dif}} = e_y(1) - e_x(0) \quad (9.40)$$

giving:

$$\boxed{y_{r,\text{dif}} = e_y} \quad (9.41)$$

Therefore, when the robot is aligned with the global X -axis, the global and robot-frame errors are identical.

9.13 Special Case: Robot Rotated by 90°

Consider:

$$\phi = \frac{\pi}{2}. \quad (9.42)$$

Then:

$$\cos \phi = 0 \quad (9.43)$$

and

$$\sin \phi = 1. \quad (9.44)$$

Therefore:

$$x_{r,\text{dif}} = e_x(0) + e_y(1) \quad (9.45)$$

giving:

$$\boxed{x_{r,\text{dif}} = e_y} \quad (9.46)$$

and:

$$y_{r,\text{dif}} = e_y(0) - e_x(1) \quad (9.47)$$

giving:

$$\boxed{y_{r,\text{dif}} = -e_x} \quad (9.48)$$

This demonstrates why the robot's orientation must be included in the transformation.

9.14 Example of Coordinate Transformation

Consider:

$$e_x = 0.10 \text{ m} \quad (9.49)$$

$$e_y = 0.05 \text{ m} \quad (9.50)$$

and:

$$\phi = 30^\circ. \quad (9.51)$$

Since:

$$\cos 30^\circ \approx 0.866 \quad (9.52)$$

and

$$\sin 30^\circ = 0.5, \quad (9.53)$$

the forward error is:

$$x_{r,\text{dif}} = (0.10)(0.866) + (0.05)(0.5) \quad (9.54)$$

$$= 0.0866 + 0.025 \quad (9.55)$$

Therefore:

$$\boxed{x_{r,\text{dif}} \approx 0.1116 \text{ m}} \quad (9.56)$$

The lateral error is:

$$y_{r,\text{dif}} = (0.05)(0.866) - (0.10)(0.5) \quad (9.57)$$

$$= 0.0433 - 0.05 \quad (9.58)$$

Therefore:

$$\boxed{y_{r,\text{dif}} \approx -0.0067 \text{ m}} \quad (9.59)$$

Thus, although the global errors are:

$$(0.10, 0.05), \quad (9.60)$$

the robot sees them approximately as:

$$\boxed{(0.1116, -0.0067)} \quad (9.61)$$

This illustrates the importance of considering the robot's current orientation.

9.15 Coordinate Transformation in Simulink

The transformation is implemented in Simulink using mathematical blocks.

The required signals are:

$$e_x, \quad e_y, \quad \phi. \quad (9.62)$$

The angle ϕ is supplied to:

$$\cos(\phi) \quad (9.63)$$

and

$$\sin(\phi) \quad (9.64)$$

blocks.

The resulting signals are multiplied with the corresponding position errors.

The two resulting terms are then added/subtracted according to:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (9.65)$$

and

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (9.66)$$

The Simulink implementation therefore follows the mathematical transformation directly.

9.16 Signal Flow of the Transformation Block

The transformation section can be represented as:

$$\begin{array}{|c|} \hline e_x \\ \hline \downarrow \\ \hline \times \cos \phi \\ \hline \end{array} + \begin{array}{|c|} \hline e_y \\ \hline \downarrow \\ \hline \times \sin \phi \\ \hline \end{array} \rightarrow \boxed{x_{r,\text{dif}}} \quad (9.67)$$

Similarly:

$$\begin{array}{|c|} \hline e_y \\ \hline \downarrow \\ \hline \times \cos \phi \\ \hline \end{array} - \begin{array}{|c|} \hline e_x \\ \hline \downarrow \\ \hline \times \sin \phi \\ \hline \end{array} \rightarrow \boxed{y_{r,\text{dif}}} \quad (9.68)$$

The orientation error is passed separately:

$$\boxed{\phi_d - \phi \rightarrow \phi_{r,\text{dif}}} \quad (9.69)$$

9.17 Relationship With Trajectory Tracking

The coordinate transformation creates the connection between the global reference trajectory and the robot's physical motion.

The reference trajectory provides:

$$x_d, \quad y_d, \quad \phi_d. \quad (9.70)$$

The odometry provides:

$$x, \quad y, \quad \phi. \quad (9.71)$$

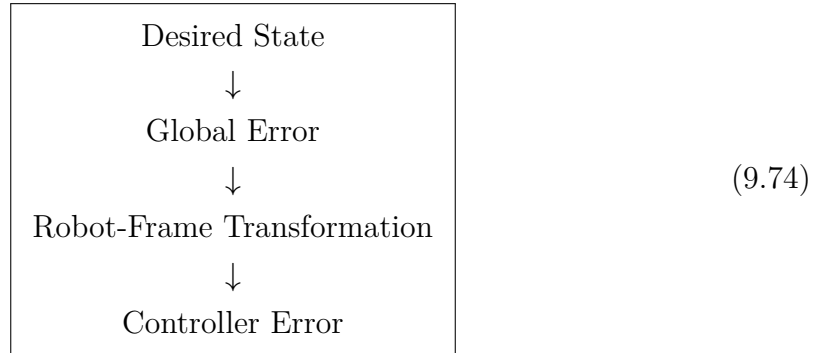
These are used to calculate:

$$e_x, \quad e_y, \quad e_\phi. \quad (9.72)$$

The coordinate transformation then produces:

$$x_{r,dif}, \quad y_{r,dif}, \quad \phi_{r,dif}. \quad (9.73)$$

Therefore:



9.18 Relationship With the Kinematic Controller

The robot-frame errors are directly used by the nonlinear kinematic controller.

The controller uses:

$$x_{r,dif} \quad (9.75)$$

for linear-motion correction and uses:

$$y_{r,\text{dif}} \quad (9.76)$$

and

$$\phi_{r,\text{dif}} \quad (9.77)$$

for angular-motion correction.

The controller equations are:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (9.78)$$

and:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (9.79)$$

The transformed errors therefore provide the required inputs to the nonlinear kinematic controller.

9.19 Effect of Forward Error

The forward error contributes directly to the commanded linear velocity.

The feedback contribution is:

$$k_x x_{r,\text{dif}}. \quad (9.80)$$

Therefore, an increase in forward error results in a corresponding change in the linear velocity command.

The linear velocity equation is:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (9.81)$$

The first term represents the forward-error feedback contribution.

9.20 Effect of Lateral Error

The lateral error contributes to the angular velocity command through:

$$45v_d y_{r,\text{dif}}. \quad (9.82)$$

Therefore:

$$\boxed{y_{r,\text{dif}} \rightarrow \omega_c} \quad (9.83)$$

This allows the robot to turn toward the desired trajectory when it is displaced laterally.

The lateral-error term is:

$$\boxed{45v_d y_{r,\text{dif}}} \quad (9.84)$$

9.21 Effect of Orientation Error

The orientation error contributes through:

$$k_\phi \sin(\phi_{r,\text{dif}}). \quad (9.85)$$

Therefore:

$$\boxed{\phi_{r,\text{dif}} \rightarrow \text{Heading Correction} \rightarrow \omega_c} \quad (9.86)$$

This causes the robot to adjust its orientation toward the desired heading.

9.22 Combined Error-Control Relationship

The complete relationship can be represented as:

$$\boxed{\begin{array}{lcl} x_{r,\text{dif}} & \rightarrow & v_c \\ y_{r,\text{dif}} & \rightarrow & \omega_c \\ \phi_{r,\text{dif}} & \rightarrow & \omega_c \end{array}} \quad (9.87)$$

More specifically:

$$\boxed{x_{r,\text{dif}} \rightarrow k_x x_{r,\text{dif}} \rightarrow v_c} \quad (9.88)$$

and:

$$\begin{array}{c} y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{array} \rightarrow \omega_c \quad (9.89)$$

This demonstrates how the coordinate transformation prepares the error signals for the controller.

9.23 Importance of Correct Sign Convention

The signs used in the coordinate transformation are important.

The implemented equations are:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (9.90)$$

and

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (9.91)$$

The negative sign in:

$$-e_x \sin \phi \quad (9.92)$$

is essential for obtaining the correct lateral error according to the selected coordinate convention.

An incorrect sign would reverse the interpretation of the lateral error and could cause the robot to steer in the wrong direction.

Therefore, the transformation must be implemented exactly according to the selected coordinate-frame convention.

9.24 Transformation and Error Convergence

The purpose of transforming the error is ultimately to allow the controller to reduce the tracking error.

The desired condition is:

$$x_{r,\text{dif}} \rightarrow 0 \quad (9.93)$$

$$y_{r,\text{dif}} \rightarrow 0 \quad (9.94)$$

and

$$\phi_{r,\text{dif}} \rightarrow 0. \quad (9.95)$$

When these conditions are achieved:

$$\mathbf{e}_r \rightarrow \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \quad (9.96)$$

which indicates that the robot has converged to the desired position and orientation.

9.25 Complete Mathematical Flow

The complete mathematical process from the desired and actual states to the controller inputs is:

Desired State

$$\mathbf{q}_d = \begin{bmatrix} x_d \\ y_d \\ \phi_d \end{bmatrix} \quad (9.97)$$

Actual State

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (9.98)$$

Global-Frame Error

$$\mathbf{e} = \begin{bmatrix} x_d - x \\ y_d - y \\ \phi_d - \phi \end{bmatrix} \quad (9.99)$$

Position Transformation

$$\begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \end{bmatrix} = \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} x_d - x \\ y_d - y \end{bmatrix} \quad (9.100)$$

Robot-Frame Error

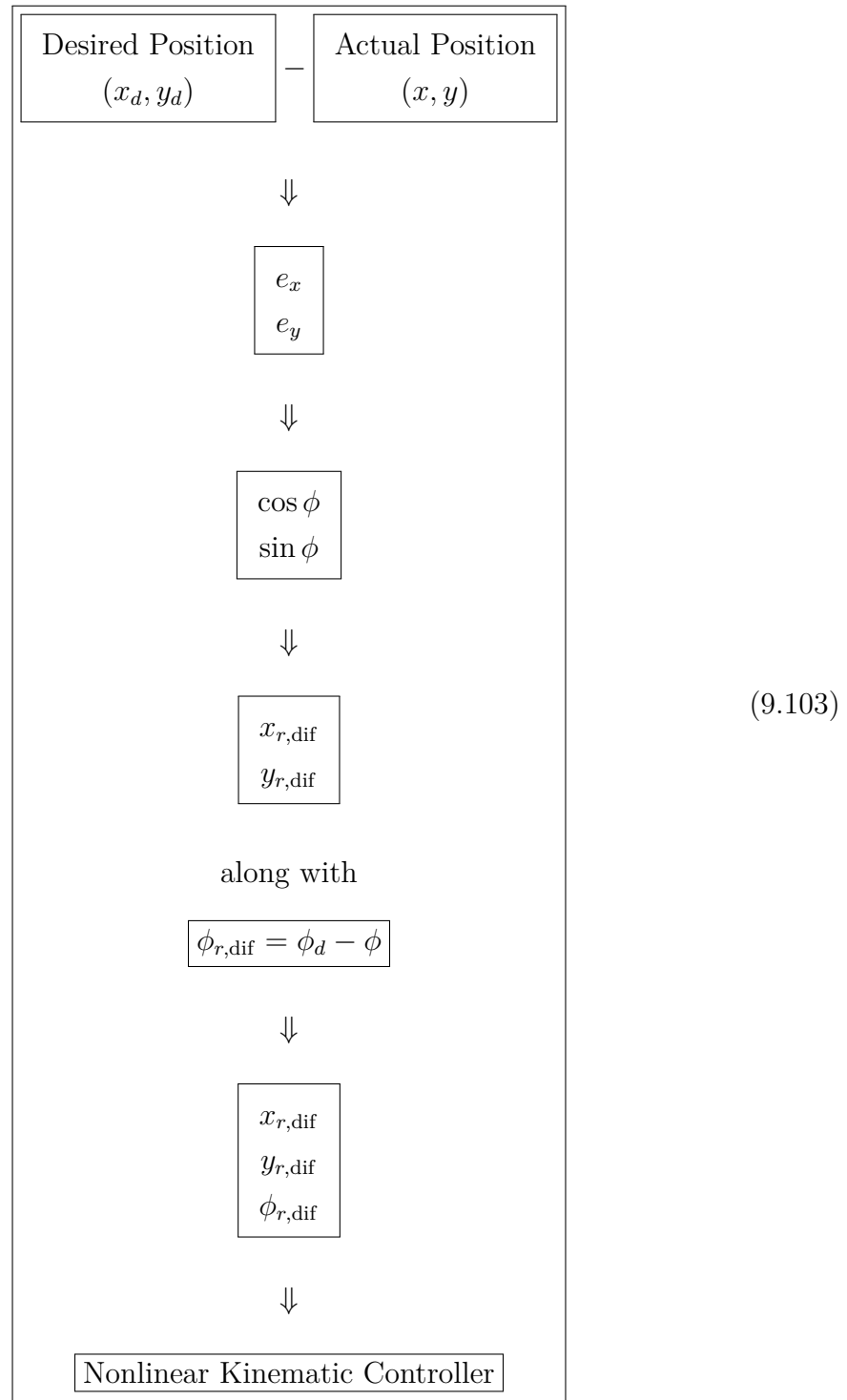
$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_d - \phi \end{bmatrix} \quad (9.101)$$

Controller

$$\mathbf{e}_r \rightarrow (v_c, \omega_c) \quad (9.102)$$

9.26 Overall Signal Flow

The complete signal flow of this chapter can be summarized as:



9.27 Summary

This chapter presented the **global-to-robot coordinate transformation** used in the TurtleBot3 trajectory-tracking system.

The desired and actual robot positions are first compared in the global coordinate frame:

$$e_x = x_d - x \quad (9.104)$$

and

$$e_y = y_d - y. \quad (9.105)$$

Because TurtleBot3 moves according to its own body-fixed coordinate frame, these errors are transformed using the robot's current yaw angle ϕ .

The forward error is:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (9.106)$$

and the lateral error is:

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (9.107)$$

while the orientation error is:

$$\phi_{r,\text{dif}} = \phi_d - \phi \quad (9.108)$$

Thus, the complete robot-frame error vector is:

$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{bmatrix} \quad (9.109)$$

These signals provide a physically meaningful representation of the tracking error and are supplied to the nonlinear kinematic controller.

The overall process is:

$$\boxed{\text{Global Position Error} \rightarrow \text{Rotation by Robot Heading} \rightarrow \text{Robot-Frame Error} \rightarrow \text{Kinematic Controller}} \quad (9.110)$$

The objective of this transformation is to enable the controller to distinguish between **forward displacement**, **lateral displacement**, and **heading error**, allowing it to generate appropriate linear and angular velocity commands for accurate trajectory tracking.

Nonlinear Kinematic Controller

10.1 Introduction

The nonlinear kinematic controller is the main control stage responsible for converting the trajectory-tracking errors into suitable velocity commands for TurtleBot3.

After the desired trajectory and actual robot state are obtained, the tracking errors are calculated and transformed from the global coordinate frame into the robot coordinate frame. The resulting errors are:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}. \quad (10.1)$$

These errors, together with the desired linear and angular velocities,

$$v_d, \quad \omega_d, \quad (10.2)$$

are supplied to the nonlinear kinematic controller.

The controller generates:

$$\boxed{v_c} \quad (10.3)$$

and

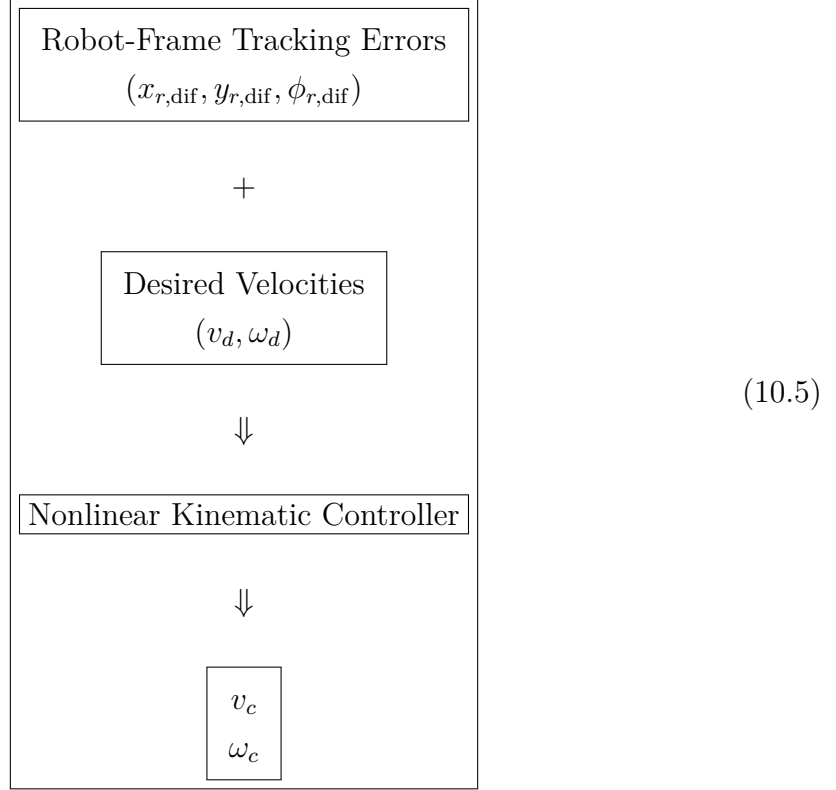
$$\boxed{\omega_c} \quad (10.4)$$

where:

- v_c = commanded linear velocity,
- ω_c = commanded angular velocity.

The controller is a nonlinear feedback-feedforward trajectory-tracking controller.

The overall control process is:



10.2 Purpose of the Kinematic Controller

The purpose of the kinematic controller is to determine the velocity commands required to reduce the tracking errors.

The controller receives information about:

- forward position error,
- lateral position error,
- orientation error,
- desired linear velocity,
- desired angular velocity.

It then calculates the appropriate:

$$v_c \tag{10.6}$$

and

$$\omega_c. \quad (10.7)$$

The fundamental objective is:

$$\boxed{x_{r,\text{dif}} \rightarrow 0} \quad (10.8)$$

$$\boxed{y_{r,\text{dif}} \rightarrow 0} \quad (10.9)$$

and

$$\boxed{\phi_{r,\text{dif}} \rightarrow 0} \quad (10.10)$$

Thus, the controller attempts to make the actual robot state converge toward the desired state.

10.3 Inputs to the Controller

The nonlinear kinematic controller receives five primary inputs:

$$\boxed{x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}, \quad v_d, \quad \omega_d} \quad (10.11)$$

where:

$x_{r,\text{dif}}$

Forward/backward tracking error in the robot frame.

$y_{r,\text{dif}}$

Lateral tracking error in the robot frame.

$\phi_{r,\text{dif}}$

Orientation or heading error.

v_d

Desired linear velocity.

ω_d

Desired angular velocity.

These quantities form the inputs to the nonlinear kinematic controller.

10.4 Outputs of the Controller

The controller generates two outputs:

$$\boxed{v_c} \tag{10.12}$$

and

$$\boxed{\omega_c} \tag{10.13}$$

The linear velocity command determines the forward motion of TurtleBot3.

The angular velocity command determines how the robot changes its orientation.

Therefore:

$$\boxed{(v_c, \omega_c)} \tag{10.14}$$

completely specifies the commanded planar motion of the differential-drive robot at the kinematic level.

10.5 Feedback and Feedforward Control

The controller combines two types of control action:

1. Feedback control
2. Feedforward control

Feedback control uses the current tracking error to correct deviations from the desired trajectory.

Feedforward control uses the desired trajectory information directly to anticipate the motion required to follow the reference.

Therefore, the controller can be represented as:

$$\boxed{\text{Control Input} = \text{Feedback Correction} + \text{Feedforward Action}} \quad (10.15)$$

This combination is used for both the linear and angular velocity commands.

10.6 Linear Velocity Controller

The commanded linear velocity is given by:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (10.16)$$

This equation contains two major components.

Feedback Component

$$\boxed{k_x x_{r,\text{dif}}} \quad (10.17)$$

Feedforward Component

$$\boxed{v_d \cos(\phi_{r,\text{dif}})} \quad (10.18)$$

The first term represents the forward-error feedback contribution, while the second term represents the desired-velocity feedforward contribution with an orientation-alignment factor.

10.7 Forward-Error Feedback Term

The feedback component of the linear velocity controller is:

$$k_x x_{r,\text{dif}}. \quad (10.19)$$

Here,

$$k_x \quad (10.20)$$

is the gain associated with the forward tracking error.

The purpose of this term is to correct the robot's forward/backward displacement relative to the desired trajectory.

If:

$$x_{r,\text{dif}} > 0, \quad (10.21)$$

the feedback term contributes in the positive direction.

If:

$$x_{r,\text{dif}} < 0, \quad (10.22)$$

the feedback contribution changes accordingly.

Thus:

$$\boxed{x_{r,\text{dif}} \rightarrow k_x x_{r,\text{dif}} \rightarrow v_c} \quad (10.23)$$

10.8 Feedforward Linear Velocity Term

The feedforward component is:

$$v_d \cos(\phi_{r,\text{dif}}). \quad (10.24)$$

This term uses the desired linear velocity:

$$v_d. \quad (10.25)$$

The factor:

$$\cos(\phi_{r,\text{dif}}) \quad (10.26)$$

accounts for the heading difference between the desired and actual orientation.

When:

$$\phi_{r,\text{dif}} = 0, \quad (10.27)$$

we have:

$$\cos(0) = 1. \quad (10.28)$$

Therefore:

$$v_d \cos(0) = v_d. \quad (10.29)$$

Thus, when the robot is correctly aligned, the feedforward term becomes the desired linear velocity.

10.9 Complete Linear Velocity Equation

Combining the feedback and feedforward terms:

$$v_c = k_x x_{r,dif} + v_d \cos(\phi_{r,dif}) \quad (10.30)$$

Therefore:

$$\boxed{\text{Linear Command} = \text{Forward-Error Correction} + \text{Desired Linear Motion}} \quad (10.31)$$

This equation allows the robot to maintain the desired forward motion while correcting its forward-position error.

10.10 Angular Velocity Controller

The commanded angular velocity is given by:

$$\omega_c = k_\phi \sin(\phi_{r,dif}) + 45v_d y_{r,dif} + \omega_d \quad (10.32)$$

This equation contains three components:

1. Heading-error correction
2. Lateral-error correction
3. Desired angular-velocity feedforward

These components perform different control functions.

10.11 Heading-Error Correction

The first term is:

$$\boxed{k_\phi \sin(\phi_{r,\text{dif}})} \quad (10.33)$$

where:

$$k_\phi \quad (10.34)$$

is the orientation-error gain.

The purpose of this term is to correct the heading difference between the robot and the desired trajectory.

For small orientation errors:

$$\sin(\phi_{r,\text{dif}}) \approx \phi_{r,\text{dif}}. \quad (10.35)$$

Therefore, for small errors:

$$k_\phi \sin(\phi_{r,\text{dif}}) \approx k_\phi \phi_{r,\text{dif}}. \quad (10.36)$$

This provides an approximately proportional heading correction.

10.12 Lateral-Error Correction

The second term in the angular controller is:

$$\boxed{45v_d y_{r,\text{dif}}} \quad (10.37)$$

This term uses the lateral error:

$$y_{r,\text{dif}}. \quad (10.38)$$

The purpose of this term is to steer the robot toward the desired trajectory when it is displaced laterally.

The gain factor used in the project is:

$$45. \quad (10.39)$$

Thus:

$$\boxed{y_{r,\text{dif}} \rightarrow 45v_d y_{r,\text{dif}} \rightarrow \omega_c} \quad (10.40)$$

The correction is scaled by the desired velocity.

10.13 Desired Angular Velocity Feedforward

The third term is:

$$\boxed{\omega_d} \quad (10.41)$$

This is the feedforward angular velocity corresponding to the desired trajectory.

For the circular trajectory:

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (10.42)$$

Therefore, even when the tracking errors are small, the controller has information about the angular velocity required for the robot to continue following the circular reference.

10.14 Complete Angular Velocity Equation

Combining the three components gives:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (10.43)$$

Therefore:

$$\begin{aligned} \omega_c = & \text{Heading Correction} \\ & + \text{Lateral Correction} \\ & + \text{Feedforward Angular Velocity} \end{aligned} \quad (10.44)$$

This enables the controller to simultaneously correct heading and lateral tracking errors while maintaining the desired angular motion.

10.15 Interpretation of the Controller

The nonlinear kinematic controller can be understood as three basic correction mechanisms.

Forward Position Correction

$$x_{r,\text{dif}} \rightarrow v_c \quad (10.45)$$

Lateral Position Correction

$$y_{r,\text{dif}} \rightarrow \omega_c \quad (10.46)$$

Orientation Correction

$$\phi_{r,\text{dif}} \rightarrow \omega_c \quad (10.47)$$

Therefore:

$x_{r,\text{dif}}$ $\downarrow$ Linear Velocity Correction	$y_{r,\text{dif}}$ $\phi_{r,\text{dif}}$ $\downarrow$ Angular Velocity Correction	(10.48)
--	---	---------

10.16 Controller Operation When Errors Are Zero

Consider the ideal condition:

$$x_{r,\text{dif}} = 0 \quad (10.49)$$

$$y_{r,\text{dif}} = 0 \quad (10.50)$$

and

$$\phi_{r,\text{dif}} = 0. \quad (10.51)$$

The linear velocity becomes:

$$v_c = k_x(0) + v_d \cos(0) \quad (10.52)$$

$$v_c = v_d. \quad (10.53)$$

Therefore:

$$\boxed{v_c = v_d} \quad (10.54)$$

The angular velocity becomes:

$$\omega_c = k_\phi \sin(0) + 45v_d(0) + \omega_d \quad (10.55)$$

Therefore:

$$\boxed{\omega_c = \omega_d} \quad (10.56)$$

Thus, when the robot is exactly following the desired trajectory, the feedback corrections disappear and the controller produces the desired feedforward velocities.

10.17 Controller Operation With Position Error

Suppose the robot has a forward error:

$$x_{r,\text{dif}} \neq 0. \quad (10.57)$$

Then:

$$k_x x_{r,\text{dif}} \quad (10.58)$$

becomes nonzero.

The linear velocity is therefore adjusted from the desired value.

Similarly, if:

$$y_{r,\text{dif}} \neq 0, \quad (10.59)$$

the angular velocity receives the correction:

$$45v_d y_{r,\text{dif}}. \quad (10.60)$$

Thus, the controller automatically modifies the robot's motion according to its deviation from the desired trajectory.

10.18 Controller Operation With Orientation Error

If:

$$\phi_{r,\text{dif}} \neq 0, \quad (10.61)$$

the heading-correction term becomes:

$$k_\phi \sin(\phi_{r,\text{dif}}). \quad (10.62)$$

At the same time, the linear velocity is influenced by:

$$v_d \cos(\phi_{r,\text{dif}}). \quad (10.63)$$

Therefore, orientation error affects both the linear and angular command equations.

This helps the robot align itself with the desired trajectory while controlling its forward motion.

10.19 Role of Nonlinearity

The controller is described as nonlinear because it contains trigonometric functions of the orientation error:

$$\cos(\phi_{r,\text{dif}}) \quad (10.64)$$

and

$$\sin(\phi_{r,\text{dif}}). \quad (10.65)$$

Therefore, the controller equations are not purely linear functions of the track-

ing error.

The nonlinear terms allow the controller to account for the robot's orientation while generating its motion commands.

10.20 Controller Gains

The gains determine the strength of the feedback correction.

The linear velocity controller uses:

$$k_x. \quad (10.66)$$

The angular velocity controller uses:

$$k_\phi. \quad (10.67)$$

These gains determine how strongly the controller responds to forward and orientation errors, respectively.

The exact numerical values of k_x and k_ϕ should be reported exactly as used in the final Simulink model.

The controller also specifies the fixed lateral correction factor:

$$45. \quad (10.68)$$

Therefore, the controller contains the gain-related terms:

$$k_x x_{r,\text{dif}} \quad (10.69)$$

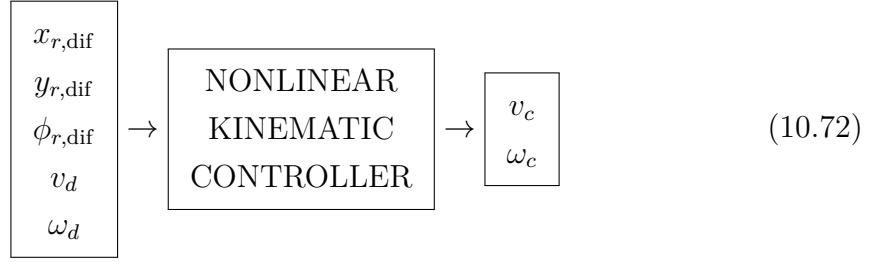
$$k_\phi \sin(\phi_{r,\text{dif}}) \quad (10.70)$$

and:

$$45 v_d y_{r,\text{dif}}. \quad (10.71)$$

10.21 Controller Block Inputs and Outputs

The nonlinear controller can be represented as:



This block forms the main outer-loop control law of the trajectory-tracking system.

10.22 Velocity Saturation After the Controller

The output of the nonlinear controller is not directly allowed to take unlimited values.

The commanded linear velocity is limited according to:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (10.73)$$

The commanded angular velocity is limited according to:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (10.74)$$

These saturation limits form part of the controller implementation.

Therefore, the actual signals passed to the next stage are the saturated versions of the calculated commands.

10.23 Complete Kinematic-Control Structure

The complete nonlinear kinematic-control process can be represented as:

$$\begin{array}{|c|} \hline x_{r,dif} \\ y_{r,dif} \\ \phi_{r,dif} \\ \hline \end{array} + \begin{array}{|c|} \hline v_d \\ \omega_d \\ \hline \end{array} \quad (10.75)$$

$$\Downarrow \quad (10.76)$$

$$\boxed{\text{Nonlinear Kinematic Controller}} \quad (10.77)$$

$$\Downarrow \quad (10.78)$$

$$\begin{bmatrix} v_c \\ \omega_c \end{bmatrix} \quad (10.79)$$

$$\Downarrow \quad (10.80)$$

$$\boxed{\text{Velocity Saturation}} \quad (10.81)$$

$$\Downarrow \quad (10.82)$$

$$\begin{bmatrix} v_{c,\text{sat}} \\ \omega_{c,\text{sat}} \end{bmatrix} \quad (10.83)$$

$$\Downarrow \quad (10.84)$$

$$\boxed{\text{Dynamic Velocity Controller}} \quad (10.85)$$

10.24 Relationship With Dynamic Controller

The nonlinear kinematic controller forms the outer control loop.

Its outputs:

$$v_c, \quad \omega_c \quad (10.86)$$

are supplied to the dynamic velocity-control stage.

The dynamic controller then attempts to make the actual robot velocities follow these commanded values.

Therefore:

$$\boxed{\text{Trajectory Error} \rightarrow \text{Kinematic Controller} \rightarrow (v_c, \omega_c) \rightarrow \text{Dynamic Controller} \rightarrow \text{Robot}} \quad (10.87)$$

The subsequent dynamic controller compares actual v, ω with v_c, ω_c and uses the resulting errors for dynamic control.

10.25 Controller Algorithm

The nonlinear kinematic controller can be implemented according to the following sequence.

Step 1 – Obtain Robot-Frame Errors

Receive:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}. \quad (10.88)$$

Step 2 – Obtain Desired Velocities

Receive:

$$v_d, \quad \omega_d. \quad (10.89)$$

Step 3 – Calculate Linear Velocity

Use:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (10.90)$$

Step 4 – Calculate Angular Velocity

Use:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45 v_d y_{r,\text{dif}} + \omega_d} \quad (10.91)$$

Step 5 – Apply Velocity Limits

Limit v_c to:

$$[-0.22, 0.22] \text{ m/s} \quad (10.92)$$

and ω_c to:

$$[-2.5, 2.5] \text{ rad/s.} \quad (10.93)$$

Step 6 – Pass Commands to Dynamic Controller

The resulting velocity commands are supplied to the next control stage.

10.26 Mathematical Summary of the Controller

The nonlinear kinematic controller is defined by the following two equations:

$$v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \quad (10.94)$$

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d \quad (10.95)$$

with the robot-frame errors given by:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (10.96)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (10.97)$$

and:

$$\phi_{r,\text{dif}} = \phi_d - \phi \quad (10.98)$$

The desired circular trajectory provides:

$$v_d = 0.2 \text{ m/s} \quad (10.99)$$

and:

$$\omega_d = 0.5 \text{ rad/s} \quad (10.100)$$

10.27 Overall Control-Law Flow

The complete mathematical flow can be represented as:

$$\begin{array}{|c|} \hline (x_d, y_d, \phi_d) \\ \hline \text{Desired State} \\ \hline \end{array} - \begin{array}{|c|} \hline (x, y, \phi) \\ \hline \text{Actual State} \\ \hline \end{array} \quad (10.101)$$

$$\Downarrow \quad (10.102)$$

$$\begin{array}{|c|} \hline (e_x, e_y, e_\phi) \\ \hline \end{array} \quad (10.103)$$

$$\Downarrow \quad (10.104)$$

$$\begin{array}{|c|} \hline (x_{r,\text{dif}}, y_{r,\text{dif}}, \phi_{r,\text{dif}}) \\ \hline \end{array} \quad (10.105)$$

$$\Downarrow \quad (10.106)$$

$$\begin{array}{|c|} \hline v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \\ \omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45 v_d y_{r,\text{dif}} + \omega_d \\ \hline \end{array} \quad (10.107)$$

$$\Downarrow \quad (10.108)$$

$$\begin{array}{|c|} \hline \text{Velocity Saturation} \\ \hline \end{array} \quad (10.109)$$

$$\Downarrow \quad (10.110)$$

$$\begin{array}{|c|} \hline (v_c, \omega_c) \\ \hline \end{array} \quad (10.111)$$

$$\Downarrow \quad (10.112)$$

$$\begin{array}{|c|} \hline \text{Dynamic Velocity Controller} \\ \hline \end{array} \quad (10.113)$$

10.28 Expected Controller Behaviour

A properly operating controller should exhibit the following behaviour:

When Forward Error Increases

The controller modifies:

$$v_c \quad (10.114)$$

to correct the forward position.

When Lateral Error Increases

The controller modifies:

$$\omega_c \quad (10.115)$$

to steer the robot back toward the trajectory.

When Orientation Error Increases

The controller generates a heading correction through:

$$k_\phi \sin(\phi_{r,\text{dif}}). \quad (10.116)$$

When All Errors Approach Zero

The controller approaches:

$$v_c \rightarrow v_d \quad (10.117)$$

and:

$$\omega_c \rightarrow \omega_d. \quad (10.118)$$

Therefore, the desired trajectory becomes the dominant component of the control input once tracking is achieved.

10.29 Importance of the Kinematic Controller

The nonlinear kinematic controller is the key link between trajectory tracking and robot motion.

Without the controller, the desired trajectory would simply represent a mathematical path without providing a mechanism for the robot to follow it.

The controller performs the conversion:

$$\boxed{\text{Trajectory Error} \rightarrow \text{Velocity Command}} \quad (10.119)$$

Specifically:

$$\boxed{(x_{r,\text{dif}}, y_{r,\text{dif}}, \phi_{r,\text{dif}}) \rightarrow (v_c, \omega_c)} \quad (10.120)$$

This makes the controller a central component of the complete trajectory-tracking architecture.

10.30 Summary

This chapter presented the nonlinear kinematic controller used to control TurtleBot3 along the desired trajectory.

The controller receives:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}, \quad v_d, \quad \omega_d. \quad (10.121)$$

The commanded linear velocity is:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (10.122)$$

and the commanded angular velocity is:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (10.123)$$

The controller therefore combines **feedback correction** with **feedforward trajectory information**. The forward error primarily affects the linear velocity, while lateral and orientation errors affect the angular velocity.

The outputs are constrained by:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (10.124)$$

and:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (10.125)$$

The resulting commands are then supplied to the dynamic velocity controller.

The complete control sequence is:

$$\boxed{\text{Tracking Error} \rightarrow \text{Robot-Frame Error} \rightarrow \text{Nonlinear Kinematic Controller} \rightarrow (v_c, \omega_c) \rightarrow \text{Velocity Setpoint}} \quad (10.126)$$

The ultimate objective of the nonlinear kinematic controller is to ensure that:

$$\boxed{x_{r,\text{dif}} \rightarrow 0, \quad y_{r,\text{dif}} \rightarrow 0, \quad \phi_{r,\text{dif}} \rightarrow 0} \quad (10.127)$$

so that TurtleBot3 follows the desired circular trajectory accurately.

Velocity Saturation

11.1 Need for Saturation

In the proposed TurtleBot3 trajectory-tracking control system, the nonlinear kinematic controller generates the commanded linear and angular velocities:

$$\boxed{v_c} \quad (11.1)$$

and:

$$\boxed{\omega_c}. \quad (11.2)$$

Since the controller calculates these commands based on the instantaneous tracking errors, the resulting commands may become excessively large when the robot experiences a significant position or orientation error.

In a practical robotic system, the velocity commands cannot be allowed to increase without bound. The robot and its actuators have finite operating capabilities. Therefore, saturation limits are introduced after the kinematic controller.

The saturation block ensures that the commanded velocities remain within the specified operating range:

$$\boxed{v_c \in [v_{\min}, v_{\max}]} \quad (11.3)$$

and:

$$\boxed{\omega_c \in [\omega_{\min}, \omega_{\max}]} \quad (11.4)$$

For the implemented system, the specified limits are:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (11.5)$$

and:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (11.6)$$

Thus, even if the controller produces a command outside these ranges, the saturation block restricts it to the allowable value.

11.2 Linear Velocity Limit

The linear velocity command generated by the controller is restricted to:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (11.7)$$

The positive limit is:

$$\boxed{v_{c,\max} = 0.22 \text{ m/s}} \quad (11.8)$$

and the negative limit is:

$$\boxed{v_{c,\min} = -0.22 \text{ m/s}} \quad (11.9)$$

Therefore, the total available linear-velocity range is:

$$\Delta v = v_{c,\max} - v_{c,\min} \quad (11.10)$$

$$= 0.22 - (-0.22) \quad (11.11)$$

Therefore:

$$\boxed{\Delta v = 0.44 \text{ m/s}} \quad (11.12)$$

The nominal desired linear velocity used for the circular trajectory is:

$$v_d = 0.2 \text{ m/s}. \quad (11.13)$$

This value is within the specified limit because:

$$0.2 < 0.22. \quad (11.14)$$

The available positive margin between the nominal desired velocity and the maximum allowable velocity is:

$$0.22 - 0.20 \quad (11.15)$$

Therefore:

$$\boxed{0.02 \text{ m/s}} \quad (11.16)$$

In percentage terms, the nominal desired velocity uses:

$$\frac{0.20}{0.22} \times 100 \quad (11.17)$$

Therefore:

$$\boxed{90.91\%} \quad (11.18)$$

of the positive linear velocity limit.

Therefore, the selected reference linear velocity operates relatively close to the maximum permitted forward velocity.

11.3 Angular Velocity Limit

The angular velocity command is restricted according to:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (11.19)$$

The maximum positive angular velocity is:

$$\boxed{\omega_{c,\max} = 2.5 \text{ rad/s}} \quad (11.20)$$

and the minimum angular velocity is:

$$\boxed{\omega_{c,\min} = -2.5 \text{ rad/s}} \quad (11.21)$$

Therefore, the total angular velocity range is:

$$\Delta\omega = 2.5 - (-2.5) \quad (11.22)$$

Therefore:

$$\boxed{\Delta\omega = 5 \text{ rad/s}} \quad (11.23)$$

For the circular reference trajectory:

$$\omega_d = 0.5 \text{ rad/s.} \quad (11.24)$$

The desired angular velocity is therefore well within the specified limit:

$$0.5 < 2.5. \quad (11.25)$$

The remaining positive angular-velocity margin is:

$$2.5 - 0.5 \quad (11.26)$$

Therefore:

$$\boxed{2.0 \text{ rad/s}} \quad (11.27)$$

The nominal reference angular velocity uses:

$$\frac{0.5}{2.5} \times 100 \quad (11.28)$$

Therefore:

$$\boxed{20\%} \quad (11.29)$$

of the positive angular velocity limit.

Thus, compared with the linear velocity, the nominal angular velocity has a considerably larger operating margin.

11.4 Saturation Function

The saturation operation can be mathematically represented as:

$$v_{\text{sat}} = \begin{cases} -0.22, & v_c < -0.22 \\ v_c, & -0.22 \leq v_c \leq 0.22 \\ 0.22, & v_c > 0.22 \end{cases} \quad (11.30)$$

Similarly, the angular velocity is saturated according to:

$$\omega_{\text{sat}} = \begin{cases} -2.5, & \omega_c < -2.5 \\ \omega_c, & -2.5 \leq \omega_c \leq 2.5 \\ 2.5, & \omega_c > 2.5 \end{cases} \quad (11.31)$$

Therefore, the controller output supplied to the subsequent stage always satisfies:

$$|v_{\text{sat}}| \leq 0.22 \text{ m/s} \quad (11.32)$$

and:

$$|\omega_{\text{sat}}| \leq 2.5 \text{ rad/s} \quad (11.33)$$

11.5 Effect of Saturation

Velocity saturation has several important effects on the overall control system.

11.5.1 Safety

The primary purpose of limiting the velocity commands is to prevent the robot from receiving unrealistically large motion commands.

Without saturation, a large tracking error could cause the nonlinear controller to generate a large:

$$v_c \quad (11.34)$$

or:

$$\omega_c. \quad (11.35)$$

The saturation block prevents such commands from exceeding the defined operating limits.

11.5.2 Actuator Limitations

A real robotic system has physical limitations.

The motors, wheels, drive mechanism, and controller interface cannot provide unlimited velocity.

Therefore:

$$\boxed{\text{Controller Output} \leq \text{Physical Operating Limit}} \quad (11.36)$$

The selected limits:

$$|v_c| \leq 0.22 \text{ m/s} \quad (11.37)$$

and:

$$|\omega_c| \leq 2.5 \text{ rad/s} \quad (11.38)$$

ensure that the control commands remain within the specified operating range of the system.

11.5.3 Prevention of Unrealistic Commands

The nonlinear controller contains feedback terms based on the tracking errors.

For example:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (11.39)$$

Similarly:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (11.40)$$

If the tracking errors become large, the feedback terms can increase the magnitude of the controller output.

The saturation block prevents these values from becoming unrealistically large.

11.5.4 Effect on Tracking Performance

Saturation can affect tracking performance when the controller demands a velocity greater than the permitted value.

For example, if:

$$v_c = 0.30 \text{ m/s}, \quad (11.41)$$

the saturation block produces:

$$v_{\text{sat}} = 0.22 \text{ m/s}. \quad (11.42)$$

Similarly, if:

$$\omega_c = 3 \text{ rad/s}, \quad (11.43)$$

then:

$$\omega_{\text{sat}} = 2.5 \text{ rad/s}. \quad (11.44)$$

Thus:

$$\boxed{\text{Large Command} \rightarrow \text{Saturation} \rightarrow \text{Limited Command}} \quad (11.45)$$

When saturation occurs, the robot cannot respond as aggressively as requested by the unsaturated controller. Consequently, the tracking error may temporarily remain larger or decrease more slowly.

11.6 Saturation During the Circular Trajectory

For the selected reference trajectory:

$$v_d = 0.2 \text{ m/s} \quad (11.46)$$

and:

$$\omega_d = 0.5 \text{ rad/s}. \quad (11.47)$$

Both values are within their corresponding limits.

For linear velocity:

$$0.2 < 0.22 \quad (11.48)$$

and for angular velocity:

$$0.5 < 2.5. \quad (11.49)$$

Therefore, the nominal reference itself does not require saturation.

However, the controller also contains feedback terms. Consequently, during periods of tracking error, the commanded values may differ from the nominal reference.

The saturation blocks are therefore necessary to ensure:

$$|v_c| \leq 0.22 \quad (11.50)$$

and:

$$|\omega_c| \leq 2.5. \quad (11.51)$$

11.7 Linear Velocity Operating Margin

The positive linear velocity margin is:

$$M_v = 0.22 - 0.20 \quad (11.52)$$

Therefore:

$$\boxed{M_v = 0.02 \text{ m/s}} \quad (11.53)$$

As a percentage of the maximum allowable positive velocity:

$$\frac{0.02}{0.22} \times 100 \quad (11.54)$$

Therefore:

$$\boxed{M_v \approx 9.09\%} \quad (11.55)$$

Thus, the nominal desired velocity has approximately 9.09% of the positive velocity limit remaining above it.

This relatively small margin makes the linear-velocity saturation particularly important during transient tracking conditions.

11.8 Angular Velocity Operating Margin

The positive angular velocity margin is:

$$M_{\omega} = 2.5 - 0.5 \quad (11.56)$$

Therefore:

$$\boxed{M_{\omega} = 2.0 \text{ rad/s}} \quad (11.57)$$

As a percentage of the maximum angular velocity:

$$\frac{2.0}{2.5} \times 100 \quad (11.58)$$

Therefore:

$$\boxed{M_{\omega} = 80\%} \quad (11.59)$$

Therefore, the nominal angular velocity has a much larger operating margin than the nominal linear velocity.

11.9 Comparison of Velocity Margins

The velocity margins can be summarized as:

Table 11.1: Comparison of Linear and Angular Velocity Margins

Parameter	Nominal Value	Maximum Limit	Remaining Margin
Linear velocity	0.20 m/s	0.22 m/s	0.02 m/s
Angular velocity	0.50 rad/s	2.50 rad/s	2.00 rad/s

In percentage terms:

Table 11.2: Percentage of Positive Velocity Limit Used

Parameter	Percentage of Positive Limit Used
Linear velocity	90.91%
Angular velocity	20%

This shows that the linear velocity operates much closer to its upper limit than the angular velocity.

11.10 Importance of Saturation in the Proposed Controller

The saturation stage is positioned between the kinematic controller and the subsequent velocity-control stage.

The overall sequence is:

$$\text{Tracking Errors} \rightarrow \text{Kinematic Controller} \rightarrow \text{Velocity Saturation} \rightarrow \text{Dynamic Controller} \quad (11.60)$$

The saturation stage therefore acts as an interface between the mathematically calculated controller output and the physically allowable velocity command.

It ensures that the controller remains compatible with the specified robot operating constraints.

11.11 Effect of Saturation on Transient Response

During the initial transient, the tracking errors can be comparatively large.

The controller responds to these errors by changing:

$$v_c \quad (11.61)$$

and:

$$\omega_c. \quad (11.62)$$

If these commands exceed the specified limits, the saturation block clips them.

Consequently, the actual robot response may be slower than the response predicted by the unsaturated controller.

This provides a trade-off:

$$\boxed{\text{Large Control Action} \quad \leftrightarrow \quad \text{Faster Error Correction}} \quad (11.63)$$

versus:

$$\boxed{\text{Limited Control Action} \quad \leftrightarrow \quad \text{Safe and Realistic Motion}} \quad (11.64)$$

The saturation limits therefore represent an important practical constraint in the controller design.

11.12 Relationship Between Saturation and Observed Trajectory

The X - Y simulation result presented in Chapter 13 shows that the actual trajectory deviates from the ideal circular reference, particularly during part of the trajectory.

The saturation stage can influence this behaviour whenever the controller requests a velocity outside the allowed range.

When saturation occurs:

$$v_{\text{sat}} \neq v_c \quad (11.65)$$

or:

$$\omega_{\text{sat}} \neq \omega_c. \quad (11.66)$$

Consequently, the robot cannot immediately apply the complete correction requested by the unsaturated controller.

This can contribute to a temporary increase in tracking deviation.

However, the exact contribution of saturation to the observed trajectory error should be determined from the saturation signals and controller outputs in the simulation.

11.13 Practical Significance

The velocity limits provide a connection between the theoretical controller and the practical robot.

A theoretical controller may generate any value mathematically, but the actual robotic system must operate within defined constraints.

Therefore:

$$\boxed{\text{Theoretical Controller} \rightarrow \text{Constraint Handling} \rightarrow \text{Practical Robot Command}} \quad (11.67)$$

The saturation stage is consequently essential for implementing the controller in a realistic robotic environment.

11.14 Summary

The velocity-saturation stage limits the outputs of the nonlinear kinematic controller to the specified operating range.

The linear velocity constraint is:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (11.68)$$

and the angular velocity constraint is:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (11.69)$$

For the selected circular trajectory:

$$v_d = 0.2 \text{ m/s} \quad (11.70)$$

and:

$$\omega_d = 0.5 \text{ rad/s.} \quad (11.71)$$

The nominal linear velocity uses approximately:

$$\boxed{90.91\%} \quad (11.72)$$

of the positive linear velocity limit, leaving a margin of:

$$\boxed{0.02 \text{ m/s}} \quad (11.73)$$

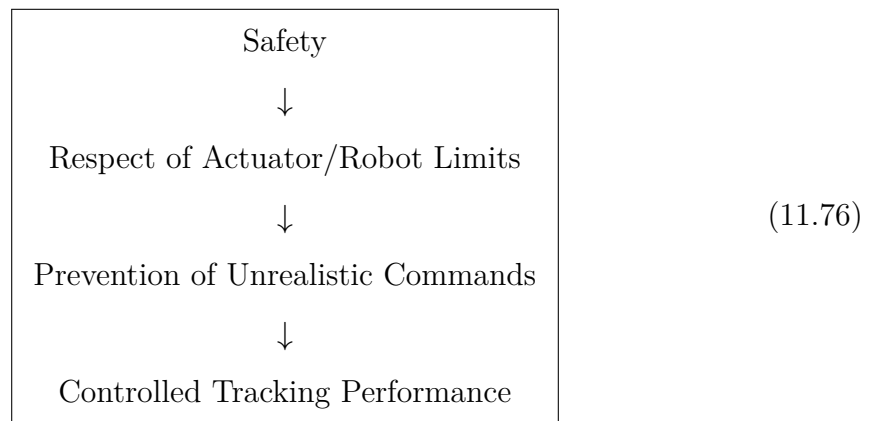
The nominal angular velocity uses:

$$\boxed{20\%} \quad (11.74)$$

of the positive angular velocity limit, leaving a margin of:

$$\boxed{2.0 \text{ rad/s}} \quad (11.75)$$

Thus, the saturation block performs four important functions:



The velocity-saturation stage is therefore an essential part of the complete trajectory-tracking controller and provides the necessary constraint between the controller's theoretical output and the practical operating limits of the TurtleBot3 system.

Dynamic Velocity Controller

12.1 Introduction

The dynamic velocity controller is the second major control stage of the proposed TurtleBot3 trajectory-tracking system. The nonlinear kinematic controller developed in the previous chapter generates the desired or commanded linear and angular velocities:

$$\boxed{v_c} \tag{12.1}$$

and

$$\boxed{\omega_c}. \tag{12.2}$$

However, these commands cannot be assumed to be followed perfectly by the physical robot. The actual robot velocities may differ from the commanded values because of the robot's dynamics, wheel motion, inertia, friction, actuator limitations, and other practical effects.

Therefore, a dynamic velocity controller is used to reduce the difference between the commanded velocities and the actual velocities.

The basic control objective is:

$$\boxed{v \rightarrow v_c} \tag{12.3}$$

and

$$\boxed{\omega \rightarrow \omega_c} \tag{12.4}$$

where:

- v_c = commanded linear velocity,
- ω_c = commanded angular velocity,
- v = actual linear velocity,
- ω = actual angular velocity.

The dynamic velocity controller therefore forms the inner velocity control loop of the complete trajectory-tracking system.

12.2 Need for Dynamic Velocity Control

The nonlinear kinematic controller determines what velocity the robot should ideally have. However, the robot does not necessarily achieve the commanded velocity instantaneously.

For example, if the kinematic controller generates:

$$v_c = 0.2 \text{ m/s}, \quad (12.5)$$

the actual velocity may initially be:

$$v = 0.15 \text{ m/s}. \quad (12.6)$$

Similarly, the commanded angular velocity may be:

$$\omega_c = 0.5 \text{ rad/s}, \quad (12.7)$$

while the actual robot may temporarily operate at:

$$\omega = 0.4 \text{ rad/s}. \quad (12.8)$$

Therefore, velocity errors must be calculated.

The linear velocity error is:

$$e_v = v_c - v \quad (12.9)$$

and the angular velocity error is:

$$\boxed{e_\omega = \omega_c - \omega} \quad (12.10)$$

These errors are supplied to the dynamic controller.

12.3 Dynamic Controller Inputs

The dynamic controller receives the commanded and actual velocity information.

The commanded velocities obtained from the nonlinear kinematic controller are:

$$v_c, \quad \omega_c \quad (12.11)$$

while the actual robot velocities are:

$$v, \quad \omega. \quad (12.12)$$

Therefore, the velocity errors are:

$$\boxed{e_v = v_c - v} \quad (12.13)$$

and:

$$\boxed{e_\omega = \omega_c - \omega} \quad (12.14)$$

The controller uses these errors to generate corrective control actions.

12.4 Linear Velocity Error

The linear velocity tracking error is defined as:

$$\boxed{e_v = v_c - v} \quad (12.15)$$

where:

- v_c is the commanded linear velocity,
- v is the actual linear velocity.

If:

$$e_v > 0, \quad (12.16)$$

the actual linear velocity is lower than the commanded value.

If:

$$e_v < 0, \quad (12.17)$$

the actual linear velocity is higher than the commanded value.

If:

$$e_v = 0, \quad (12.18)$$

then:

$$v = v_c, \quad (12.19)$$

which means that the commanded linear velocity has been achieved.

12.5 Angular Velocity Error

Similarly, the angular velocity error is:

$$\boxed{e_\omega = \omega_c - \omega} \quad (12.20)$$

where:

- ω_c is the commanded angular velocity,
- ω is the actual angular velocity.

If:

$$e_\omega > 0, \quad (12.21)$$

the actual angular velocity is below the commanded value.

If:

$$e_\omega < 0, \quad (12.22)$$

the actual angular velocity is above the commanded value.

If:

$$e_\omega = 0, \quad (12.23)$$

then:

$$\omega = \omega_c. \quad (12.24)$$

Thus, the dynamic controller attempts to drive both velocity errors toward zero.

12.6 Complete Velocity Error Vector

The two velocity errors can be represented using a vector:

$$\mathbf{e}_v = \begin{bmatrix} e_v \\ e_\omega \end{bmatrix} \quad (12.25)$$

or:

$$\mathbf{e}_v = \begin{bmatrix} v_c - v \\ \omega_c - \omega \end{bmatrix} \quad (12.26)$$

This vector represents the difference between the commanded and actual robot motion.

The dynamic controller uses this information to generate the appropriate control action.

12.7 Dynamic Control Structure

The overall dynamic control structure is:

$$\begin{bmatrix} v_c \\ \omega_c \end{bmatrix} - \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (12.27)$$

$$\Downarrow \quad (12.28)$$

$$\begin{bmatrix} e_v \\ e_\omega \end{bmatrix} \quad (12.29)$$

$$\Downarrow \quad (12.30)$$

$$\boxed{\text{Dynamic Velocity Controller}} \quad (12.31)$$

$$\Downarrow \quad (12.32)$$

$$\boxed{\text{Robot / Actuator Dynamics}} \quad (12.33)$$

$$\Downarrow \quad (12.34)$$

$$\begin{bmatrix} v \\ \omega \end{bmatrix} \quad (12.35)$$

This forms the inner control loop of the complete trajectory-tracking system.

12.8 Relationship Between Kinematic and Dynamic Controllers

The two controllers perform different functions.

The **kinematic controller** determines the required robot velocities based on trajectory-tracking error.

The **dynamic velocity controller** ensures that the actual robot motion follows these commanded velocities.

Therefore:

$$\boxed{\text{Trajectory Error} \rightarrow \text{Kinematic Controller} \rightarrow (v_c, \omega_c)} \quad (12.36)$$

followed by:

$$\boxed{(v_c, \omega_c) \rightarrow \text{Dynamic Controller} \rightarrow \text{Robot}} \quad (12.37)$$

The complete control architecture is therefore:

$$\boxed{\text{Trajectory} \rightarrow \text{Kinematic Control} \rightarrow \text{Velocity Control} \rightarrow \text{Robot Motion}} \quad (12.38)$$

12.9 Inner and Outer Control Loops

The complete system can be viewed as having two control loops.

12.9.1 Outer Loop

The outer loop is the nonlinear kinematic trajectory-tracking controller.

Its purpose is to reduce:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}}. \quad (12.39)$$

It generates:

$$v_c, \quad \omega_c. \quad (12.40)$$

12.9.2 Inner Loop

The inner loop is the dynamic velocity controller.

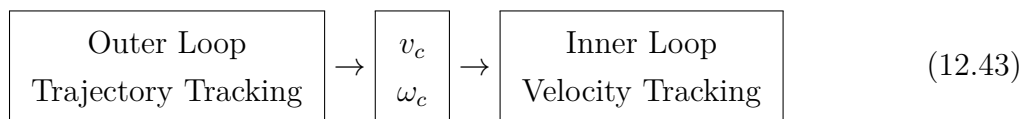
Its purpose is to reduce:

$$e_v = v_c - v \quad (12.41)$$

and:

$$e_\omega = \omega_c - \omega. \quad (12.42)$$

The two-loop architecture can therefore be represented as:



12.10 Dynamic Model of Linear Motion

The simplified linear dynamic relationship can be represented by Newton's second law:

$$\boxed{F = m\dot{v}} \quad (12.44)$$

where:

- F is the resultant longitudinal force,
- m is the robot mass,
- $\dot{v}$ is the linear acceleration.

This relationship indicates that the applied force determines how quickly the robot's linear velocity changes.

If a higher acceleration is required, a larger force is necessary.

The velocity controller therefore needs to account for the difference between the commanded and actual velocity.

12.11 Dynamic Model of Rotational Motion

The rotational motion can be represented by:

$$\boxed{\tau = J\dot{\omega}} \quad (12.45)$$

where:

- τ is the resultant rotational torque,
- J is the robot's rotational inertia,
- $\dot{\omega}$ is the angular acceleration.

This relationship indicates that torque determines the rate at which the robot's angular velocity changes.

The dynamic controller therefore acts to make:

$$\boxed{\omega \rightarrow \omega_c} \quad (12.46)$$

12.12 Linear Velocity Control

The linear velocity controller operates using:

$$e_v = v_c - v. \quad (12.47)$$

The controller generates a corrective action based on this error.

A general proportional control relationship can be expressed as:

$$\boxed{u_v = K_v e_v} \quad (12.48)$$

where K_v is the linear velocity-control gain.

If the velocity error is large, the controller produces a larger corrective action.

As:

$$e_v \rightarrow 0, \quad (12.49)$$

the corrective action decreases.

12.13 Angular Velocity Control

Similarly, the angular velocity controller operates using:

$$e_\omega = \omega_c - \omega. \quad (12.50)$$

A general proportional control relationship can be written as:

$$\boxed{u_\omega = K_\omega e_\omega} \quad (12.51)$$

where K_ω is the angular velocity-control gain.

The controller therefore increases or decreases the rotational control action according to the angular velocity error.

12.14 Combined Dynamic Control

The two control actions can be represented as:

$$\begin{bmatrix} u_v \\ u_\omega \end{bmatrix} = \begin{bmatrix} K_v & 0 \\ 0 & K_\omega \end{bmatrix} \begin{bmatrix} e_v \\ e_\omega \end{bmatrix} \quad (12.52)$$

or equivalently:

$$\begin{aligned} u_v &= K_v e_v, \\ u_\omega &= K_\omega e_\omega. \end{aligned} \quad (12.53)$$

This represents the basic structure of the dynamic velocity-control stage.

The exact controller implementation and gain values should be reported according to the final Simulink model used in the project.

12.15 Velocity Tracking

The purpose of the dynamic controller is to ensure:

$$v \rightarrow v_c \quad (12.54)$$

and:

$$\omega \rightarrow \omega_c \quad (12.55)$$

Therefore:

$$\lim_{t \rightarrow \infty} (v_c - v) = 0 \quad (12.56)$$

and:

$$\lim_{t \rightarrow \infty} (\omega_c - \omega) = 0. \quad (12.57)$$

Thus:

$$e_v \rightarrow 0 \quad (12.58)$$

and:

$$e_\omega \rightarrow 0 \quad (12.59)$$

When this condition is achieved, the actual robot motion follows the commands generated by the outer kinematic controller.

12.16 Interaction With the Robot Model

The robot dynamics convert the applied control action into actual velocity.

The simplified relationship can be represented as:

$$\boxed{\text{Control Action} \rightarrow \text{Robot Dynamics} \rightarrow (v, \omega)} \quad (12.60)$$

These actual velocities are then fed back to the dynamic controller.

Thus:

$$\boxed{\text{Command} \rightarrow \text{Controller} \rightarrow \text{Robot} \rightarrow \text{Velocity Feedback} \rightarrow \text{Controller}} \quad (12.61)$$

This feedback loop allows the dynamic controller to continuously correct velocity deviations.

12.17 Importance of Feedback

Without feedback, the dynamic controller would not know whether the robot had successfully achieved the commanded velocity.

For example, suppose:

$$v_c = 0.2 \text{ m/s} \quad (12.62)$$

but:

$$v = 0.15 \text{ m/s.} \quad (12.63)$$

The velocity error is:

$$e_v = 0.2 - 0.15 \quad (12.64)$$

Therefore:

$$\boxed{e_v = 0.05 \text{ m/s}} \quad (12.65)$$

The controller recognizes this deviation and produces a corrective action.

Similarly, if:

$$\omega_c = 0.5 \text{ rad/s} \quad (12.66)$$

and:

$$\omega = 0.4 \text{ rad/s}, \quad (12.67)$$

then:

$$e_\omega = 0.5 - 0.4 \quad (12.68)$$

Therefore:

$$\boxed{e_\omega = 0.1 \text{ rad/s}} \quad (12.69)$$

The angular control loop then attempts to eliminate this error.

12.18 Velocity Saturation

The velocity commands generated by the nonlinear kinematic controller are constrained before being supplied to the dynamic stage.

The specified linear velocity limits are:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (12.70)$$

and the angular velocity limits are:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (12.71)$$

as specified in the project implementation.

Therefore:

$$\boxed{v_{c,\text{sat}} = \text{sat}(v_c)} \quad (12.72)$$

and:

$$\boxed{\omega_{c,\text{sat}} = \text{sat}(\omega_c)} \quad (12.73)$$

These limited commands are used by the dynamic controller.

12.19 Why Velocity Saturation Is Required

A physical robot has limitations on how fast it can move and rotate.

If the controller generates an excessively large command, the robot may not be able to achieve it.

Saturation ensures that:

$$v_c \tag{12.74}$$

and:

$$\omega_c \tag{12.75}$$

remain within the specified operating limits.

It also prevents unrealistically large commands during large transient tracking errors.

Therefore:

$$\boxed{\text{Controller Output} \rightarrow \text{Saturation} \rightarrow \text{Safe/Specified Command}} \tag{12.76}$$

12.20 Dynamic Controller and Circular Trajectory

For the desired circular trajectory:

$$v_d = 0.2 \text{ m/s} \tag{12.77}$$

and:

$$\omega_d = 0.5 \text{ rad/s.} \tag{12.78}$$

The nonlinear kinematic controller uses the tracking errors to generate:

$$v_c \tag{12.79}$$

and:

$$\omega_c. \quad (12.80)$$

The dynamic controller then attempts to make the actual velocities follow these commands:

$$v \rightarrow v_c \quad (12.81)$$

and:

$$\omega \rightarrow \omega_c. \quad (12.82)$$

Therefore, the dynamic controller plays an important role in ensuring that the velocity behaviour required for circular trajectory tracking is actually achieved by the robot.

12.21 Complete Dynamic Control Architecture

The complete dynamic-control architecture can be represented as:

$$\begin{array}{|c|} \hline v_c \\ \hline \omega_c \\ \hline \end{array} \quad (12.83)$$

$$\Downarrow \quad (12.84)$$

$$\text{Velocity Saturation} \quad (12.85)$$

$$\Downarrow \quad (12.86)$$

$$\begin{array}{|c|} \hline v_{c,\text{sat}} \\ \hline \omega_{c,\text{sat}} \\ \hline \end{array} \quad (12.87)$$

$$\begin{array}{|c|} \hline v \\ \hline \omega \\ \hline \end{array} \quad (12.88)$$

$$\Downarrow \quad (12.89)$$

$$\begin{bmatrix} e_v \\ e_\omega \end{bmatrix} \quad (12.90)$$

$$\Downarrow \quad (12.91)$$

$$\boxed{\text{Dynamic Velocity Controller}} \quad (12.92)$$

$$\Downarrow \quad (12.93)$$

$$\boxed{\text{Robot Dynamics}} \quad (12.94)$$

$$\Downarrow \quad (12.95)$$

$$\begin{bmatrix} v \\ \omega \end{bmatrix} \quad (12.96)$$

The feedback loop continuously compares the actual velocities with the commanded velocities.

12.22 Overall Closed-Loop System

Combining the previous chapters, the complete trajectory-tracking system can be represented as:

$$\boxed{\text{Reference Trajectory}} \quad (12.97)$$

$$\Downarrow \quad (12.98)$$

$$\boxed{\text{Desired State}} \quad (12.99)$$

$$\Downarrow \quad (12.100)$$

$$\boxed{\text{Tracking Error}} \quad (12.101)$$

$$\Downarrow \quad (12.102)$$

$$\boxed{\text{Global-to-Robot Transformation}} \quad (12.103)$$

$$\Downarrow \quad (12.104)$$

$$\boxed{\text{Nonlinear Kinematic Controller}} \quad (12.105)$$

$$\Downarrow \quad (12.106)$$

$$\boxed{\begin{matrix} v_c \\ \omega_c \end{matrix}} \quad (12.107)$$

$$\Downarrow \quad (12.108)$$

$$\boxed{\text{Dynamic Velocity Controller}} \quad (12.109)$$

$$\Downarrow \quad (12.110)$$

$$\boxed{\text{TurtleBot3}} \quad (12.111)$$

$$\Downarrow \quad (12.112)$$

$$\boxed{\text{Odometry / Velocity Feedback}} \quad (12.113)$$

The resulting feedback completes the closed-loop trajectory-tracking architecture.

12.23 Algorithm for Dynamic Velocity Control

The dynamic velocity-control process can be summarized as follows.

Step 1: Receive Commands

Receive:

$$v_c, \quad \omega_c \quad (12.114)$$

from the nonlinear kinematic controller.

Step 2: Apply Command Limits

Apply:

$$-0.22 \leq v_c \leq 0.22 \quad (12.115)$$

and:

$$-2.5 \leq \omega_c \leq 2.5. \quad (12.116)$$

Step 3: Obtain Actual Velocities

Obtain:

$$v, \quad \omega \quad (12.117)$$

from the robot feedback.

Step 4: Calculate Linear Velocity Error

$$e_v = v_c - v. \quad (12.118)$$

Step 5: Calculate Angular Velocity Error

$$e_\omega = \omega_c - \omega. \quad (12.119)$$

Step 6: Apply Dynamic Control Law

Generate the corresponding linear and angular control actions.

Step 7: Apply Control to Robot

The control action produces changes in the robot velocity.

Step 8: Feed Back Actual Velocity

The new values of:

$$v, \quad \omega \quad (12.120)$$

are fed back to the controller.

Step 9: Repeat

The process is continuously repeated during trajectory tracking.

12.24 Expected Dynamic Controller Response

A properly functioning dynamic velocity controller should produce the following behaviour:

1. The actual velocity initially differs from the commanded velocity.
2. The controller detects the velocity error.
3. A corrective control action is generated.
4. The actual velocity moves toward the commanded value.
5. The velocity error decreases.
6. The actual velocity eventually tracks the commanded velocity with a small error.

Therefore:

$$\boxed{e_v(t) \rightarrow 0} \quad (12.121)$$

and:

$$\boxed{e_\omega(t) \rightarrow 0} \quad (12.122)$$

This allows the outer kinematic controller to operate effectively.

12.25 Importance of the Dynamic Controller in Trajectory Tracking

The kinematic controller determines the required velocity based on geometric tracking errors, but the dynamic controller ensures that these velocity commands are actually achieved by the robot.

Therefore, both controllers are necessary.

The kinematic controller addresses:

$$\boxed{\text{Where should the robot go?}} \quad (12.123)$$

The dynamic controller addresses:

$$\boxed{\text{How accurately can the robot achieve the required motion?}} \quad (12.124)$$

Thus:

$$\boxed{\text{Kinematic Control} + \text{Dynamic Control} = \text{Complete Trajectory Tracking}} \quad (12.125)$$

12.26 Controller Performance Parameters

The performance of the dynamic velocity controller can be evaluated using the following parameters.

12.26.1 Linear Velocity Tracking Error

$$e_v(t) = v_c(t) - v(t) \quad (12.126)$$

12.26.2 Angular Velocity Tracking Error

$$e_{\omega}(t) = \omega_c(t) - \omega(t) \quad (12.127)$$

12.26.3 Maximum Linear Velocity Error

$$e_{v,\max} = \max |e_v(t)| \quad (12.128)$$

12.26.4 Maximum Angular Velocity Error

$$e_{\omega,\max} = \max |e_{\omega}(t)| \quad (12.129)$$

12.26.5 Linear Velocity RMSE

$$RMSE_v = \sqrt{\frac{1}{N} \sum_{k=1}^N e_v^2[k]} \quad (12.130)$$

12.26.6 Angular Velocity RMSE

$$RMSE_{\omega} = \sqrt{\frac{1}{N} \sum_{k=1}^N e_{\omega}^2[k]} \quad (12.131)$$

These values can be used in the performance-analysis section of the report.

12.27 Relation Between Velocity Error and Trajectory Error

The dynamic velocity error affects the overall trajectory-tracking performance.

If:

$$v \neq v_c \quad (12.132)$$

or:

$$\omega \neq \omega_c, \quad (12.133)$$

then the robot's actual motion differs from the motion requested by the kinematic controller.

This can cause:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}} \quad (12.134)$$

to increase.

Therefore, accurate velocity tracking contributes directly to accurate trajectory tracking.

The complete relationship is:

$$\boxed{\text{Velocity Tracking} \rightarrow \text{Robot Motion} \rightarrow \text{Trajectory Tracking Accuracy}} \quad (12.135)$$

12.28 Summary

This chapter presented the **dynamic velocity controller** used as the second control stage in the TurtleBot3 trajectory-tracking system.

The nonlinear kinematic controller generates:

$$v_c, \quad \omega_c. \quad (12.136)$$

These commands are compared with the actual robot velocities:

$$v, \quad \omega. \quad (12.137)$$

The corresponding velocity errors are:

$$\boxed{e_v = v_c - v} \quad (12.138)$$

and:

$$\boxed{e_\omega = \omega_c - \omega} \quad (12.139)$$

The dynamic controller uses these errors to generate corrective control actions so that:

$$\boxed{v \rightarrow v_c} \quad (12.140)$$

and:

$$\boxed{\omega \rightarrow \omega_c} \quad (12.141)$$

The specified velocity limits are:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (12.142)$$

and:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (12.143)$$

The complete two-level control architecture is:

$$\begin{array}{l} \text{Trajectory} \rightarrow \text{Tracking Error} \rightarrow \text{Kinematic Controller} \\ \quad \rightarrow (v_c, \omega_c) \rightarrow \text{Dynamic Velocity Controller} \\ \quad \rightarrow \text{TurtleBot3} \rightarrow \text{Feedback} \end{array} \quad (12.144)$$

The dynamic controller therefore forms the **inner velocity-control loop**, while the nonlinear kinematic controller forms the **outer trajectory-tracking loop**.

Its successful operation is essential for ensuring that the velocity commands generated by the kinematic controller are accurately reproduced by the robot, thereby improving the overall trajectory-tracking performance.

Simulation Setup

13.1 Simulation Environment

The simulation of the proposed TurtleBot3 trajectory-tracking control system is carried out using a combination of MATLAB, Simulink, ROS 2, and the TurtleBot3 model. These tools provide the required environment for implementing the controller, generating the reference trajectory, obtaining the robot state, and evaluating the tracking performance.

The main components of the simulation environment are:

- **MATLAB:** Used for mathematical calculations, controller development, signal processing, and analysis of simulation results.
- **Simulink:** Used to develop the complete closed-loop control architecture, including the trajectory generator, error calculation, coordinate transformation, kinematic controller, velocity saturation, and dynamic controller.
- **ROS 2:** Used as the communication framework between the controller and the TurtleBot3 system. It provides the required feedback and command interfaces.
- **Gazebo:** Used as the simulation environment when required for representing the robot and its operating environment.
- **TurtleBot3 Model:** Used as the mobile robot model for evaluating the proposed trajectory-tracking controller.

13.2 Robot Parameters

The robot parameters used in the simulation are summarized in Table 13.1.

Table 13.1: Robot parameters used in the simulation

Parameter	Value
Robot model	TurtleBot3
Control framework	ROS 2
Simulation platform	MATLAB/Simulink

13.3 Controller Parameters

The controller parameters used in the implemented control system are listed in Table 13.2.

Table 13.2: Controller parameters

Parameter	Value
k_x	1
k_ϕ	0.33
K_a	25
K_b	8
Lateral gain	45

The parameter k_x determines the contribution of the forward-position tracking error to the commanded linear velocity. The parameter k_ϕ determines the contribution of the orientation error to the commanded angular velocity.

The parameters K_a and K_b are used in the dynamic velocity control stage. The lateral gain is associated with the lateral tracking-error correction in the angular-velocity control law.

The values listed in Table 13.2 are the values used in the implemented simulation model.

13.4 Trajectory Parameters

A circular reference trajectory is used for evaluating the trajectory-tracking performance of the TurtleBot3 system.

The trajectory parameters are given in Table 13.3.

Table 13.3: Reference trajectory parameters

Parameter	Value
Radius R	0.4 m
Angular velocity ω	0.5 rad/s
Linear velocity v_d	0.2 m/s

The desired circular trajectory is described by

$$x_d(t) = R \cos(\omega t), \quad (13.1)$$

$$y_d(t) = R \sin(\omega t). \quad (13.2)$$

Using the specified radius and angular velocity,

$$x_d(t) = 0.4 \cos(0.5t), \quad (13.3)$$

and

$$y_d(t) = 0.4 \sin(0.5t). \quad (13.4)$$

The corresponding linear velocity is given by

$$v_d = R\omega. \quad (13.5)$$

Substituting the simulation parameters gives

$$v_d = (0.4)(0.5) \quad (13.6)$$

and therefore,

$$\boxed{v_d = 0.2 \text{ m/s}}. \quad (13.7)$$

13.5 Velocity Limits

To ensure that the controller commands remain within the specified operating range, saturation limits are applied to the linear and angular velocity commands.

The velocity limits used in the simulation are summarized in Table 13.4.

Table 13.4: Velocity saturation limits

Variable	Minimum	Maximum
v_c	-0.22 m/s	0.22 m/s
ω_c	-2.5 rad/s	2.5 rad/s

The linear velocity command is therefore constrained according to

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (13.8)$$

while the angular velocity command satisfies

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s.}} \quad (13.9)$$

The saturation limits prevent the controller from generating commands outside the specified operating range of the robot.

13.6 Simulation Configuration Summary

The complete simulation configuration is summarized in Table 13.5.

Table 13.5: Complete simulation configuration

Parameter	Value
Robot	TurtleBot3
MATLAB/Simulink	Used
ROS 2	Used
Reference trajectory	Circular
Radius	0.4 m
Angular velocity	0.5 rad/s
Linear velocity	0.2 m/s
k_x	1
k_ϕ	0.33
K_a	25
K_b	8
Lateral gain	45
Linear velocity limit	$\pm 0.22 \text{ m/s}$
Angular velocity limit	$\pm 2.5 \text{ rad/s}$

13.7 Summary

This chapter described the simulation environment and the parameters used for evaluating the proposed TurtleBot3 trajectory-tracking controller.

The simulation uses MATLAB and Simulink for implementation of the control system, ROS 2 for communication, and the TurtleBot3 model for robot-motion evaluation. A circular reference trajectory with a radius of 0.4 m, angular velocity of 0.5 rad/s, and linear velocity of 0.2 m/s is used.

The controller parameters are

$$k_x = 1, \quad k_\phi = 0.33, \quad K_a = 25, \quad K_b = 8,$$

with a lateral gain of

$$45.$$

The velocity commands are constrained by

$$-0.22 \leq v_c \leq 0.22 \text{ m/s}$$

and

$$-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}.$$

These parameters form the basis for the simulation results and performance analysis presented in the subsequent chapter.

Simulation and Implementation

14.1 Introduction

The complete trajectory-tracking system developed for TurtleBot3 is implemented and evaluated using a simulation-based approach. The simulation provides a safe and repeatable environment for testing the proposed control architecture before applying the controller to the physical robot.

The complete system combines:

- reference trajectory generation,
- ROS 2 odometry feedback,
- position and orientation error calculation,
- global-to-robot coordinate transformation,
- nonlinear kinematic control,
- velocity saturation,
- dynamic velocity control,
- robot motion,
- feedback of the actual robot state.

The overall closed-loop structure is:

Reference Trajectory → Tracking Error → Kinematic Controller
 → Dynamic Controller → Robot → Odometry Feedback

(14.1)

The purpose of the simulation is to verify whether the robot can follow the specified circular trajectory while maintaining acceptable position, orientation, and velocity errors.

14.2 Simulation Objective

The main objective of the simulation is to verify the performance of the complete trajectory-tracking controller.

The simulation is used to determine whether:

$$\boxed{x \rightarrow x_d} \quad (14.2)$$

$$\boxed{y \rightarrow y_d} \quad (14.3)$$

and:

$$\boxed{\phi \rightarrow \phi_d} \quad (14.4)$$

At the same time, the inner velocity-control loop should ensure:

$$\boxed{v \rightarrow v_c} \quad (14.5)$$

and:

$$\boxed{\omega \rightarrow \omega_c} \quad (14.6)$$

Therefore, the simulation evaluates both:

$$\boxed{\text{Trajectory Tracking}} \quad (14.7)$$

and:

$$\boxed{\text{Velocity Tracking}} \quad (14.8)$$

14.3 Simulation Environment

The proposed control system is implemented using a combination of MATLAB/Simulink and ROS 2.

The main components of the simulation environment are:

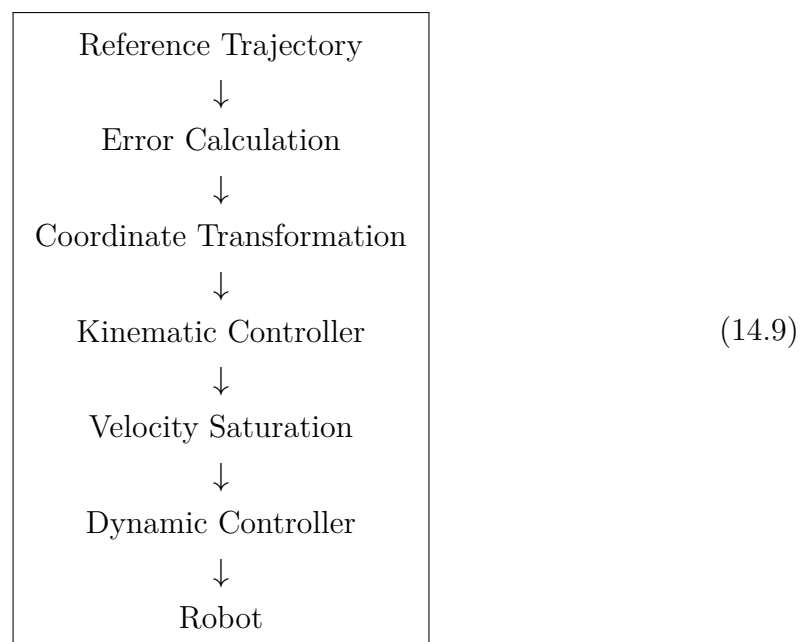
1. MATLAB/Simulink
2. ROS 2 communication
3. TurtleBot3 model
4. Odometry feedback
5. Reference trajectory generator
6. Kinematic controller
7. Dynamic velocity controller

The interaction between these components allows the controller to receive robot-state information and generate the required velocity commands.

14.4 Overall Simulink Model

The complete Simulink model consists of several interconnected subsystems.

The major sections are:



The robot output is fed back through odometry:

$$\boxed{\text{Robot} \rightarrow \text{ROS 2 /odom} \rightarrow \text{Simulink}} \quad (14.10)$$

This creates a closed-loop system.

The complete Simulink model developed for the proposed trajectory-tracking system is shown in Figure 14.1. The model integrates the reference trajectory generation, ROS 2 odometry feedback, tracking-error calculation, coordinate transformation, nonlinear kinematic controller, velocity saturation, dynamic velocity controller, and robot motion.

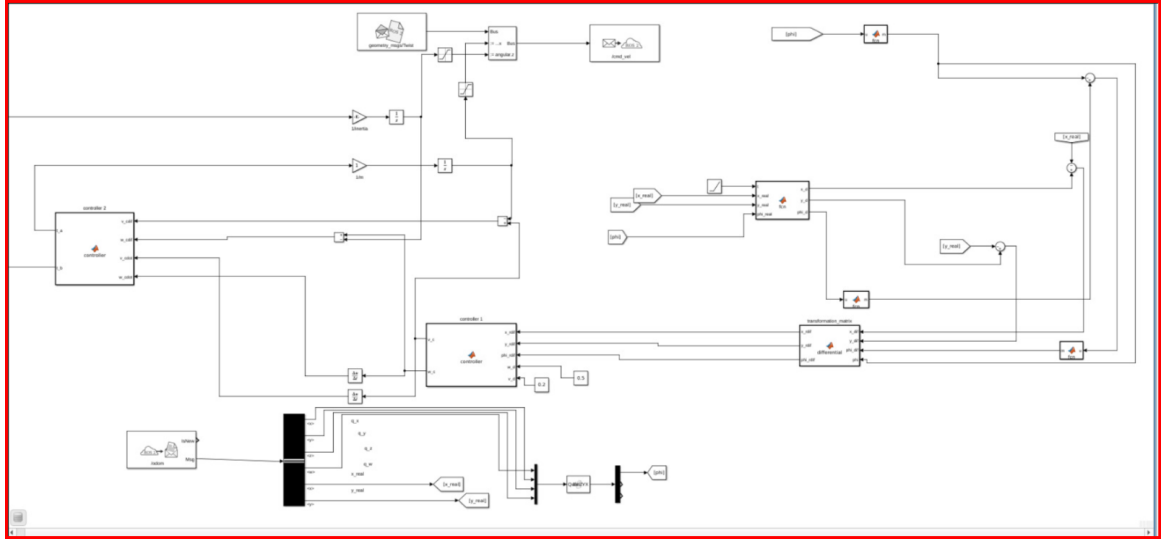


Figure 14.1: Complete Simulink model of the TurtleBot3 trajectory-tracking system

As shown in Figure 14.1, the complete control architecture is implemented using interconnected Simulink blocks. The robot-state information obtained through ROS 2 odometry is processed to obtain the actual position and orientation. These signals are compared with the desired trajectory to calculate the tracking errors.

The tracking errors are then transformed into the robot coordinate frame and supplied to the nonlinear kinematic controller. The controller generates the commanded linear and angular velocities, which are subsequently passed through the velocity-control stages before being applied to the robot.

The robot's actual state is continuously fed back through the ROS 2 odometry path, thereby completing the closed-loop trajectory-tracking system.

14.5 Reference Trajectory Configuration

The reference trajectory used for simulation is circular.

The desired position is given by:

$$\boxed{x_d(t) = R \cos(\omega_d t)} \quad (14.11)$$

and:

$$\boxed{y_d(t) = R \sin(\omega_d t)} \quad (14.12)$$

with:

$$R = 0.4 \text{ m} \quad (14.13)$$

and:

$$\omega_d = 0.5 \text{ rad/s.} \quad (14.14)$$

The desired orientation is:

$$\boxed{\phi_d(t) = \omega_d t} \quad (14.15)$$

The corresponding desired linear velocity is:

$$v_d = R\omega_d \quad (14.16)$$

Substituting the values:

$$v_d = (0.4)(0.5) \quad (14.17)$$

Therefore:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (14.18)$$

Hence, the reference inputs used during simulation are:

$$\boxed{v_d = 0.2 \text{ m/s}, \quad \omega_d = 0.5 \text{ rad/s}} \quad (14.19)$$

14.6 Initial Condition

The initial condition is selected such that the reference trajectory is synchronized with the robot's initial state.

Therefore:

$$x_d(0) = x(0) \quad (14.20)$$

$$y_d(0) = y(0) \quad (14.21)$$

and:

$$\phi_d(0) = \phi(0). \quad (14.22)$$

This ensures that the simulation begins with consistent desired and actual states.

The initial synchronization also prevents an unnecessarily large initial transient error.

14.7 Odometry Feedback

The actual robot state is obtained from the ROS 2 odometry feedback.

The required position signals are:

$$x_{\text{real}} \quad (14.23)$$

and:

$$y_{\text{real}}. \quad (14.24)$$

The orientation is obtained from the quaternion:

$$[q_x, q_y, q_z, q_w]. \quad (14.25)$$

The quaternion is processed through the quaternion-to-Euler conversion stage to obtain the yaw:

$$\phi. \quad (14.26)$$

Therefore:

$$\boxed{\text{Odometry} \rightarrow (x, y, q_x, q_y, q_z, q_w) \rightarrow \phi} \quad (14.27)$$

The actual state supplied to the controller is:

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ \phi \end{bmatrix} \quad (14.28)$$

14.8 Tracking-Error Calculation

The desired and actual states are compared to determine the tracking errors.

The X -position error is:

$$\boxed{e_x = x_d - x} \quad (14.29)$$

The Y -position error is:

$$\boxed{e_y = y_d - y} \quad (14.30)$$

The orientation error is:

$$\boxed{e_\phi = \phi_d - \phi} \quad (14.31)$$

Thus:

$$\mathbf{e} = \begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} \quad (14.32)$$

This error vector is used by the coordinate-transformation stage.

14.9 Global-to-Robot Transformation

The global position errors are transformed according to the current robot heading.

The forward error is:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (14.33)$$

The lateral error is:

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi \quad (14.34)$$

The orientation error is:

$$\phi_{r,\text{dif}} = \phi_d - \phi \quad (14.35)$$

Therefore, the controller receives:

$$\mathbf{e}_r = \begin{bmatrix} x_{r,\text{dif}} \\ y_{r,\text{dif}} \\ \phi_{r,\text{dif}} \end{bmatrix} \quad (14.36)$$

14.10 Nonlinear Kinematic Controller

The robot-frame errors are supplied to the nonlinear kinematic controller.

The commanded linear velocity is:

$$v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \quad (14.37)$$

The commanded angular velocity is:

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d \quad (14.38)$$

The controller therefore combines:

$$\text{Feedback} + \text{Feedforward} \quad (14.39)$$

to generate the desired velocity commands.

14.11 Velocity Saturation

The commands generated by the nonlinear kinematic controller are passed through saturation blocks.

The linear velocity is limited to:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (14.40)$$

and the angular velocity is limited to:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (14.41)$$

The purpose of saturation is to ensure that the controller commands remain within the specified operating limits.

The saturated commands are then supplied to the dynamic velocity-control stage.

14.12 Dynamic Velocity Control

The dynamic controller compares the commanded velocities with the actual velocities.

The linear velocity error is:

$$\boxed{e_v = v_c - v} \quad (14.42)$$

and the angular velocity error is:

$$\boxed{e_\omega = \omega_c - \omega} \quad (14.43)$$

These errors are used to generate corrective dynamic control actions.

The objective is:

$$\boxed{e_v \rightarrow 0} \quad (14.44)$$

and:

$$\boxed{e_\omega \rightarrow 0} \quad (14.45)$$

Thus, the actual robot velocities should closely follow the commands generated

by the outer kinematic controller.

14.13 Robot Motion Model

The commanded motion is represented using the differential-drive kinematic model:

$$\dot{x} = v \cos \phi \quad (14.46)$$

$$\dot{y} = v \sin \phi \quad (14.47)$$

$$\dot{\phi} = \omega \quad (14.48)$$

The actual robot velocity therefore determines the rate of change of:

$$x, \quad y, \quad \phi. \quad (14.49)$$

These states are continuously fed back to the controller.

14.14 Closed-Loop Simulation

The complete simulation operates as a closed-loop system.

At each simulation instant, the following sequence is executed.

Step 1

Generate the desired trajectory:

$$x_d, \quad y_d, \quad \phi_d. \quad (14.50)$$

Step 2

Obtain the actual robot state:

$$x, \quad y, \quad \phi. \quad (14.51)$$

Step 3

Calculate tracking errors:

$$e_x, \quad e_y, \quad e_\phi. \quad (14.52)$$

Step 4

Transform the errors into the robot frame:

$$x_{r,dif}, \quad y_{r,dif}, \quad \phi_{r,dif}. \quad (14.53)$$

Step 5

Calculate:

$$v_c, \quad \omega_c. \quad (14.54)$$

Step 6

Apply velocity saturation.

Step 7

Use the dynamic controller to track the commanded velocities.

Step 8

Apply the resulting control action to the robot.

Step 9

Obtain updated odometry and repeat the process.

Thus:

Reference → Error → Control → Robot
→ Feedback → Error

(14.55)

14.15 Simulation Time

The simulation time should be selected according to the number of trajectory revolutions or the required observation interval.

For one complete revolution of the reference circle:

$$T = \frac{2\pi}{\omega_d}. \quad (14.56)$$

With:

$$\omega_d = 0.5 \text{ rad/s}, \quad (14.57)$$

we obtain:

$$T = \frac{2\pi}{0.5} \quad (14.58)$$

Therefore:

$$\boxed{T \approx 12.57 \text{ s}} \quad (14.59)$$

A simulation covering approximately 12.57 seconds is sufficient to observe one complete ideal reference revolution.

A longer simulation can be used to evaluate repeated trajectory tracking.

14.16 Signals to Be Observed

Several signals should be monitored during simulation.

Desired Position

$$x_d(t), \quad y_d(t) \quad (14.60)$$

Actual Position

$$x(t), \quad y(t) \quad (14.61)$$

Desired Orientation

$$\phi_d(t) \quad (14.62)$$

Actual Orientation

$$\phi(t) \quad (14.63)$$

Position Errors

$$e_x(t), \quad e_y(t) \quad (14.64)$$

Orientation Error

$$e_\phi(t) \quad (14.65)$$

Commanded Velocities

$$v_c(t), \quad \omega_c(t) \quad (14.66)$$

Actual Velocities

$$v(t), \quad \omega(t) \quad (14.67)$$

Velocity Errors

$$e_v(t), \quad e_\omega(t) \quad (14.68)$$

These signals provide the necessary information for evaluating the controller.

14.17 Desired Versus Actual Trajectory

One of the most important simulation plots is the X - Y trajectory plot.

The desired path is:

$$(x_d(t), y_d(t)) \quad (14.69)$$

and the actual robot path is:

$$(x(t), y(t)). \quad (14.70)$$

The two trajectories should be plotted on the same graph.

An ideal result would show the actual trajectory closely overlapping the desired circular trajectory.

The trajectory comparison can be represented as:

$$\boxed{\text{Desired Circular Path} \quad \text{vs.} \quad \text{Actual Robot Path}} \quad (14.71)$$

This plot provides a direct visual indication of the trajectory-tracking performance.

14.18 Position Error Results

The position errors:

$$e_x(t) \quad (14.72)$$

and:

$$e_y(t) \quad (14.73)$$

should be plotted against time.

A successful controller should show that the errors remain bounded and generally decrease after the initial transient.

The desired behaviour is:

$$\boxed{e_x(t) \rightarrow 0} \quad (14.74)$$

and:

$$\boxed{e_y(t) \rightarrow 0} \quad (14.75)$$

Small oscillations may occur depending on the controller gains and robot dynamics.

14.19 Orientation Error Results

The orientation error:

$$e_\phi(t) \tag{14.76}$$

should also be plotted.

The desired condition is:

$$\boxed{e_\phi(t) \rightarrow 0} \tag{14.77}$$

This indicates that the robot's orientation is converging toward the desired heading.

The orientation response is particularly important for a circular trajectory because the desired heading continuously changes with time.

14.20 Velocity Tracking Results

The commanded and actual velocities should be compared.

For linear velocity:

$$v_c(t) \tag{14.78}$$

should be compared with:

$$v(t). \tag{14.79}$$

For angular velocity:

$$\omega_c(t) \tag{14.80}$$

should be compared with:

$$\omega(t). \tag{14.81}$$

The desired result is:

$$\boxed{v(t) \approx v_c(t)} \tag{14.82}$$

and:

$$\boxed{\omega(t) \approx \omega_c(t)} \quad (14.83)$$

The corresponding errors:

$$e_v = v_c - v \quad (14.84)$$

and:

$$e_\omega = \omega_c - \omega \quad (14.85)$$

should remain small.

14.21 Position Tracking Performance

The overall positional error can be calculated as:

$$\boxed{e_p = \sqrt{e_x^2 + e_y^2}} \quad (14.86)$$

This provides a scalar measure of the robot's distance from the desired position.

A smaller value of e_p indicates better trajectory tracking.

The maximum positional error can be calculated as:

$$\boxed{e_{p,\max} = \max(e_p)} \quad (14.87)$$

The average positional error can also be evaluated.

14.22 RMSE Analysis

Root Mean Square Error (RMSE) can be used to quantify the tracking performance.

For the X -position:

$$\boxed{RMSE_x = \sqrt{\frac{1}{N} \sum_{k=1}^N e_x^2[k]}} \quad (14.88)$$

For the Y -position:

$$RMSE_y = \sqrt{\frac{1}{N} \sum_{k=1}^N e_y^2[k]} \quad (14.89)$$

For orientation:

$$RMSE_\phi = \sqrt{\frac{1}{N} \sum_{k=1}^N e_\phi^2[k]} \quad (14.90)$$

These values provide numerical measures of tracking accuracy.

14.23 Velocity RMSE

The velocity-control performance can similarly be evaluated using:

$$RMSE_v = \sqrt{\frac{1}{N} \sum_{k=1}^N e_v^2[k]} \quad (14.91)$$

and:

$$RMSE_\omega = \sqrt{\frac{1}{N} \sum_{k=1}^N e_\omega^2[k]} \quad (14.92)$$

Smaller RMSE values indicate better velocity tracking.

14.24 Expected Simulation Behaviour

For a properly tuned controller, the simulation should demonstrate:

1. The robot starts close to the initial reference position.
2. The actual trajectory follows the circular reference.
3. The position errors remain bounded.
4. The orientation error remains controlled.
5. The commanded velocities remain within the specified limits.
6. The actual velocity tracks the commanded velocity.
7. The robot completes the circular trajectory without divergence.

Therefore, the expected overall behaviour is:

$$\boxed{\text{Stable and Accurate Circular Trajectory Tracking}} \quad (14.93)$$

14.25 Effect of Controller Gains

The controller gains influence the response of the robot.

The forward-position gain:

$$k_x \quad (14.94)$$

affects the response of the linear velocity to forward tracking error.

The orientation gain:

$$k_\phi \quad (14.95)$$

affects the response of angular velocity to heading error.

If the gains are too small, the robot may respond slowly to tracking errors.

If the gains are too large, the system may exhibit excessive corrections or oscillations.

Therefore, appropriate gain selection is important for obtaining a stable and accurate response.

The final report should state the exact gain values used in the final simulation model.

14.26 Simulation Procedure

The complete simulation procedure can be written as follows.

Step 1 — Initialize the Robot

Set the initial robot position and orientation.

Step 2 — Initialize the Reference

Synchronize:

$$x_d(0), \quad y_d(0), \quad \phi_d(0) \quad (14.96)$$

with the initial robot state.

Step 3 — Generate Circular Reference

Generate:

$$x_d(t), \quad y_d(t), \quad \phi_d(t). \quad (14.97)$$

Step 4 — Obtain Odometry

Read:

$$x, \quad y, \quad \phi. \quad (14.98)$$

Step 5 — Calculate Errors

Calculate:

$$e_x, \quad e_y, \quad e_\phi. \quad (14.99)$$

Step 6 — Transform Errors

Calculate:

$$x_{r,dif}, \quad y_{r,dif}, \quad \phi_{r,dif}. \quad (14.100)$$

Step 7 — Apply Kinematic Controller

Calculate:

$$v_c, \quad \omega_c. \quad (14.101)$$

Step 8 — Apply Saturation

Limit:

$$v_c \tag{14.102}$$

and:

$$\omega_c. \tag{14.103}$$

Step 9 — Apply Dynamic Controller

Calculate the velocity errors and corresponding dynamic control actions.

Step 10 — Update Robot Motion

Update:

$$x, \quad y, \quad \phi. \tag{14.104}$$

Step 11 — Feed Back the New State

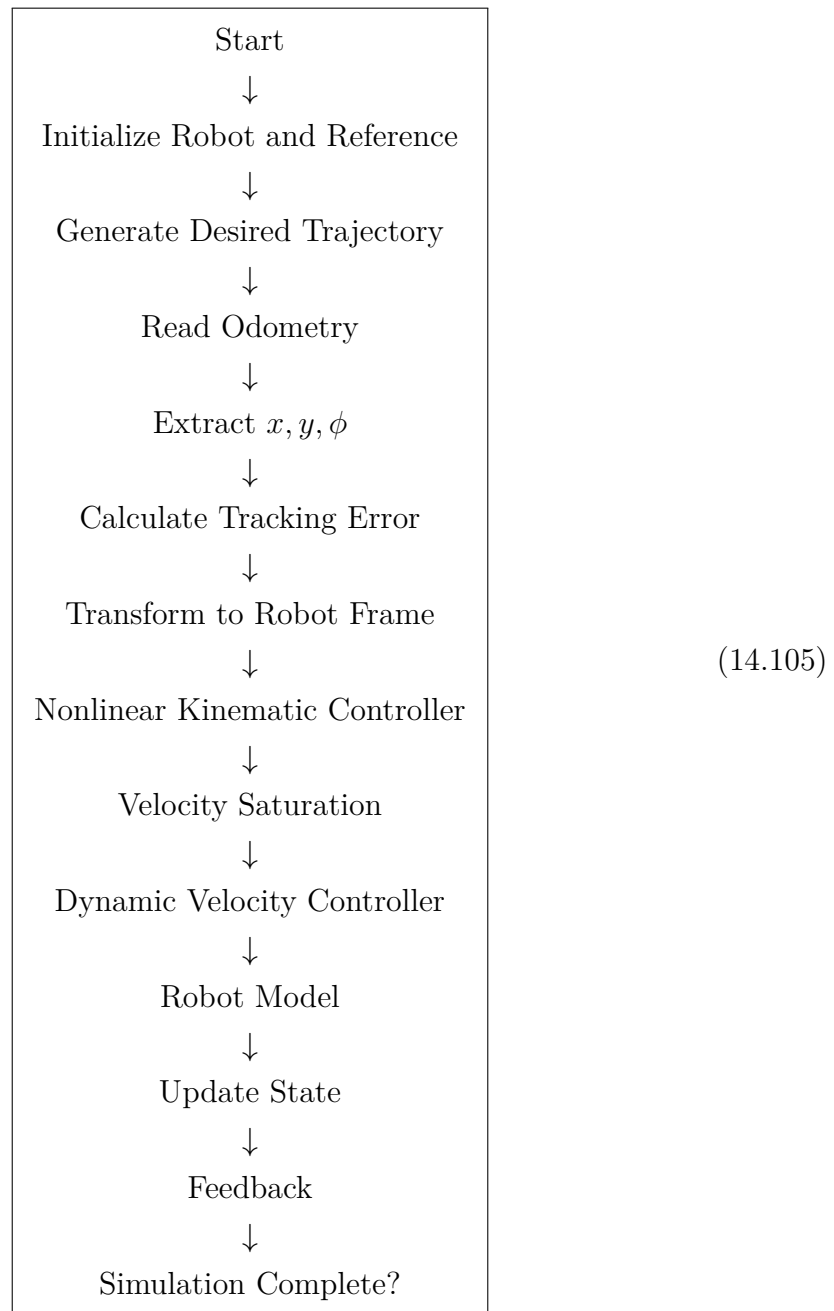
Return the updated state to the controller.

Step 12 — Continue Until Simulation Ends

Repeat the complete process for the selected simulation time.

14.27 Complete Simulation Flowchart

The simulation algorithm can be represented as:



If the simulation is not complete, the process returns to the odometry-reading stage.

14.28 Implementation of ROS 2 Communication

ROS 2 provides the communication interface between the robot and the control system.

The feedback path is:

$$\boxed{\text{Robot} \rightarrow \text{/odom} \rightarrow \text{ROS 2 Subscriber} \rightarrow \text{Simulink}} \quad (14.106)$$

The control-command path operates in the opposite direction:

$$\boxed{\text{Simulink} \rightarrow \text{Velocity Command} \rightarrow \text{ROS 2} \rightarrow \text{Robot}} \quad (14.107)$$

Thus, ROS 2 acts as the communication layer connecting the controller and robot.

14.29 Implementation of Feedback Loop

The feedback loop continuously updates the controller.

The robot produces:

$$x, \quad y, \quad \phi, \quad v, \quad \omega. \quad (14.108)$$

These quantities are fed back to the control system.

The controller compares them with:

$$x_d, \quad y_d, \quad \phi_d, \quad v_c, \quad \omega_c. \quad (14.109)$$

Therefore:

$$\boxed{\text{Desired} - \text{Actual} = \text{Error}} \quad (14.110)$$

The error is then used to generate the next control command.

This process repeats throughout the simulation.

14.30 Simulation Results to Be Included

The following results should be included in the report after running the final simulation.

1. Desired and Actual X-Y Trajectory

Plot:

$$(x_d, y_d) \tag{14.111}$$

and:

$$(x, y). \tag{14.112}$$

2. X-Position Tracking

Plot:

$$x_d(t) \tag{14.113}$$

and:

$$x(t). \tag{14.114}$$

3. Y-Position Tracking

Plot:

$$y_d(t) \tag{14.115}$$

and:

$$y(t). \tag{14.116}$$

4. Orientation Tracking

Plot:

$$\phi_d(t) \tag{14.117}$$

and:

$$\phi(t). \tag{14.118}$$

5. Tracking Errors

Plot:

$$e_x(t), \quad e_y(t), \quad e_\phi(t). \quad (14.119)$$

6. Linear Velocity

Compare:

$$v_c(t) \quad (14.120)$$

and:

$$v(t). \quad (14.121)$$

7. Angular Velocity

Compare:

$$\omega_c(t) \quad (14.122)$$

and:

$$\omega(t). \quad (14.123)$$

8. Velocity Errors

Plot:

$$e_v(t) \quad (14.124)$$

and:

$$e_\omega(t). \quad (14.125)$$

These plots provide the evidence required to evaluate the proposed control system.

14.31 Performance Evaluation Table

After obtaining the simulation results, the following table can be included in the report.

Table 14.1: Simulation Performance Parameters

Performance Parameter	Result
Maximum X -position error	To be obtained
Maximum Y -position error	To be obtained
Maximum orientation error	To be obtained
Position RMSE	To be obtained
Orientation RMSE	To be obtained
Linear velocity RMSE	To be obtained
Angular velocity RMSE	To be obtained
Maximum linear velocity	To be obtained
Maximum angular velocity	To be obtained

The values should be calculated from the actual simulation data rather than estimated.

14.32 Discussion of Simulation Results

The simulation results should be discussed based on the following observations.

Trajectory Tracking

The actual trajectory should be compared with the reference circular trajectory.

If the two paths are closely matched, the controller demonstrates good trajectory-tracking performance.

Position Error

The X - and Y -position errors should be examined to determine whether they converge to small values.

Orientation

The actual heading should follow the desired heading throughout the circular trajectory.

Velocity Tracking

The actual linear and angular velocities should closely follow their commanded values.

Stability

The errors should remain bounded and should not increase continuously with time.

14.33 Overall Simulation Architecture

The complete simulation architecture can be summarized as:

$$\begin{array}{c} \boxed{\text{Circular Reference Generator}} \\ \downarrow \\ x_d, y_d, \phi_d, v_d, \omega_d \end{array} \quad (14.126)$$

$$\Downarrow \quad (14.127)$$

$$\boxed{\text{Tracking Error Calculation}} \quad (14.128)$$

$$\Downarrow \quad (14.129)$$

$$\boxed{e_x, e_y, e_\phi} \quad (14.130)$$

$$\Downarrow \quad (14.131)$$

$$\boxed{\text{Global-to-Robot Transformation}} \quad (14.132)$$

$$\Downarrow \quad (14.133)$$

$$\boxed{x_{r,\text{dif}}, y_{r,\text{dif}}, \phi_{r,\text{dif}}} \quad (14.134)$$

$$\Downarrow \quad (14.135)$$

$$\boxed{\text{Nonlinear Kinematic Controller}} \quad (14.136)$$

$$\Downarrow \quad (14.137)$$

$$\boxed{v_c, \omega_c} \quad (14.138)$$

$$\Downarrow \quad (14.139)$$

$$\boxed{\text{Saturation}} \quad (14.140)$$

$$\Downarrow \quad (14.141)$$

$$\boxed{\text{Dynamic Velocity Controller}} \quad (14.142)$$

$$\Downarrow \quad (14.143)$$

$$\boxed{\text{TurtleBot3}} \quad (14.144)$$

$$\Downarrow \quad (14.145)$$

$$\boxed{\text{ROS 2 /odom Feedback}} \quad (14.146)$$

$$\circlearrowright \quad (14.147)$$

14.34 Summary

This chapter presented the simulation and implementation of the complete TurtleBot3 trajectory-tracking system.

The system combines a circular reference trajectory with a nonlinear kinematic controller and a dynamic velocity-control loop.

The reference trajectory is:

$$x_d = 0.4 \cos(0.5t) \quad (14.148)$$

$$y_d = 0.4 \sin(0.5t) \quad (14.149)$$

$$\phi_d = 0.5t. \quad (14.150)$$

The corresponding desired velocities are:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (14.151)$$

and:

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (14.152)$$

The tracking errors are calculated as:

$$e_x = x_d - x \quad (14.153)$$

$$e_y = y_d - y \quad (14.154)$$

$$e_\phi = \phi_d - \phi. \quad (14.155)$$

These errors are transformed into the robot frame:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (14.156)$$

$$y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi. \quad (14.157)$$

The nonlinear kinematic controller then generates:

$$v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}}) \quad (14.158)$$

and:

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d. \quad (14.159)$$

The velocity commands are constrained by:

$$-0.22 \leq v_c \leq 0.22 \text{ m/s} \quad (14.160)$$

and:

$$-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}. \quad (14.161)$$

The dynamic velocity controller then attempts to ensure:

$$v \rightarrow v_c \quad (14.162)$$

and:

$$\omega \rightarrow \omega_c. \quad (14.163)$$

The complete closed-loop implementation is:

Reference Trajectory $\rightarrow$ Tracking Error $\rightarrow$ Coordinate Transformation $\rightarrow$ Nonlinear Kinematic Controller $\rightarrow$ Velocity Saturation $\rightarrow$ Dynamic Controller $\rightarrow$ TurtleBot3 $\rightarrow$ ROS 2 Feedback

(14.164)

The simulation results obtained from the X - Y trajectory, position errors, orientation response, velocity tracking, and error metrics provide the basis for evaluating the effectiveness of the proposed controller.

Simulation Results and Performance Analysis

15.1 Introduction

This chapter presents the simulation results obtained from the complete TurtleBot3 trajectory-tracking system. The main objective of the simulation is to evaluate the ability of the proposed controller to make the robot follow the desired circular trajectory.

The complete control architecture consists of:

- circular reference trajectory generation,
- actual robot-state feedback,
- tracking-error calculation,
- global-to-robot coordinate transformation,
- nonlinear kinematic controller,
- velocity saturation,
- dynamic velocity controller,
- robot motion,
- feedback of the actual robot state.

The principal simulation result is the X - Y trajectory obtained from the Simulink model. The graph shows the desired circular path and the simulated robot trajectory. The result is analyzed quantitatively in this chapter.

15.2 Simulation Parameters

The reference trajectory used for the simulation is a circular trajectory.

The desired X - and Y -coordinates are defined as:

$$\boxed{x_d(t) = R \cos(\omega_d t)} \quad (15.1)$$

and:

$$\boxed{y_d(t) = R \sin(\omega_d t)} \quad (15.2)$$

where:

$$R = 0.4 \text{ m} \quad (15.3)$$

and:

$$\omega_d = 0.5 \text{ rad/s.} \quad (15.4)$$

Therefore:

$$\boxed{x_d(t) = 0.4 \cos(0.5t)} \quad (15.5)$$

$$\boxed{y_d(t) = 0.4 \sin(0.5t)} \quad (15.6)$$

The desired orientation is:

$$\boxed{\phi_d(t) = 0.5t} \quad (15.7)$$

and the desired linear velocity is:

$$v_d = R\omega_d. \quad (15.8)$$

Substituting the values:

$$v_d = (0.4)(0.5) \quad (15.9)$$

Therefore:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (15.10)$$

Thus, the main reference parameters are:

Table 15.1: Simulation Reference Parameters

Parameter	Symbol	Value
Reference radius	R	0.4 m
Desired linear velocity	v_d	0.2 m/s
Desired angular velocity	ω_d	0.5 rad/s

15.3 Theoretical Circular Trajectory

The reference trajectory satisfies:

$$x_d^2 + y_d^2 = R^2. \quad (15.11)$$

Since:

$$R = 0.4 \text{ m}, \quad (15.12)$$

we obtain:

$$x_d^2 + y_d^2 = (0.4)^2 \quad (15.13)$$

and therefore:

$$\boxed{x_d^2 + y_d^2 = 0.16 \text{ m}^2} \quad (15.14)$$

Therefore, the theoretical reference trajectory is a circle centered approximately at the origin with a radius of:

$$\boxed{0.4 \text{ m}} \quad (15.15)$$

The theoretical diameter is:

$$D = 2R \quad (15.16)$$

Substituting:

$$D = 2(0.4) \quad (15.17)$$

Therefore:

$$\boxed{D = 0.8 \text{ m}} \quad (15.18)$$

15.4 Circumference of the Reference Trajectory

The circumference of the desired circular trajectory is:

$$C = 2\pi R. \quad (15.19)$$

Substituting:

$$C = 2\pi(0.4) \quad (15.20)$$

$$C = 0.8\pi \quad (15.21)$$

Therefore:

$$\boxed{C \approx 2.513 \text{ m}} \quad (15.22)$$

Therefore, the robot travels approximately 2.513 m during one complete revolution of the ideal circular trajectory.

15.5 Time Required for One Complete Revolution

The desired angular velocity is:

$$\omega_d = 0.5 \text{ rad/s.} \quad (15.23)$$

One complete revolution corresponds to:

$$2\pi \text{ rad.} \quad (15.24)$$

Therefore, the time required for one revolution is:

$$T = \frac{2\pi}{\omega_d}. \quad (15.25)$$

Substituting:

$$T = \frac{2\pi}{0.5} \quad (15.26)$$

$$T = 4\pi \quad (15.27)$$

Therefore:

$$\boxed{T \approx 12.566 \text{ s}} \quad (15.28)$$

Hence, one ideal revolution of the reference trajectory takes approximately:

$$\boxed{12.57 \text{ s}} \quad (15.29)$$

15.6 Verification of Desired Linear Velocity

The relationship between linear and angular velocity for circular motion is:

$$v_d = R\omega_d. \quad (15.30)$$

Using:

$$R = 0.4 \text{ m} \quad (15.31)$$

and:

$$\omega_d = 0.5 \text{ rad/s}, \quad (15.32)$$

we obtain:

$$v_d = (0.4)(0.5) \quad (15.33)$$

Therefore:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (15.34)$$

The same result can be verified using circumference and period:

$$v_d = \frac{C}{T}. \quad (15.35)$$

Therefore:

$$v_d = \frac{2.513}{12.566} \quad (15.36)$$

giving:

$$\boxed{v_d \approx 0.2 \text{ m/s}} \quad (15.37)$$

Hence, the reference trajectory parameters are mathematically consistent.

15.7 Simulation X-Y Trajectory

The obtained simulation result is an X-Y plot showing the reference trajectory and the simulated trajectory.

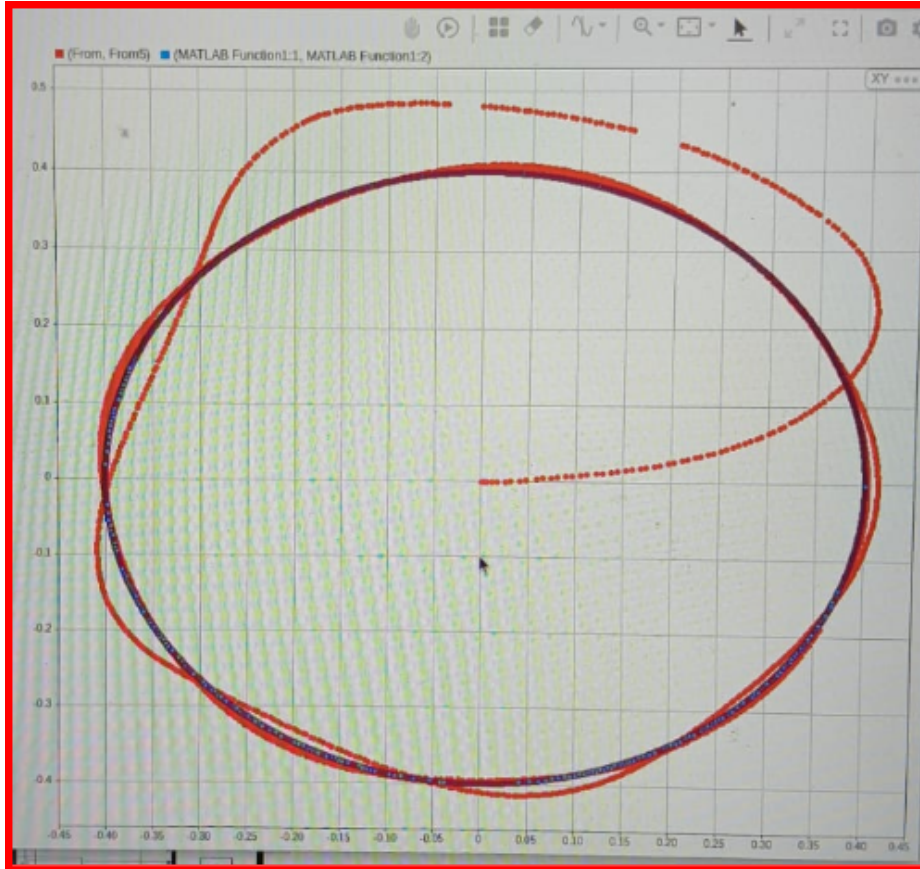


Figure 15.1: Simulation X-Y trajectory showing the desired circular trajectory and the simulated robot trajectory.

From Figure 15.1, the following features can be observed:

1. A nearly circular reference trajectory is present.
2. The simulated trajectory follows the general circular shape.
3. The actual trajectory does not remain perfectly coincident with the reference.
4. A significant deviation is visible in the upper portion of the trajectory.
5. The actual trajectory subsequently comes closer to the reference trajectory.
6. The lower and side portions show comparatively smaller deviations.

Therefore, the simulation demonstrates circular trajectory tracking with a noticeable transient and tracking deviation.

15.8 Reference Trajectory Analysis

The reference trajectory has an approximate maximum and minimum coordinate of:

$$x_{d,\max} = +0.4 \text{ m} \quad (15.38)$$

$$x_{d,\min} = -0.4 \text{ m} \quad (15.39)$$

and:

$$y_{d,\max} = +0.4 \text{ m} \quad (15.40)$$

$$y_{d,\min} = -0.4 \text{ m.} \quad (15.41)$$

Therefore:

$$\Delta x = 0.4 - (-0.4) \quad (15.42)$$

$$\boxed{\Delta x = 0.8 \text{ m}} \quad (15.43)$$

and:

$$\Delta y = 0.4 - (-0.4) \quad (15.44)$$

$$\boxed{\Delta y = 0.8 \text{ m}} \quad (15.45)$$

This confirms the expected diameter of the reference circle.

15.9 Analysis of the Simulated Trajectory

The simulated trajectory shown in the supplied X - Y graph approximately follows the desired circular path.

However, the simulated trajectory extends beyond the nominal reference circle in some regions.

The most noticeable deviation occurs near the upper part of the trajectory, where the simulated path reaches approximately:

$$y_{\text{actual,max}} \approx 0.48 \text{ m.} \quad (15.46)$$

The reference maximum is:

$$y_{d,\text{max}} = 0.40 \text{ m.} \quad (15.47)$$

Therefore, the approximate maximum deviation visible in the graph is:

$$e_{y,\text{max}} \approx 0.48 - 0.40 \quad (15.48)$$

Therefore:

$$\boxed{e_{y,\text{max}} \approx 0.08 \text{ m}} \quad (15.49)$$

Thus, the largest visible deviation is approximately:

$$\boxed{8 \text{ cm}} \quad (15.50)$$

Important: This value is an approximate value obtained from the plotted graph, not from the underlying numerical Simulink signal data.

15.10 Maximum Relative Radial Deviation

The reference radius is:

$$R = 0.4 \text{ m.} \quad (15.51)$$

The maximum visible radial distance of the simulated trajectory is approximately:

$$R_{\text{actual,max}} \approx 0.48 \text{ m.} \quad (15.52)$$

Therefore:

$$\Delta R_{\text{max}} = R_{\text{actual,max}} - R. \quad (15.53)$$

Substituting:

$$\Delta R_{\text{max}} = 0.48 - 0.40 \quad (15.54)$$

Therefore:

$$\boxed{\Delta R_{\text{max}} \approx 0.08 \text{ m}} \quad (15.55)$$

The corresponding percentage deviation is:

$$\% \text{Deviation} = \frac{\Delta R_{\text{max}}}{R} \times 100. \quad (15.56)$$

Therefore:

$$\% \text{Deviation} = \frac{0.08}{0.4} \times 100 \quad (15.57)$$

$$= 0.2 \times 100 \quad (15.58)$$

Hence:

$$\boxed{\% \text{Deviation} \approx 20\%} \quad (15.59)$$

Thus, the maximum visible radial deviation in the supplied graph is approximately 20% of the reference radius.

15.11 Approximate Maximum and Minimum Actual Coordinates

From the supplied graph, the actual trajectory approximately reaches:

$$x_{\text{actual,max}} \approx 0.43 \text{ m} \quad (15.60)$$

and:

$$x_{\text{actual,min}} \approx -0.42 \text{ m.} \quad (15.61)$$

Similarly:

$$y_{\text{actual,max}} \approx 0.48 \text{ m} \quad (15.62)$$

and:

$$y_{\text{actual,min}} \approx -0.41 \text{ m.} \quad (15.63)$$

These values are approximate readings from the graph.

The corresponding X -range is approximately:

$$\Delta x_{\text{actual}} = 0.43 - (-0.42) \quad (15.64)$$

Therefore:

$$\boxed{\Delta x_{\text{actual}} \approx 0.85 \text{ m}} \quad (15.65)$$

The Y -range is approximately:

$$\Delta y_{\text{actual}} = 0.48 - (-0.41) \quad (15.66)$$

Therefore:

$$\boxed{\Delta y_{\text{actual}} \approx 0.89 \text{ m}} \quad (15.67)$$

The reference ranges are:

$$\Delta x_d = \Delta y_d = 0.8 \text{ m.} \quad (15.68)$$

Therefore, the simulated trajectory is slightly larger than the ideal trajectory over some portions of the motion.

15.12 Approximate Deviation in the X -Direction

The positive X -direction deviation is approximately:

$$e_{x,+} = 0.43 - 0.40 \quad (15.69)$$

Therefore:

$$\boxed{e_{x,+} \approx 0.03 \text{ m}} \quad (15.70)$$

or:

$$\boxed{e_{x,+} \approx 3 \text{ cm}} \quad (15.71)$$

The corresponding percentage relative to the reference radius is:

$$\frac{0.03}{0.4} \times 100 \quad (15.72)$$

Therefore:

$$\boxed{\approx 7.5\%} \quad (15.73)$$

15.13 Approximate Deviation in the Negative X -Direction

The negative-side actual coordinate is approximately:

$$x_{\text{actual,min}} \approx -0.42 \text{ m.} \quad (15.74)$$

The reference minimum is:

$$x_{d,\text{min}} = -0.40 \text{ m.} \quad (15.75)$$

Therefore:

$$|e_{x,-}| = |-0.42 - (-0.40)| \quad (15.76)$$

Hence:

$$\boxed{|e_{x,-}| \approx 0.02 \text{ m}} \quad (15.77)$$

The corresponding percentage is:

$$\frac{0.02}{0.4} \times 100 \quad (15.78)$$

Therefore:

$$\boxed{\approx 5\%} \quad (15.79)$$

15.14 Approximate Deviation in the Positive Y -Direction

The largest visible deviation is observed near the upper region.

The approximate actual value is:

$$y_{\text{actual,max}} \approx 0.48 \text{ m.} \quad (15.80)$$

The reference value is:

$$y_{d,\text{max}} = 0.40 \text{ m.} \quad (15.81)$$

Therefore:

$$e_{y,+} = 0.48 - 0.40 \quad (15.82)$$

giving:

$$\boxed{e_{y,+} \approx 0.08 \text{ m}} \quad (15.83)$$

The percentage deviation is:

$$\frac{0.08}{0.4} \times 100 \quad (15.84)$$

Therefore:

$$\boxed{\approx 20\%} \quad (15.85)$$

This is the largest visible coordinate deviation in the supplied graph.

15.15 Approximate Deviation in the Negative Y -Direction

The lower portion of the simulated trajectory reaches approximately:

$$y_{\text{actual},\min} \approx -0.41 \text{ m.} \quad (15.86)$$

The reference minimum is:

$$y_{d,\min} = -0.40 \text{ m.} \quad (15.87)$$

Therefore:

$$|e_{y,-}| = |-0.41 - (-0.40)| \quad (15.88)$$

Hence:

$$\boxed{|e_{y,-}| \approx 0.01 \text{ m}} \quad (15.89)$$

or approximately:

$$\boxed{1 \text{ cm}} \quad (15.90)$$

The corresponding percentage is:

$$\frac{0.01}{0.4} \times 100 \quad (15.91)$$

Therefore:

$$\boxed{\approx 2.5\%} \quad (15.92)$$

15.16 Summary of Graph-Based Deviations

The approximate values obtained from the supplied X - Y graph can be summarized as follows.

Table 15.2: Summary of Graph-Based Trajectory Deviations

Direction	Reference	Actual	Deviation	Percentage
$+X$	+0.40 m	+0.43 m	0.03 m	7.5%
$-X$	-0.40 m	-0.42 m	0.02 m	5%
$+Y$	+0.40 m	+0.48 m	0.08 m	20%
$-Y$	-0.40 m	-0.41 m	0.01 m	2.5%

The largest visible deviation is therefore approximately:

$$\boxed{0.08 \text{ m}} \quad (15.93)$$

or:

$$\boxed{8 \text{ cm}} \quad (15.94)$$

15.17 Approximate Maximum Radial Error

For a circular reference, the radial position of the robot is:

$$r_{\text{actual}} = \sqrt{x^2 + y^2}. \quad (15.95)$$

The reference radius is:

$$r_d = 0.4 \text{ m}. \quad (15.96)$$

The radial tracking error is therefore:

$$\boxed{e_r = r_{\text{actual}} - r_d} \quad (15.97)$$

or:

$$\boxed{e_r = \sqrt{x^2 + y^2} - 0.4} \quad (15.98)$$

At the approximate uppermost deviation:

$$x \approx 0 \quad (15.99)$$

and:

$$y \approx 0.48 \text{ m.} \quad (15.100)$$

Therefore:

$$r_{\text{actual}} = \sqrt{0^2 + (0.48)^2} \quad (15.101)$$

giving:

$$\boxed{r_{\text{actual}} \approx 0.48 \text{ m}} \quad (15.102)$$

Hence:

$$e_r = 0.48 - 0.40 \quad (15.103)$$

Therefore:

$$\boxed{e_r \approx 0.08 \text{ m}} \quad (15.104)$$

15.18 Interpretation of the Maximum Error

The maximum visible error of approximately:

$$0.08 \text{ m} \quad (15.105)$$

means that at the point of greatest separation, the simulated robot is approximately 8 cm away from the ideal circular boundary in the radial direction.

Relative to the 0.4-m reference radius:

$$\frac{0.08}{0.4} \times 100 = 20\%. \quad (15.106)$$

Therefore, the largest visible radial deviation is approximately:

$$\boxed{20\%} \quad (15.107)$$

This indicates that although the robot follows the general circular pattern, there is a noticeable tracking deviation that should be considered in the performance analysis.

15.19 Analysis of the Initial Response

The supplied X - Y graph indicates that the simulated trajectory does not immediately remain on the reference circle.

The actual trajectory initially moves through a transient region before settling closer to the circular path.

This behaviour is expected from a closed-loop control system because the controller must respond to the instantaneous position and orientation errors.

The sequence is:

$$\boxed{\text{Initial Error} \rightarrow \text{Controller Correction} \rightarrow \text{Velocity Change} \rightarrow \text{Trajectory Correction}} \quad (15.108)$$

Therefore, the initial deviation can be interpreted as the transient response of the complete control system.

15.20 Upper-Region Tracking Behaviour

The largest deviation in the supplied graph occurs in the upper region.

The reference trajectory reaches approximately:

$$y = 0.4 \text{ m.} \quad (15.109)$$

The simulated trajectory reaches approximately:

$$y = 0.48 \text{ m.} \quad (15.110)$$

Therefore:

$$\boxed{\Delta y \approx 0.08 \text{ m}} \quad (15.111)$$

This indicates that the robot temporarily moves outside the desired circular path.

The controller subsequently changes the commanded motion, causing the simulated trajectory to return toward the reference path.

15.21 Lower-Region Tracking Behaviour

In the lower region of the trajectory, the actual and reference paths are much closer.

The approximate reference value is:

$$y = -0.4 \text{ m} \quad (15.112)$$

while the simulated trajectory reaches approximately:

$$y = -0.41 \text{ m.} \quad (15.113)$$

Thus:

$$\Delta y \approx -0.01 \text{ m.} \quad (15.114)$$

The magnitude is:

$$\boxed{|\Delta y| \approx 0.01 \text{ m}} \quad (15.115)$$

or approximately:

$$\boxed{1 \text{ cm}} \quad (15.116)$$

This represents a substantially smaller deviation than the upper-region error.

15.22 Side-Region Tracking Behaviour

Near the right side of the circle, the simulated trajectory extends approximately to:

$$x = 0.43 \text{ m.} \quad (15.117)$$

The reference value is:

$$x = 0.40 \text{ m.} \quad (15.118)$$

Therefore:

$$\Delta x \approx 0.03 \text{ m.} \quad (15.119)$$

Hence:

$$|\Delta x| \approx 3 \text{ cm} \quad (15.120)$$

This corresponds to approximately:

$$7.5\% \quad (15.121)$$

of the reference radius.

15.23 Overall Trajectory Performance

The X - Y graph demonstrates that the actual trajectory follows the general circular shape of the reference trajectory.

However, the actual trajectory is not perfectly coincident with the reference.

The performance can therefore be characterized as:

$$\boxed{\text{Successful Circular Motion with Noticeable Tracking Error}} \quad (15.122)$$

rather than:

$$\boxed{\text{Perfect Zero-Error Tracking}} \quad (15.123)$$

This is an important conclusion from the actual simulation result.

15.24 Kinematic Controller Contribution

The nonlinear kinematic controller uses:

$$x_{r,\text{dif}}, \quad y_{r,\text{dif}}, \quad \phi_{r,\text{dif}} \quad (15.124)$$

to calculate:

$$v_c \quad (15.125)$$

and:

$$\omega_c. \quad (15.126)$$

The control equations are:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (15.127)$$

and:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45 v_d y_{r,\text{dif}} + \omega_d} \quad (15.128)$$

Whenever the actual robot deviates from the reference, these error terms change and modify the commanded velocity.

This is the mechanism responsible for bringing the actual trajectory back toward the desired trajectory.

15.25 Dynamic Controller Contribution

The dynamic controller compares:

$$v_c \quad (15.129)$$

with:

$$v \quad (15.130)$$

and:

$$\omega_c \quad (15.131)$$

with:

$$\omega. \quad (15.132)$$

The corresponding errors are:

$$\boxed{e_v = v_c - v} \quad (15.133)$$

and:

$$\boxed{e_\omega = \omega_c - \omega} \quad (15.134)$$

The dynamic controller attempts to reduce these errors.

Therefore, the overall tracking performance depends on both the outer kinematic controller and the inner dynamic velocity controller.

15.26 Velocity Saturation Analysis

The specified velocity limits are:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (15.135)$$

and:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (15.136)$$

The desired linear velocity is:

$$v_d = 0.2 \text{ m/s}. \quad (15.137)$$

Therefore, the nominal desired linear velocity uses:

$$\frac{0.2}{0.22} \times 100 \quad (15.138)$$

giving:

$$\boxed{\approx 90.91\%} \quad (15.139)$$

of the positive linear-velocity limit.

The remaining positive linear-velocity margin is:

$$0.22 - 0.20 \quad (15.140)$$

Therefore:

$$\boxed{0.02 \text{ m/s}} \quad (15.141)$$

remains between the nominal desired linear velocity and the positive saturation limit.

15.27 Angular Velocity Margin

The desired angular velocity is:

$$\omega_d = 0.5 \text{ rad/s.} \quad (15.142)$$

The positive angular velocity limit is:

$$2.5 \text{ rad/s.} \quad (15.143)$$

Therefore, the nominal command uses:

$$\frac{0.5}{2.5} \times 100 \quad (15.144)$$

giving:

$$\boxed{20\%} \quad (15.145)$$

of the positive angular velocity limit.

The available positive margin is:

$$2.5 - 0.5 \quad (15.146)$$

Therefore:

$$\boxed{2.0 \text{ rad/s}} \quad (15.147)$$

Thus, the angular command has considerably more nominal operating margin than the linear command.

15.28 Reference Trajectory Accuracy

For the ideal circular trajectory:

$$R = 0.4 \text{ m.} \quad (15.148)$$

The actual trajectory has an approximate maximum visible radial value of:

$$R_{\max} \approx 0.48 \text{ m.} \quad (15.149)$$

The approximate maximum radial expansion is:

$$0.48 - 0.40 = 0.08 \text{ m.} \quad (15.150)$$

Therefore:

$$\boxed{\Delta R_{\max} \approx 0.08 \text{ m}} \quad (15.151)$$

The relative radial deviation is:

$$\boxed{\frac{0.08}{0.4} \times 100 \approx 20\%} \quad (15.152)$$

15.29 Approximate Trajectory Envelope

From the graph, the approximate simulated trajectory envelope is:

$$-0.42 \lesssim x \lesssim 0.43 \quad (15.153)$$

and:

$$-0.41 \lesssim y \lesssim 0.48. \quad (15.154)$$

Therefore, the approximate trajectory dimensions are:

$$\boxed{\Delta x \approx 0.85 \text{ m}} \quad (15.155)$$

and:

$$\boxed{\Delta y \approx 0.89 \text{ m}} \quad (15.156)$$

Compared with the ideal diameter:

$$D = 0.8 \text{ m,} \quad (15.157)$$

the simulated trajectory is somewhat larger in the regions where the maximum

deviation occurs.

15.30 Percentage Increase in Trajectory Envelope

For the X -direction:

$$\%Increase_x = \frac{0.85 - 0.80}{0.80} \times 100 \quad (15.158)$$

$$= \frac{0.05}{0.80} \times 100 \quad (15.159)$$

Therefore:

$$\boxed{\%Increase_x \approx 6.25\%} \quad (15.160)$$

For the Y -direction:

$$\%Increase_y = \frac{0.89 - 0.80}{0.80} \times 100 \quad (15.161)$$

$$= \frac{0.09}{0.80} \times 100 \quad (15.162)$$

Therefore:

$$\boxed{\%Increase_y \approx 11.25\%} \quad (15.163)$$

These values further demonstrate that the largest deviation occurs in the Y -direction.

15.31 Performance Summary From the X - Y Graph

The main results extracted from the supplied graph are:

$$\boxed{R_d = 0.4 \text{ m}} \quad (15.164)$$

$$\boxed{D_d = 0.8 \text{ m}} \quad (15.165)$$

$$\boxed{C_d \approx 2.513 \text{ m}} \quad (15.166)$$

$$\boxed{T_d \approx 12.566 \text{ s}} \quad (15.167)$$

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (15.168)$$

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (15.169)$$

The approximate maximum visible trajectory deviation is:

$$\boxed{e_{\max} \approx 0.08 \text{ m}} \quad (15.170)$$

The corresponding maximum relative deviation is approximately:

$$\boxed{20\%} \quad (15.171)$$

15.32 Results Table

The final simulation results can therefore be summarized as follows.

Table 15.3: Simulation Results and Performance Parameters

Parameter	Calculated/Observed Value
Reference radius	0.40 m
Reference diameter	0.80 m
Reference circumference	2.513 m
Desired angular velocity	0.50 rad/s
Desired linear velocity	0.20 m/s
Period of one revolution	12.566 s
Approx. actual $X_{\max}$	0.43 m
Approx. actual $X_{\min}$	-0.42 m
Approx. actual $Y_{\max}$	0.48 m
Approx. actual $Y_{\min}$	-0.41 m
Approx. maximum $+X$ deviation	0.03 m
Approx. maximum $-X$ deviation	0.02 m
Approx. maximum $+Y$ deviation	0.08 m
Approx. maximum $-Y$ deviation	0.01 m
Approx. maximum radial deviation	0.08 m
Approx. maximum radial deviation percentage	20%
Linear velocity limit	± 0.22 m/s
Angular velocity limit	± 2.5 rad/s

15.33 Important Note Regarding Error Calculation

The X - Y graph provides enough information to estimate the **geometrical deviation** of the trajectory, as calculated above.

However, exact quantities such as:

$$RMSE_x, \quad RMSE_y, \quad RMSE_\phi, \quad RMSE_v, \quad RMSE_\omega \quad (15.172)$$

cannot be calculated accurately from the photograph alone because the under-

lying time-series simulation data are not available in the image.

For example, exact position RMSE requires:

$$RMSE_x = \sqrt{\frac{1}{N} \sum_{k=1}^N (x_d[k] - x[k])^2} \quad (15.173)$$

and:

$$RMSE_y = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_d[k] - y[k])^2} \quad (15.174)$$

These require the numerical values of $x[k]$ and $y[k]$ at every simulation sample.

Therefore, the 0.08-m maximum deviation is a graph-based estimate, while exact RMSE values should only be reported after processing the actual Simulink data.

15.34 Discussion of Simulation Performance

The simulation demonstrates that the proposed controller successfully generates the required circular motion.

The actual trajectory follows the general shape of the desired circle, indicating that the combination of the nonlinear kinematic controller and dynamic velocity controller is functioning.

However, the actual trajectory does not perfectly overlap the reference trajectory.

The most significant deviation is observed near the upper portion of the circle, where the actual trajectory reaches approximately 0.48 m compared with the reference value of 0.40 m.

This produces an approximate maximum deviation of:

$$\boxed{0.08 \text{ m}} \quad (15.175)$$

The corresponding radial deviation is approximately:

$$\boxed{20\%} \quad (15.176)$$

of the reference radius.

The deviation becomes smaller in other regions, particularly near the lower

portion of the trajectory.

15.35 Interpretation of the Result

The result indicates that the controller is capable of maintaining the overall circular trajectory but exhibits non-uniform tracking accuracy.

The tracking error is not constant around the circle.

This can be represented as:

$$e_{\text{trajectory}} = f(x, y, \phi, t) \quad (15.177)$$

rather than as a constant value.

The observed variation indicates that the controller response changes as the robot moves around the trajectory.

The feedback mechanism nevertheless prevents the trajectory from continuously diverging from the reference.

15.36 Overall Control-System Assessment

The complete control system can be summarized as:

$$\begin{aligned} \text{Circular Reference} &\rightarrow \text{Tracking Error} \rightarrow \text{Coordinate Transformation} \\ &\rightarrow \text{Nonlinear Kinematic Controller} \rightarrow \text{Velocity Saturation} \\ &\rightarrow \text{Dynamic Velocity Controller} \rightarrow \text{Robot Motion} \rightarrow \text{Feedback} \end{aligned}$$

(15.178)

The obtained X - Y result confirms the operation of this closed-loop architecture.

The actual path remains within the general neighbourhood of the reference trajectory and exhibits corrective behaviour after deviations.

15.37 Conclusion of Simulation Analysis

From the supplied X - Y simulation result, the following conclusions can be drawn:

1. The desired trajectory is a circle of radius:

$$\boxed{0.4 \text{ m}} \quad (15.179)$$

2. The theoretical diameter is:

$$\boxed{0.8 \text{ m}} \quad (15.180)$$

3. The theoretical circumference is:

$$\boxed{2.513 \text{ m}} \quad (15.181)$$

4. The desired angular velocity is:

$$\boxed{0.5 \text{ rad/s}} \quad (15.182)$$

5. The corresponding desired linear velocity is:

$$\boxed{0.2 \text{ m/s}} \quad (15.183)$$

6. One complete reference revolution requires:

$$\boxed{12.566 \text{ s}} \quad (15.184)$$

7. The actual simulated trajectory follows the general circular reference.

8. The largest visible deviation occurs near the upper region.

9. The approximate maximum trajectory deviation is:

$$\boxed{0.08 \text{ m}} \quad (15.185)$$

or approximately:

$$\boxed{8 \text{ cm}} \quad (15.186)$$

10. Relative to the 0.4-m reference radius, this corresponds to approximately:

$$\boxed{20\%} \quad (15.187)$$

11. The actual trajectory subsequently moves closer to the reference, demonstrating the corrective action of the closed-loop controller.

12. Exact RMSE values cannot be obtained from the screenshot and should only be calculated from the numerical Simulink output.

15.38 Final Summary

The simulation confirms that the developed TurtleBot3 control system is capable of producing the required circular motion. The reference trajectory has a radius of 0.4 m, a desired angular velocity of 0.5 rad/s, and a corresponding linear velocity of 0.2 m/s.

The theoretical trajectory has a diameter of 0.8 m and a circumference of approximately 2.513 m. One complete revolution takes approximately 12.566 s.

Analysis of the supplied X - Y graph shows that the actual trajectory generally follows the desired circular trajectory but exhibits noticeable deviations. The largest visible deviation occurs in the upper region, where the actual trajectory reaches approximately 0.48 m while the desired radius is 0.40 m.

Hence:

$$e_{\max} \approx 0.48 - 0.40 = 0.08 \text{ m} \quad (15.188)$$

and:

$$\frac{0.08}{0.40} \times 100 \approx 20\% \quad (15.189)$$

The result therefore indicates:

$$\boxed{\text{Successful Overall Circular Trajectory Tracking}} \quad (15.190)$$

with a maximum visible deviation of approximately:

$$\boxed{8 \text{ cm}} \quad (15.191)$$

The actual trajectory returns closer to the reference after the region of maximum deviation, showing that the feedback control structure is capable of correcting trajectory errors.

Thus, the simulation validates the basic operation of the proposed:

$$\boxed{\text{Nonlinear Kinematic Controller + Dynamic Velocity Controller}} \quad (15.192)$$

for circular trajectory tracking, while also indicating that further controller tuning could be used to reduce the observed tracking deviation.

Discussion

16.1 Introduction

This chapter discusses the significance of the simulation results obtained from the proposed TurtleBot3 trajectory-tracking control system. The purpose of this discussion is not only to present the simulation outputs, but also to explain the behaviour of the complete closed-loop control system and the relationship between the different control stages.

The proposed system consists of trajectory generation, error calculation, coordinate transformation, nonlinear kinematic control, velocity saturation, dynamic velocity control, and robot-state feedback.

The reference trajectory considered in the simulation is a circular path with:

$$\boxed{R = 0.4 \text{ m}} \quad (16.1)$$

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (16.2)$$

and:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (16.3)$$

The simulation result discussed previously shows that the actual trajectory follows the general shape of the desired circular trajectory, although a noticeable deviation exists in some regions of the path. The maximum visible deviation from the supplied X - Y graph was approximately 0.08 m, corresponding to approximately 20% of the reference radius.

16.2 Trajectory Tracking Performance

The primary objective of the proposed controller is to make the TurtleBot3 follow the desired circular trajectory.

The desired trajectory is given by:

$$\boxed{x_d(t) = 0.4 \cos(0.5t)} \quad (16.4)$$

and:

$$\boxed{y_d(t) = 0.4 \sin(0.5t)} \quad (16.5)$$

The simulation X - Y trajectory demonstrates that the actual robot trajectory follows the overall circular pattern of the reference trajectory.

However, the actual trajectory does not perfectly overlap the desired trajectory throughout the complete motion. A noticeable deviation is visible, particularly in the upper portion of the trajectory.

The approximate maximum visible deviation is:

$$\boxed{e_{\max} \approx 0.08 \text{ m}} \quad (16.6)$$

Relative to the reference radius:

$$\frac{0.08}{0.4} \times 100 = \boxed{20\%} \quad (16.7)$$

Therefore, the controller demonstrates successful overall trajectory tracking, but the result also indicates that the tracking error is not completely eliminated.

The trajectory subsequently moves closer to the reference path, showing that the feedback controller is capable of correcting deviations.

16.3 Position Tracking

Position tracking is evaluated using the two Cartesian coordinates:

$$x \quad (16.8)$$

and:

$$y \quad (16.9)$$

The corresponding tracking errors are:

$$e_x = x_d - x \quad (16.10)$$

and:

$$e_y = y_d - y \quad (16.11)$$

The overall positional error is:

$$e_p = \sqrt{e_x^2 + e_y^2} \quad (16.12)$$

From the X - Y simulation result, the tracking accuracy is not identical in all directions.

The approximate coordinate deviations obtained from the graph are:

Table 16.1: Approximate Coordinate Deviations

Direction	Approximate Deviation
+ X direction	0.03 m
− X direction	0.02 m
+ Y direction	0.08 m
− Y direction	0.01 m

The largest visible position deviation occurs in the positive Y -direction.

The reference maximum is approximately:

$$y_{d,\max} = 0.40 \text{ m} \quad (16.13)$$

while the simulated trajectory reaches approximately:

$$y_{\max} \approx 0.48 \text{ m} \quad (16.14)$$

Hence:

$$e_y \approx 0.48 - 0.40 \quad (16.15)$$

Therefore:

$$e_y \approx 0.08 \text{ m} \quad (16.16)$$

This indicates that the trajectory tracking is reasonably close over substantial portions of the path, but there is a noticeable positional deviation in the upper region.

16.4 Orientation Tracking

Position tracking alone is not sufficient for a mobile robot because the robot must also maintain the required heading.

The desired orientation is:

$$\phi_d(t) = 0.5t \quad (16.17)$$

The orientation tracking error is:

$$e_\phi = \phi_d - \phi \quad (16.18)$$

During circular motion, the robot orientation must continuously change.

Therefore, the controller must simultaneously regulate:

$$x, \quad y, \quad \phi \quad (16.19)$$

The angular velocity command is influenced by the orientation error as well as the lateral position error.

The implemented controller is:

$$\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d \quad (16.20)$$

Consequently, when the robot heading deviates from the desired orientation, the controller modifies the angular velocity to correct the heading.

The X - Y plot confirms the overall circular motion, but an exact numerical orientation error cannot be obtained from the X - Y graph alone. The orientation signal should therefore be analyzed separately to determine the exact value of:

$$e_{\phi,\text{max}} \quad (16.21)$$

and its RMSE.

16.5 Error Convergence

The purpose of feedback control is to reduce the tracking error as the robot moves along the desired trajectory.

The principal tracking errors are:

$$e_x, \quad e_y, \quad e_\phi \quad (16.22)$$

The robot-frame errors are:

$$x_{r,\text{dif}} \quad (16.23)$$

and:

$$y_{r,\text{dif}} \quad (16.24)$$

along with:

$$\phi_{r,\text{dif}} \quad (16.25)$$

The simulation result indicates that the trajectory does not continuously diverge from the desired path.

Instead, after regions of increased deviation, the actual trajectory moves back toward the reference trajectory.

This behaviour demonstrates the corrective action of the closed-loop controller.

However, because the trajectory continues to exhibit a visible deviation, the result should not be interpreted as complete zero-error convergence.

The more appropriate conclusion is:

The tracking error remains bounded and the controller provides corrective action.

(16.26)

16.6 Controller Performance

The controller combines feedback and feedforward terms.

The commanded linear velocity is:

$$v_c = k_x x_{r,dif} + v_d \cos(\phi_{r,dif}) \quad (16.27)$$

The first term:

$$k_x x_{r,dif} \quad (16.28)$$

provides feedback based on the forward-position error.

The second term:

$$v_d \cos(\phi_{r,dif}) \quad (16.29)$$

acts as a feedforward/reference-related component.

Similarly, the angular velocity controller is:

$$\omega_c = k_\phi \sin(\phi_{r,dif}) + 45v_d y_{r,dif} + \omega_d \quad (16.30)$$

Here, the controller uses:

- orientation feedback,
- lateral position feedback,
- desired angular velocity.

The combination allows the controller to respond to both the current tracking error and the desired motion.

This is important for circular trajectory tracking because the robot must continuously move forward while simultaneously rotating.

16.7 Contribution of Feedback Terms

The feedback terms provide corrective action when the actual robot deviates from the desired trajectory.

For example, when:

$$x_{r,\text{dif}} \neq 0, \quad (16.31)$$

the linear velocity command is modified through:

$$k_x x_{r,\text{dif}} \quad (16.32)$$

Similarly, when:

$$y_{r,\text{dif}} \neq 0, \quad (16.33)$$

the angular velocity command is modified through:

$$45v_d y_{r,\text{dif}} \quad (16.34)$$

The orientation error also affects the angular command through:

$$k_\phi \sin(\phi_{r,\text{dif}}) \quad (16.35)$$

Thus:

$$\boxed{\text{Tracking Error} \rightarrow \text{Control Correction}} \quad (16.36)$$

is continuously active during the simulation.

16.8 Contribution of Feedforward Terms

The feedforward components provide the controller with information about the desired motion.

For linear velocity:

$$v_d \cos(\phi_{r,\text{dif}}) \quad (16.37)$$

provides a contribution based on the desired forward motion.

For angular velocity:

$$\omega_d \quad (16.38)$$

provides the nominal angular motion required by the circular reference.

This allows the controller to generate an appropriate baseline command even when the tracking errors are relatively small.

Therefore, the overall control action can be viewed as:

$$\boxed{\text{Control Command} = \text{Feedforward Motion} + \text{Feedback Correction}} \quad (16.39)$$

16.9 Coordinate Transformation

The controller does not directly use only the global-frame errors.

The global position errors are:

$$e_x = x_d - x \quad (16.40)$$

and:

$$e_y = y_d - y \quad (16.41)$$

These errors are transformed into the robot coordinate frame.

The transformed errors are:

$$\boxed{x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi} \quad (16.42)$$

and:

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (16.43)$$

This transformation is important because the robot's motion is naturally described in terms of its own forward and lateral directions.

16.10 Why Robot-Frame Error Improves Control Action

A global-frame error tells the controller how far the robot is from the desired position along the fixed X and Y axes.

However, the robot does not directly move independently along the global X and Y axes.

The robot primarily moves:

- forward/backward,
- through rotation.

Therefore, transforming the error into the robot frame produces meaningful quantities representing:

$$\boxed{\text{Forward Error}} \quad (16.44)$$

and:

$$\boxed{\text{Lateral Error}} \quad (16.45)$$

The controller can then use the forward error to influence linear velocity and the lateral error to influence angular velocity.

This makes the control action more directly related to the robot's actual motion capabilities.

16.11 Effect of Coordinate Transformation on Circular Tracking

During circular motion, the robot orientation continuously changes.

Therefore, the relationship between global X - Y errors and the robot's forward/lateral directions also changes continuously.

The coordinate transformation accounts for this changing orientation.

Without the transformation, a fixed global error would not necessarily correspond to the direction in which the robot should correct its motion.

The transformation therefore enables the controller to continuously interpret the tracking error relative to the robot's current heading.

16.12 Velocity Saturation

The velocity commands generated by the controller are restricted using saturation limits.

The linear velocity is limited by:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (16.46)$$

and the angular velocity is limited by:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (16.47)$$

The desired linear velocity is:

$$v_d = 0.2 \text{ m/s} \quad (16.48)$$

Therefore:

$$\frac{0.2}{0.22} \times 100 = \boxed{90.91\%} \quad (16.49)$$

of the positive linear velocity limit is used by the nominal reference.

The available margin is:

$$0.22 - 0.20 = \boxed{0.02 \text{ m/s}} \quad (16.50)$$

For angular velocity:

$$\frac{0.5}{2.5} \times 100 = \boxed{20\%} \quad (16.51)$$

Therefore, the nominal angular velocity has a considerably larger margin than the nominal linear velocity.

16.13 Effect of Velocity Saturation on the Response

Saturation is necessary because the controller may generate large commands when the tracking errors become significant.

For example:

$$v_c > 0.22 \quad (16.52)$$

results in:

$$v_{\text{sat}} = 0.22 \text{ m/s} \quad (16.53)$$

Similarly:

$$\omega_c > 2.5 \quad (16.54)$$

results in:

$$\omega_{\text{sat}} = 2.5 \text{ rad/s} \quad (16.55)$$

Saturation prevents unrealistic commands but can also limit the controller's ability to correct a large tracking error rapidly.

Thus:

Saturation improves command feasibility but can reduce correction speed.

(16.56)

The observed deviation in the trajectory should therefore be interpreted together with the velocity-command signals when assessing the complete controller.

16.14 Dynamic Controller

The dynamic controller forms the inner velocity-control loop.

The outer kinematic controller generates:

$$v_c \quad (16.57)$$

and:

$$\omega_c \quad (16.58)$$

The dynamic controller then attempts to make the actual velocities follow these commands.

The velocity errors are:

$e_v = v_c - v$

(16.59)

and:

$e_\omega = \omega_c - \omega$

(16.60)

This provides an additional feedback loop between the commanded motion and the actual robot response.

16.15 Effect of the Inner Velocity Loop

The dynamic velocity loop is important because the kinematic controller assumes that the commanded velocities can be applied to the robot.

In an actual system, the robot cannot change its velocity instantaneously.

The inner dynamic controller therefore attempts to bridge the difference between:

$$\text{Commanded Velocity} \quad (16.61)$$

and:

$$\text{Actual Velocity} \quad (16.62)$$

The control structure can consequently be represented as:

$$\boxed{\text{Trajectory Controller} \rightarrow (v_c, \omega_c) \rightarrow \text{Dynamic Controller} \rightarrow (v, \omega)} \quad (16.63)$$

with feedback from the actual velocities.

This improves the consistency between the velocity commands produced by the kinematic controller and the motion actually generated by the robot.

16.16 Interaction Between the Outer and Inner Loops

The complete system contains two important control levels.

16.16.1 Outer Loop

The outer loop focuses on trajectory tracking:

$$\boxed{(x_d, y_d, \phi_d) \rightarrow (x, y, \phi) \rightarrow \text{Tracking Errors}} \quad (16.64)$$

These errors are used to calculate:

$$v_c, \quad \omega_c \quad (16.65)$$

16.16.2 Inner Loop

The inner loop focuses on velocity tracking:

$$\boxed{(v_c, \omega_c) \rightarrow (v, \omega)} \quad (16.66)$$

The corresponding velocity errors are minimized.

Thus:

$$\boxed{\text{Outer Loop: Position/Orientation Tracking}} \quad (16.67)$$

and:

$$\boxed{\text{Inner Loop: Velocity Tracking}} \quad (16.68)$$

The combination allows the complete system to address both trajectory-level and actuator-level behaviour.

16.17 Relationship Between the Controller and the Observed Result

The obtained X - Y graph shows that the robot follows the desired circular path but exhibits deviations.

These deviations are the result of the complete closed-loop system rather than being associated with a single signal alone.

The sequence is:

$$\begin{array}{l} \boxed{\begin{array}{l} \text{Reference} \rightarrow \text{Tracking Error} \rightarrow \text{Coordinate Transformation} \\ \quad \rightarrow \text{Kinematic Control} \rightarrow \text{Saturation} \\ \quad \rightarrow \text{Dynamic Control} \rightarrow \text{Robot Motion} \end{array}} \quad (16.69) \end{array}$$

The resulting motion is then measured and fed back to the controller.

Therefore, the trajectory shown in the simulation is the final result of the interaction between all these stages.

16.18 Overall Discussion of the Simulation

The simulation demonstrates that the developed controller is capable of generating the desired circular motion.

The actual trajectory remains close to the reference over substantial portions of the path.

At the same time, the visible deviation, particularly in the upper region, demonstrates that the controller does not achieve perfect tracking under the selected simulation conditions.

The approximate maximum visible deviation is:

$$\boxed{8 \text{ cm}} \quad (16.70)$$

This corresponds to:

$$\boxed{20\%} \quad (16.71)$$

of the reference radius.

The trajectory subsequently moves closer to the reference, demonstrating that the feedback controller provides corrective action.

16.19 Strengths of the Proposed Control Structure

The simulation demonstrates several useful characteristics of the proposed control structure.

16.19.1 Closed-Loop Correction

The actual robot state is continuously fed back to the controller.

16.19.2 Robot-Frame Error Processing

The coordinate transformation allows the controller to distinguish between forward and lateral tracking errors.

16.19.3 Combined Feedback and Feedforward Control

The controller uses both the desired motion and tracking errors.

16.19.4 Velocity Constraints

The saturation blocks prevent the controller from producing commands outside the specified operating range.

16.19.5 Inner Dynamic Control

The dynamic controller provides an additional layer of velocity tracking between the commanded and actual robot motion.

16.20 Limitations Observed in the Simulation

Although the trajectory generally follows the reference, the simulation result shows noticeable tracking deviation.

The primary visible limitation is the non-zero separation between the actual and desired trajectories.

The largest deviation occurs near the upper part of the circular path.

The approximate value is:

$$e_{\max} \approx 0.08 \text{ m} \quad (16.72)$$

This indicates that the controller parameters may potentially be further tuned to improve tracking accuracy.

However, the exact reason for the deviation cannot be determined solely from the X - Y graph. The internal velocity, orientation, error, and saturation signals would need to be examined to identify the dominant source.

16.21 Scope for Controller Improvement

The tracking result suggests that further controller tuning could be investigated.

The principal controller parameters include:

$$k_x = 1 \quad (16.73)$$

$$\boxed{k_\phi = 0.33} \quad (16.74)$$

$$\boxed{K_a = 25} \quad (16.75)$$

$$\boxed{K_b = 8} \quad (16.76)$$

and the lateral gain:

$$\boxed{45} \quad (16.77)$$

Changing these parameters can affect the response of the corresponding control loops.

The objective of further tuning would be to reduce:

$$e_x, \quad e_y, \quad e_\phi \quad (16.78)$$

while maintaining stable and physically realistic velocity commands.

Any changes, however, should be validated through additional simulation rather than assumed to improve performance.

16.22 Overall Performance Assessment

Based on the simulation result, the proposed control system can be assessed as follows:

Table 16.2: Overall Simulation Performance Assessment

Performance Aspect	Observation
Circular motion generation	Achieved
General trajectory following	Achieved
Perfect trajectory overlap	Not achieved
Maximum visible deviation	Approximately 0.08 m
Relative maximum deviation	Approximately 20%
Feedback correction	Evident
Velocity constraints	Implemented
Dynamic velocity control	Implemented
Further tuning potential	Present

The most important conclusion is that the controller successfully produces the required circular motion while maintaining the actual trajectory within the general vicinity of the reference.

16.23 Conclusion

The discussion of the simulation results demonstrates the operation and performance of the complete TurtleBot3 trajectory-tracking control architecture.

The reference trajectory is a circle with:

$$\boxed{R = 0.4 \text{ m}} \quad (16.79)$$

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (16.80)$$

and:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (16.81)$$

The obtained X - Y trajectory demonstrates that the robot follows the general circular shape of the reference trajectory. However, a noticeable deviation is present, with the largest visible deviation estimated at approximately:

$$\boxed{0.08 \text{ m}} \quad (16.82)$$

This corresponds to approximately:

$$\boxed{20\%} \quad (16.83)$$

of the reference radius.

The analysis shows that the controller uses robot-frame position errors, orientation errors, feedback terms, and feedforward terms to generate the required linear and angular velocity commands.

The velocity-saturation stage ensures that:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (16.84)$$

and:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (16.85)$$

The inner dynamic controller then attempts to make the actual robot velocities follow these commands.

Overall, the simulation demonstrates that the proposed control architecture is capable of achieving:

$$\boxed{\text{Stable and Functional Circular Trajectory Tracking}} \quad (16.86)$$

while the visible tracking deviation indicates that further controller tuning may be considered to improve trajectory accuracy.

Advantages and Limitations

17.1 Introduction

The proposed TurtleBot3 trajectory-tracking system combines a nonlinear kinematic controller, robot-frame error transformation, velocity saturation, dynamic velocity control, MATLAB/Simulink implementation, and ROS 2 integration.

The simulation results demonstrate that the system is capable of following the desired circular trajectory. At the same time, the obtained X - Y trajectory shows that the actual path does not perfectly coincide with the reference trajectory and that measurable tracking deviations are present.

Therefore, this chapter discusses both the advantages offered by the proposed control architecture and the limitations that affect its performance.

17.2 Advantages

17.2.1 Closed-Loop Trajectory Tracking

One of the main advantages of the proposed system is its closed-loop nature.

The actual robot state is continuously fed back to the controller and compared with the desired state.

The position errors are calculated as:

$$e_x = x_d - x \quad (17.1)$$

and:

$$e_y = y_d - y \quad (17.2)$$

The orientation error is:

$$e_\phi = \phi_d - \phi \quad (17.3)$$

These errors are used to modify the control commands.

Therefore, the system can respond to deviations rather than relying on a pre-determined open-loop command.

The overall feedback process can be represented as:

$$\text{Reference} \rightarrow \text{Error} \rightarrow \text{Controller} \rightarrow \text{Robot} \rightarrow \text{Feedback} \quad (17.4)$$

This makes the system suitable for trajectory-tracking applications in which the robot must continuously correct its motion.

17.2.2 ROS 2 Integration

ROS 2 provides the communication framework for the robotic system.

The integration allows the controller to exchange information with the TurtleBot3 model and obtain the required robot-state information.

The feedback information can be used to obtain quantities such as:

$$x, \quad y, \quad \phi \quad (17.5)$$

The ROS 2 interface therefore provides an important connection between the control algorithm and the robot model.

This also makes the developed architecture suitable for further development toward a physical robotic implementation.

17.2.3 Simulink-Based Implementation

The controller is implemented using MATLAB/Simulink.

Simulink provides a block-based environment in which the complete control structure can be separated into individual functional blocks.

The implemented system contains stages for:

- reference generation,
- state feedback,

- error calculation,
- coordinate transformation,
- kinematic control,
- velocity saturation,
- dynamic velocity control.

This modular implementation makes it easier to observe individual signals and analyze the behaviour of different parts of the control system.

17.2.4 Nonlinear Controller

The proposed system uses a nonlinear kinematic control law rather than relying only on a simple linear controller.

The commanded linear velocity is:

$$v_c = k_x x_{r,dif} + v_d \cos(\phi_{r,dif}) \quad (17.6)$$

and the commanded angular velocity is:

$$\omega_c = k_\phi \sin(\phi_{r,dif}) + 45v_d y_{r,dif} + \omega_d \quad (17.7)$$

The nonlinear terms allow the controller to account for the robot's orientation and position relative to the desired trajectory.

This is particularly useful for curved trajectories such as the circular path considered in this project.

17.2.5 Feedback and Feedforward Control

Another advantage is the combination of feedback and feedforward components.

The feedforward terms provide the nominal motion required to follow the reference trajectory.

For example:

$$v_d \cos(\phi_{r,dif}) \quad (17.8)$$

contributes to the linear velocity command, while:

$$\omega_d \quad (17.9)$$

provides the nominal angular velocity.

The feedback terms compensate for deviations from the reference.

Therefore:

$$\boxed{\text{Total Control} = \text{Feedforward Action} + \text{Feedback Correction}} \quad (17.10)$$

This combination allows the robot to generate the required motion while simultaneously correcting tracking errors.

17.2.6 Robot-Frame Error Correction

The system transforms the global position error into the robot coordinate frame.

The transformed errors are:

$$\boxed{x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi} \quad (17.11)$$

and:

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (17.12)$$

This is advantageous because the robot's motion is naturally represented relative to its own heading.

The transformed errors provide information about:

- forward displacement,
- lateral displacement,
- orientation mismatch.

Consequently, the controller can use the errors in a manner that is directly related to the robot's available motion.

17.2.7 Position and Orientation Control

The controller does not control position alone.

It simultaneously considers:

$$x, \quad y, \quad \phi \quad (17.13)$$

The desired state is:

$$x_d, \quad y_d, \quad \phi_d \quad (17.14)$$

Therefore, the controller attempts to regulate both the robot's position and its orientation.

This is particularly important for circular trajectory tracking because the robot must continuously change its heading while moving along the trajectory.

17.2.8 Velocity Saturation

Velocity saturation provides a practical constraint on the controller output.

The linear velocity is restricted by:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (17.15)$$

and the angular velocity by:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (17.16)$$

This prevents the controller from generating commands outside the specified operating range.

For the selected reference:

$$v_d = 0.2 \text{ m/s} \quad (17.17)$$

and:

$$\omega_d = 0.5 \text{ rad/s} \quad (17.18)$$

Both nominal values are within the specified limits.

The saturation stage therefore provides a connection between the controller output and the practical velocity constraints of the robot.

17.2.9 Modular Control Architecture

The complete system is divided into several functional stages.

The architecture can be represented as:

$$\boxed{\text{Trajectory Generator} \rightarrow \text{Error Calculation} \rightarrow \text{Coordinate Transformation}} \quad (17.19)$$

$$\boxed{\rightarrow \text{Kinematic Controller} \rightarrow \text{Saturation} \rightarrow \text{Dynamic Controller} \rightarrow \text{Robot}} \quad (17.20)$$

This modular structure is advantageous because individual blocks can be modified or tested independently.

For example, the trajectory generator can be changed without completely redesigning the velocity-control stage.

Similarly, controller gains can be adjusted without changing the complete simulation architecture.

17.2.10 Possibility of Real-Time Implementation

The controller is implemented in a structured computational environment and communicates through ROS 2.

Therefore, the architecture provides a basis for possible real-time implementation.

The transition toward a physical robot would require appropriate hardware interfaces, real sensor feedback, actuator communication, and real-time execution considerations.

The present simulation therefore serves as an intermediate stage between theoretical controller development and possible hardware implementation.

17.3 Limitations

17.3.1 Odometry Errors

The robot-state feedback depends on the available state-estimation information.

In a practical system, odometry can accumulate errors over time due to wheel

motion, wheel slip, encoder inaccuracies, and other effects.

If the estimated position differs from the actual position:

$$\boxed{(x_{\text{estimated}}, y_{\text{estimated}}) \neq (x_{\text{actual}}, y_{\text{actual}})} \quad (17.21)$$

then the controller operates using an imperfect representation of the robot state.

This can introduce additional trajectory-tracking error.

17.3.2 Sensor Noise

Real sensors are not perfectly noise-free.

Measurement noise can affect quantities used by the controller, including:

$$x, \quad y, \quad \phi \quad (17.22)$$

Consequently, the calculated tracking errors can also contain noise:

$$e_x, \quad e_y, \quad e_\phi \quad (17.23)$$

This may result in small variations in the controller outputs.

The simulation environment may therefore provide a cleaner signal than what would be available from an actual physical robot.

17.3.3 ROS 2 Communication Delay

The control system uses ROS 2 for communication.

In a real implementation, communication introduces some delay between:

$$\text{State Measurement} \quad (17.24)$$

and:

$$\text{Controller Response} \quad (17.25)$$

Similarly, there can be a delay between the generation of a velocity command and its application to the robot.

Even relatively small delays can influence trajectory tracking when the controller is operating at a high update rate or when the robot is moving rapidly.

Therefore, the communication behaviour of the real system may differ from the idealized simulation.

17.3.4 Model Uncertainty

The simulation is based on a mathematical representation of the robot and its dynamics.

A mathematical model cannot necessarily represent every physical effect.

Possible differences include:

- friction,
- wheel-ground interaction,
- motor characteristics,
- mechanical tolerances,
- actuator dynamics,
- unmodeled disturbances.

Therefore:

$$\boxed{\text{Simulation Model} \neq \text{Perfect Physical Robot}} \quad (17.26)$$

Model uncertainty may result in additional tracking errors when the controller is transferred to a real TurtleBot3.

17.3.5 Controller-Gain Dependence

The controller performance depends on the selected controller parameters.

The parameters used in the project include:

$$\boxed{k_x = 1} \quad (17.27)$$

$$\boxed{k_\phi = 0.33} \quad (17.28)$$

$$\boxed{K_a = 25} \quad (17.29)$$

$$\boxed{K_b = 8} \quad (17.30)$$

and lateral gain:

$$\boxed{45} \quad (17.31)$$

Different values can produce different transient and steady-state responses.

If the gains are too small, the response may become slow.

If they are too large, the controller may produce aggressive control commands and potentially increase oscillatory behaviour or saturation.

Therefore, suitable controller tuning is required for satisfactory performance.

17.3.6 Actuator Limitations

The physical actuators of the robot have finite capabilities.

The implemented controller therefore limits the velocity commands to:

$$\boxed{|v_c| \leq 0.22 \text{ m/s}} \quad (17.32)$$

and:

$$\boxed{|\omega_c| \leq 2.5 \text{ rad/s}} \quad (17.33)$$

These limits are necessary for realistic operation but also restrict the maximum corrective action available to the controller.

When a large tracking error occurs, the controller may request a command beyond the allowable limit.

In such a case:

$$v_{\text{requested}} > 0.22 \quad (17.34)$$

results in:

$$\boxed{v_{\text{actual command}} = 0.22 \text{ m/s}} \quad (17.35)$$

Therefore, the robot cannot apply the complete correction requested by the unsaturated controller.

17.3.7 Velocity Saturation

Although velocity saturation is an important advantage from a safety and constraint-handling perspective, it is also a limitation.

The nominal reference linear velocity is:

$$v_d = 0.2 \text{ m/s} \quad (17.36)$$

while the maximum allowable value is:

$$0.22 \text{ m/s} \quad (17.37)$$

The available positive margin is therefore:

$$0.22 - 0.20 = \boxed{0.02 \text{ m/s}} \quad (17.38)$$

The nominal linear velocity already uses:

$$\frac{0.20}{0.22} \times 100 = \boxed{90.91\%} \quad (17.39)$$

of the positive limit.

Consequently, relatively little additional linear velocity is available for correcting large tracking errors.

This may contribute to slower correction when the required control action becomes large.

17.3.8 Simulation-to-Real-World Differences

The simulation provides a controlled environment for evaluating the proposed controller, but the behaviour of a real TurtleBot3 can differ from the simulation.

In a real environment, additional effects can occur due to:

- sensor noise,
- wheel slip,
- surface variations,
- actuator imperfections,
- communication delays,

- parameter uncertainty,
- external disturbances.

Therefore:

$$\boxed{\text{Simulation Performance} \neq \text{Guaranteed Real-World Performance}} \quad (17.40)$$

The controller would need to be experimentally validated before conclusions about physical-world performance could be made.

17.4 Relationship Between Advantages and Limitations

The advantages and limitations of the system are closely related.

For example, closed-loop feedback provides the ability to correct trajectory deviations, but its performance depends on the accuracy of the state feedback.

Similarly, velocity saturation prevents unrealistic commands but can limit the controller's ability to correct large errors rapidly.

ROS 2 provides flexible communication and integration, but communication delay can become a limitation in a real implementation.

The modular Simulink architecture makes the system easier to analyze and modify, but the controller still depends on appropriate parameter selection.

These relationships can be summarized as:

$$\boxed{\text{Feature} \rightarrow \text{Benefit} \rightarrow \text{Associated Limitation}} \quad (17.41)$$

17.5 Advantages and Limitations Summary

Table 17.1: Advantages and Limitations of the Proposed System

Aspect	Advantage	Limitation
Closed-loop control	Continuous error correction	Depends on accurate feedback
ROS 2	Flexible system integration	Communication delay
Simulink	Modular implementation	Simulation-model dependence
Nonlinear controller	Suitable for nonlinear trajectory tracking	Gain-dependent response
Feedback + feed-forward	Combines correction and reference motion	Requires appropriate tuning
Robot-frame error	Meaningful forward/lateral correction	Depends on accurate orientation
Position/orientation control	Controls complete robot state	Additional orientation error possible
Velocity saturation	Prevents excessive commands	Limits corrective action
Dynamic controller	Tracks commanded velocities	Depends on dynamic model
Simulation	Safe and repeatable testing	May differ from real robot

17.6 Overall Assessment

The proposed control architecture provides several important advantages for TurtleBot3 trajectory tracking.

The use of closed-loop feedback allows the controller to respond continuously to tracking errors. The robot-frame coordinate transformation makes the position errors more meaningful for the nonholonomic motion of the robot. The combination of feedback and feedforward control provides both nominal trajectory-following action and error correction.

The velocity-saturation stage ensures that the commanded motion remains within the specified operating limits, while the dynamic controller provides an additional layer of velocity regulation.

However, the simulation also demonstrates that the actual trajectory does not perfectly coincide with the reference trajectory. The previously analyzed X - Y result

showed a maximum visible deviation of approximately:

$$\boxed{0.08 \text{ m}} \quad (17.42)$$

which corresponds to approximately:

$$\boxed{20\%} \quad (17.43)$$

of the 0.4-m reference radius.

This demonstrates that the proposed system provides functional trajectory tracking but still has room for improvement in tracking accuracy.

17.7 Conclusion

The proposed TurtleBot3 trajectory-tracking system offers a number of advantages, including closed-loop trajectory tracking, ROS 2 integration, Simulink-based implementation, nonlinear control, combined feedback and feedforward action, robot-frame error correction, position and orientation regulation, velocity saturation, modular architecture, and potential real-time implementation.

At the same time, several limitations must be considered. These include odometry errors, sensor noise, ROS 2 communication delays, model uncertainty, dependence on controller gains, actuator limitations, velocity saturation, and differences between simulation and real-world operation.

The simulation results indicate that the controller successfully produces the required circular motion, but the actual trajectory exhibits a measurable deviation from the reference.

Therefore, the proposed system can be considered a functional and structured trajectory-tracking solution, while further controller tuning, improved state estimation, and experimental validation would be required to achieve higher tracking accuracy under practical operating conditions.

Future Scope

18.1 Introduction

The present work demonstrates the implementation and simulation of a TurtleBot3 trajectory-tracking control system using a nonlinear kinematic controller, robot-frame error correction, velocity saturation, dynamic velocity control, MATLAB/Simulink, and ROS 2.

Although the simulation demonstrates that the robot can follow the required circular trajectory, several extensions can be considered to improve the practical applicability, robustness, and overall performance of the developed system.

The major areas of future development include physical robot implementation, obstacle avoidance, complex trajectory tracking, sensor fusion, adaptive control, multi-robot coordination, and robust control.

18.2 Physical Robot Implementation

The first and most important extension of the present work is implementation on an actual TurtleBot3 robot.

The current system is evaluated through simulation. A physical implementation would allow the controller to be tested under real operating conditions.

The controller could be deployed on an actual TurtleBot3 using the ROS 2 communication framework.

The implementation would require real-time acquisition of:

$$\boxed{x, \quad y, \quad \phi} \tag{18.1}$$

and generation of:

$$\boxed{v_c, \quad \omega_c} \quad (18.2)$$

The actual robot would introduce practical effects such as:

- wheel slip,
- sensor noise,
- actuator limitations,
- odometry errors,
- communication delays,
- floor-surface variations.

Therefore, experimental implementation would provide an important validation of the proposed control strategy.

18.3 Obstacle Avoidance

The present controller focuses primarily on trajectory tracking.

A natural extension is to combine trajectory tracking with obstacle avoidance.

In a practical environment, the robot may encounter static or moving obstacles while attempting to follow its reference trajectory.

The future system can therefore be developed as:

$$\boxed{\text{Trajectory Tracking} + \text{Obstacle Avoidance}} \quad (18.3)$$

The controller would need to maintain the desired trajectory while simultaneously ensuring that the robot does not collide with obstacles.

This would make the system more suitable for operation in realistic environments.

18.4 More Complex Trajectories

The present work evaluates the controller using a circular reference trajectory.

Future work can investigate the performance of the controller for more complex trajectories.

Possible trajectories include:

18.4.1 Figure-Eight Trajectory

A figure-eight path can be used to evaluate the controller during continuous changes in curvature and direction.

18.4.2 S-Shaped Trajectory

An S-shaped trajectory can be used to evaluate tracking performance during changes in the direction of curvature.

18.4.3 Spiral Trajectory

A spiral trajectory would introduce a continuously changing radius and therefore provide a more demanding tracking problem.

18.4.4 Waypoint Trajectory

A sequence of waypoints can be used to evaluate the controller for point-to-point navigation.

The future evaluation can therefore be represented as:

$$\boxed{\text{Circle} \rightarrow \text{Figure-Eight} \rightarrow \text{S-Shape} \rightarrow \text{Spiral} \rightarrow \text{Waypoint Path}} \quad (18.4)$$

Testing multiple trajectories would provide a more comprehensive evaluation of controller performance.

18.5 Sensor Fusion

The accuracy of trajectory tracking depends strongly on the quality of robot-state estimation.

Future work can therefore incorporate multiple sensors into a sensor-fusion framework.

The possible sensors include:

- odometry,
- IMU,

- LiDAR,
- camera.

18.5.1 Odometry

Odometry provides information about the robot's movement based on wheel motion.

However, accumulated odometry error can affect long-duration trajectory tracking.

18.5.2 IMU

An inertial measurement unit can provide information about angular motion and acceleration.

Combining IMU information with odometry can improve the estimation of robot orientation and motion.

18.5.3 LiDAR

LiDAR can provide environmental information and can support localization and obstacle detection.

18.5.4 Camera

Camera-based information can provide additional environmental and localization information.

The overall future architecture can therefore combine:

$$\boxed{\text{Odometry} + \text{IMU} + \text{LiDAR} + \text{Camera}} \quad (18.5)$$

to obtain a more reliable estimate of the robot state.

18.6 Adaptive Control

The current controller uses fixed controller parameters.

The parameters include:

$$\boxed{k_x = 1} \quad (18.6)$$

$$\boxed{k_\phi = 0.33} \quad (18.7)$$

$$\boxed{K_a = 25} \quad (18.8)$$

$$\boxed{K_b = 8} \quad (18.9)$$

and a lateral gain of:

$$\boxed{45} \quad (18.10)$$

In future work, these gains can be adjusted automatically according to the operating conditions.

An adaptive controller could modify the gains based on quantities such as:

- tracking error,
- robot velocity,
- trajectory curvature,
- external disturbances,
- operating conditions.

The basic concept can be represented as:

$$\boxed{\text{Tracking Condition} \rightarrow \text{Gain Adjustment} \rightarrow \text{Improved Control Response}} \quad (18.11)$$

This could potentially reduce tracking errors without requiring manual gain adjustment for every trajectory.

18.7 Multi-Robot Coordination

Another important extension is to apply the developed control approach to multiple TurtleBot3 robots.

Instead of controlling a single robot, several robots could operate simultaneously while communicating with each other.

The system could then be extended from:

$$\boxed{\text{Single Robot}} \quad (18.12)$$

to:

$$\boxed{\text{Multiple Coordinated Robots}} \quad (18.13)$$

Possible applications include:

- formation control,
- cooperative navigation,
- multi-robot transportation,
- distributed trajectory tracking,
- coordinated exploration.

The robots could exchange information about their positions, velocities, and desired trajectories through a communication network.

18.8 Robust Control

The performance of the controller can be further investigated under uncertain and disturbed conditions.

Future work can study the controller under:

18.8.1 Disturbances

External disturbances can cause the robot to deviate from its reference trajectory.

18.8.2 Noise

Sensor noise can affect the accuracy of the feedback signals.

18.8.3 Model Uncertainty

Differences between the mathematical robot model and the actual robot can influence controller performance.

18.8.4 Communication Delays

ROS 2 communication delays can introduce a time difference between state measurement and control action.

Therefore, future work can investigate:

$$\boxed{\text{Robust Control}} \quad (18.14)$$

to improve the system's ability to maintain acceptable tracking performance under these conditions.

18.9 Robustness Against Disturbances

The controller can be tested by intentionally introducing disturbances into the simulation.

For example, a disturbance can be applied to the robot's position or velocity while it is following the reference trajectory.

The resulting behaviour can be evaluated using:

$$\boxed{e_x, \quad e_y, \quad e_\phi} \quad (18.15)$$

The objective would be to determine how quickly the controller restores the robot to the desired trajectory after the disturbance.

A useful performance measure would therefore be the recovery time:

$$\boxed{T_{\text{recovery}}} \quad (18.16)$$

which represents the time required for the tracking error to return to an acceptable range.

18.10 Robustness Against Noise

Sensor noise can be introduced into the feedback signals to evaluate the sensitivity of the controller.

For example:

$$\boxed{x_{\text{measured}} = x + n_x} \quad (18.17)$$

$$\boxed{y_{\text{measured}} = y + n_y} \quad (18.18)$$

and:

$$\boxed{\phi_{\text{measured}} = \phi + n_\phi} \quad (18.19)$$

where:

$$n_x, \quad n_y, \quad n_\phi \quad (18.20)$$

represent measurement noise.

The controller can then be evaluated to determine whether it maintains stable trajectory tracking despite noisy measurements.

18.11 Robustness Against Model Uncertainty

The controller can also be evaluated when the robot parameters used in the controller do not exactly match the actual system parameters.

For example, variations can be introduced in:

- wheel parameters,
- robot mass,
- friction,
- actuator dynamics,
- velocity response.

This would allow the robustness of the controller to be evaluated under model uncertainty.

18.12 Communication Delay Analysis

The future system can also be tested with artificial ROS 2 communication delays.

A delay can be represented as:

$$\boxed{u(t) = f(x(t - \tau))} \quad (18.21)$$

where:

$$\tau \quad (18.22)$$

is the communication delay.

The controller performance can then be evaluated for different values of:

$$\tau \quad (18.23)$$

This would help determine the sensitivity of the trajectory-tracking system to communication latency.

18.13 Improvement of Trajectory Accuracy

The simulation result obtained in the present work shows a visible deviation between the actual and desired trajectories.

The maximum visible deviation was approximately:

$$0.08 \text{ m} \quad (18.24)$$

Relative to the reference radius:

$$R = 0.4 \text{ m}, \quad (18.25)$$

this represents approximately:

$$20\% \quad (18.26)$$

of the reference radius.

Future work can therefore focus on reducing this tracking error through:

- controller-gain optimization,
- adaptive control,
- improved state estimation,
- disturbance compensation,
- improved dynamic modelling,

- higher-quality feedback.

The performance objective can be expressed as:

$$e_p = \sqrt{e_x^2 + e_y^2} \rightarrow \text{minimum} \quad (18.27)$$

18.14 Real-Time Optimization

Another possible extension is optimization of the controller for real-time execution.

The computational requirements of the controller should be evaluated to ensure that the control algorithm can execute within the required sampling interval.

The objective is to satisfy:

$$T_{\text{computation}} < T_{\text{sampling}} \quad (18.28)$$

This becomes particularly important when the system is extended to more complex trajectories, sensor fusion, obstacle avoidance, or multiple robots.

18.15 Integration With Autonomous Navigation

The trajectory-tracking controller can eventually be combined with a higher-level autonomous navigation system.

A possible architecture is:

$$\text{Path Planner} \rightarrow \text{Trajectory Generator} \rightarrow \text{Trajectory Controller} \rightarrow \text{Dynamic Controller} \rightarrow \text{Robot} \quad (18.29)$$

The path planner would determine a suitable path, while the trajectory controller would ensure that the robot follows the generated trajectory accurately.

This would allow the developed control system to become part of a complete autonomous navigation framework.

18.16 Future Scope Summary

The major future extensions of the project are summarized below:

Table 18.1: Major Future Development Areas

Future Extension	Main Objective
Physical robot implementation	Validate the controller experimentally
Obstacle avoidance	Enable safe navigation around obstacles
Figure-eight trajectory	Test continuous curvature changes
S-shaped trajectory	Evaluate directional changes
Spiral trajectory	Test variable-radius motion
Waypoint trajectory	Evaluate point-to-point tracking
Sensor fusion	Improve state estimation
Adaptive control	Automatically tune controller gains
Multi-robot coordination	Extend control to multiple robots
Robust control	Improve performance under uncertainty
Disturbance rejection	Reduce effects of external disturbances
Noise rejection	Improve performance with noisy sensors
Delay analysis	Evaluate ROS 2 communication effects

18.17 Conclusion

The present work provides a foundation for further development of TurtleBot3 trajectory-tracking systems.

The current controller has been evaluated using a circular trajectory and demonstrates the ability to generate and track the required motion. However, the observed trajectory deviation indicates that there is considerable scope for improving tracking accuracy and extending the controller to more realistic operating conditions.

The most important future development is the implementation of the controller on a physical TurtleBot3. This would allow the effects of real sensor measurements, actuator behaviour, odometry errors, communication delays, and environmental disturbances to be investigated.

Further development can incorporate obstacle avoidance and more complex reference trajectories such as figure-eight, S-shaped, spiral, and waypoint trajectories.

Sensor fusion using:

$$\boxed{\text{Odometry} + \text{IMU} + \text{LiDAR} + \text{Camera}} \quad (18.30)$$

could improve state estimation, while adaptive control could automatically

modify controller gains according to the current tracking conditions.

Finally, extending the controller to multiple TurtleBot3 robots and investigating robust control under disturbances, noise, model uncertainty, and communication delays would significantly broaden the applicability of the developed control architecture.

Thus, the future scope can be summarized as:

$$\begin{array}{l} \text{Simulation} \rightarrow \text{Physical Robot} \rightarrow \text{Complex Environment} \\ \quad \rightarrow \text{Adaptive and Robust Control} \rightarrow \text{Multi-Robot System} \end{array} \quad (18.31)$$

These developments would transform the present trajectory-tracking framework into a more robust, adaptive, and practical autonomous robotic control system.

Conclusion

19.1 Introduction

This project investigated the development and simulation of a complete trajectory-tracking control system for a TurtleBot3 mobile robot. The main objective was to enable the robot to follow a predefined reference trajectory while continuously correcting its position and orientation errors.

The complete control system was developed using MATLAB/Simulink and integrated with ROS 2 for robot-state communication. A circular reference trajectory was selected for evaluating the performance of the proposed control architecture.

19.2 Summary of the Complete Work

The major stages completed in this project are summarized below.

19.2.1 TurtleBot3 Trajectory Tracking

The trajectory-tracking problem for a TurtleBot3 mobile robot was investigated.

The objective was to make the actual robot state:

$$\boxed{(x, y, \phi)} \quad (19.1)$$

follow the desired state:

$$\boxed{(x_d, y_d, \phi_d)} \quad (19.2)$$

19.2.2 ROS 2 Communication

ROS 2 was used as the communication framework between the robot model and the control system.

The robot-state information was obtained through the ROS 2 interface, allowing the controller to continuously receive feedback from the robot.

19.2.3 Odometry-Based State Acquisition

The robot's state information was acquired through the:

$$\boxed{/odom} \quad (19.3)$$

topic.

The odometry information provides the robot's position and orientation, which are required for calculating the trajectory-tracking errors.

19.2.4 Quaternion-to-Yaw Conversion

The orientation information obtained from the robot is represented using a quaternion.

For trajectory tracking, the relevant planar orientation is the yaw angle:

$$\boxed{\phi} \quad (19.4)$$

Therefore, the quaternion orientation was converted into yaw so that it could be used by the controller.

This provided the robot orientation required for calculating:

$$\boxed{\phi_d - \phi} \quad (19.5)$$

19.2.5 Circular Reference Trajectory Generation

A circular trajectory was generated as the desired reference.

The reference trajectory is given by:

$$\boxed{x_d(t) = 0.4 \cos(0.5t)} \quad (19.6)$$

and:

$$y_d(t) = 0.4 \sin(0.5t) \quad (19.7)$$

The desired angular velocity is:

$$\omega_d = 0.5 \text{ rad/s} \quad (19.8)$$

and the corresponding linear velocity is:

$$v_d = 0.2 \text{ m/s} \quad (19.9)$$

The reference radius is:

$$R = 0.4 \text{ m} \quad (19.10)$$

19.2.6 Tracking-Error Calculation

The difference between the desired and actual robot states was calculated.

The position errors are:

$$e_x = x_d - x \quad (19.11)$$

and:

$$e_y = y_d - y \quad (19.12)$$

The orientation error is:

$$e_\phi = \phi_d - \phi \quad (19.13)$$

These errors form the basis of the trajectory-tracking control action.

19.2.7 Transformation Into the Robot Frame

The global position errors were transformed into the robot coordinate frame.

The transformed errors are:

$$x_{r,\text{dif}} = e_x \cos \phi + e_y \sin \phi \quad (19.14)$$

and:

$$\boxed{y_{r,\text{dif}} = e_y \cos \phi - e_x \sin \phi} \quad (19.15)$$

This transformation allows the controller to interpret the tracking error relative to the robot's current heading.

19.2.8 Nonlinear Kinematic Controller

A nonlinear kinematic controller was implemented to generate the commanded linear and angular velocities.

The commanded linear velocity is:

$$\boxed{v_c = k_x x_{r,\text{dif}} + v_d \cos(\phi_{r,\text{dif}})} \quad (19.16)$$

and the commanded angular velocity is:

$$\boxed{\omega_c = k_\phi \sin(\phi_{r,\text{dif}}) + 45v_d y_{r,\text{dif}} + \omega_d} \quad (19.17)$$

Thus, the controller uses both tracking-error feedback and reference-motion information.

19.2.9 Dynamic Velocity Controller

The commanded velocities:

$$v_c \quad (19.18)$$

and:

$$\omega_c \quad (19.19)$$

were supplied to the dynamic velocity-control stage.

The dynamic controller attempts to make the actual robot velocities follow the commanded values.

The corresponding errors are:

$$\boxed{e_v = v_c - v} \quad (19.20)$$

and:

$$\boxed{e_\omega = \omega_c - \omega} \quad (19.21)$$

This provides an inner velocity-control loop within the overall trajectory-tracking architecture.

19.2.10 Velocity Saturation

Velocity saturation was included to ensure that the generated commands remain within the specified operating limits.

The linear velocity is limited by:

$$\boxed{-0.22 \leq v_c \leq 0.22 \text{ m/s}} \quad (19.22)$$

while the angular velocity is limited by:

$$\boxed{-2.5 \leq \omega_c \leq 2.5 \text{ rad/s}} \quad (19.23)$$

This prevents unrealistic velocity commands from being supplied to the robot model.

19.2.11 Simulink Implementation

The complete control architecture was implemented in MATLAB/Simulink.

The Simulink model integrates:

$$\boxed{\text{Reference Generation} \rightarrow \text{State Feedback} \rightarrow \text{Error Calculation}} \quad (19.24)$$

followed by:

$$\boxed{\text{Coordinate Transformation} \rightarrow \text{Kinematic Controller} \rightarrow \text{Velocity Saturation}} \quad (19.25)$$

and:

$$\boxed{\text{Dynamic Controller} \rightarrow \text{Robot Motion} \rightarrow \text{Feedback}} \quad (19.26)$$

This provides a complete closed-loop simulation.

19.3 Simulation Results

The simulation was used to evaluate the ability of the controller to track the circular reference trajectory.

The obtained X - Y trajectory demonstrated that the actual robot trajectory follows the general shape of the desired circular path.

However, the actual trajectory does not perfectly overlap the reference trajectory throughout the complete motion.

The largest visible deviation from the supplied simulation result was approximately:

$$\boxed{0.08 \text{ m}} \quad (19.27)$$

Relative to the reference radius:

$$R = 0.4 \text{ m}, \quad (19.28)$$

the approximate relative deviation is:

$$\frac{0.08}{0.4} \times 100 = \boxed{20\%} \quad (19.29)$$

The largest deviation was observed in the upper region of the trajectory, while the actual path became closer to the reference in other portions.

This demonstrates that the controller provides corrective action but does not achieve perfect zero-error tracking under the selected simulation conditions.

19.4 Overall Performance

The complete system can be represented by the following control sequence:

$$\boxed{\text{ROS 2} \rightarrow \text{Odometry} \rightarrow \text{State Estimation} \rightarrow \text{Trajectory Generation}} \quad (19.30)$$

$$\boxed{\rightarrow \text{Error Calculation} \rightarrow \text{Robot-Frame Transformation}} \quad (19.31)$$

$$\boxed{\rightarrow \text{Nonlinear Kinematic Controller} \rightarrow \text{Velocity Saturation}} \quad (19.32)$$

$$\boxed{\rightarrow \text{Dynamic Velocity Controller} \rightarrow \text{TurtleBot3} \rightarrow \text{Feedback}} \quad (19.33)$$

This closed-loop structure allows the controller to continuously compare the actual robot state with the desired state and modify the control commands accordingly.

19.5 Main Contributions of the Project

The major outcomes of the project can be summarized as follows:

- A complete TurtleBot3 trajectory-tracking problem was formulated.
- ROS 2 was integrated for robot-state communication.
- Odometry data were used for obtaining robot-state information.
- Quaternion orientation was converted into yaw for planar control.
- A circular reference trajectory was generated.
- Position and orientation tracking errors were calculated.
- Global-frame errors were transformed into the robot frame.
- A nonlinear kinematic controller was implemented.
- Commanded linear and angular velocities were generated.
- Velocity saturation was incorporated.
- A dynamic velocity-control loop was implemented.
- The complete architecture was developed in Simulink.
- The system was evaluated through simulation.
- The simulated trajectory was compared with the desired trajectory.
- The maximum visible trajectory deviation was estimated from the simulation result.

19.6 Final Conclusion

The project successfully developed and simulated a complete closed-loop trajectory-tracking control architecture for TurtleBot3.

The system combines ROS 2 communication, odometry-based state feedback, quaternion-to-yaw conversion, circular trajectory generation, tracking-error calculation, robot-frame coordinate transformation, nonlinear kinematic control, velocity saturation, and dynamic velocity control.

The reference trajectory used in the simulation was a circle of radius:

$$\boxed{0.4 \text{ m}} \quad (19.34)$$

with:

$$\boxed{v_d = 0.2 \text{ m/s}} \quad (19.35)$$

and:

$$\boxed{\omega_d = 0.5 \text{ rad/s}} \quad (19.36)$$

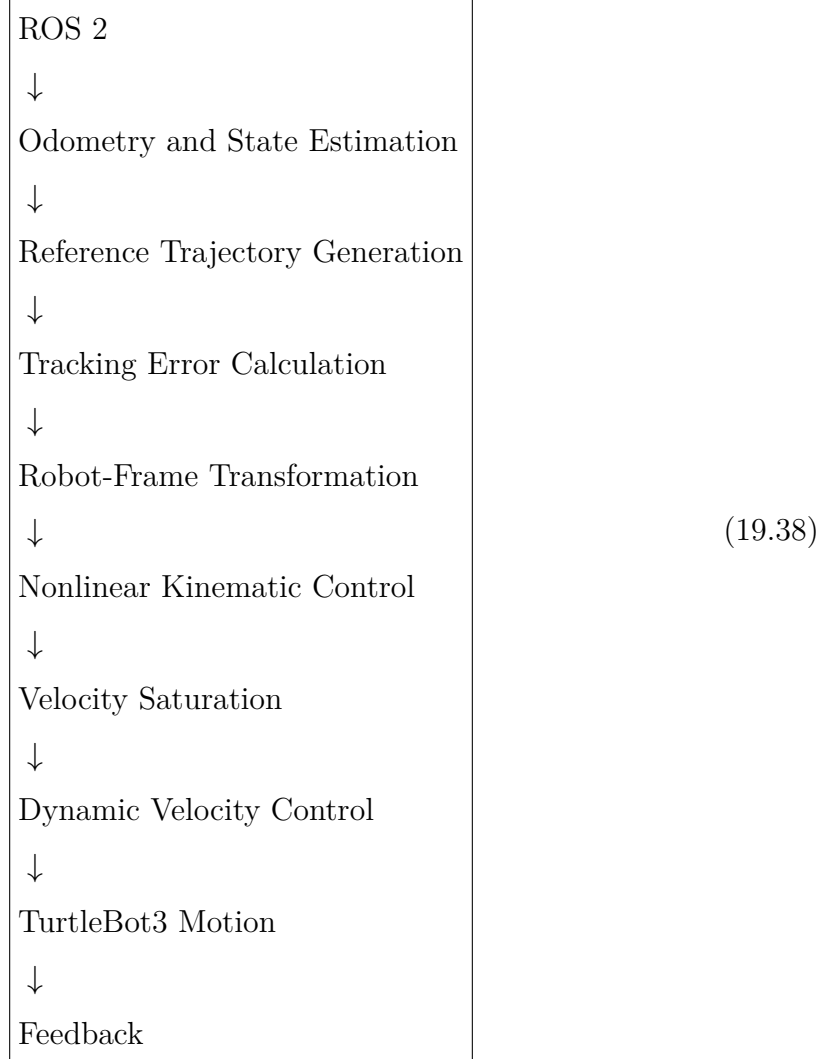
The simulation results demonstrate that the TurtleBot3 follows the general circular reference trajectory. Although a visible tracking error remains, the controller provides corrective action and maintains the robot within the general region of the desired path.

The observed maximum deviation of approximately:

$$\boxed{0.08 \text{ m}} \quad (19.37)$$

indicates that the system achieves functional trajectory tracking, while also providing a clear direction for future improvement through controller tuning, improved state estimation, robust control, and physical robot testing.

Therefore, the overall developed system can be summarized as:



In conclusion, the developed MATLAB/Simulink and ROS 2 based control framework provides a structured foundation for TurtleBot3 trajectory tracking and can be further extended toward more complex trajectories, obstacle avoidance, sensor fusion, adaptive and robust control, and real-world implementation.

References

The following references were used as the main technical and methodological sources for the development of the TurtleBot3 trajectory-tracking project.

20.1 TurtleBot3 Documentation

- [1] ROBOTIS, *TurtleBot3 e-Manual — TurtleBot3 Documentation*, ROBOTIS.

Available online: <https://emanual.robotis.com/docs/en/platform/turtlebot3/>

This documentation was used as a reference for the TurtleBot3 platform, robot operation, ROS 2 integration, and simulation environment.

- [2] ROS 2 Documentation

Open Robotics, *ROS 2 Documentation — Jazzy Jalisco*, Open Source Robotics Foundation.

Available online: <https://docs.ros.org/en/jazzy/>

This reference was used for understanding ROS 2 communication, interfaces, topics, nodes, and the general ROS 2 architecture used in the project.

- [3] MATLAB Documentation

MathWorks, *MATLAB Documentation*, The MathWorks, Inc.

Available online: <https://www.mathworks.com/help/matlab/>

MATLAB was used for mathematical calculations, controller development, signal processing, and analysis of simulation results.

- [4] Simulink Documentation

MathWorks, *Simulink Documentation — Simulation and Model-Based Design*, The MathWorks, Inc.

Available online: <https://www.mathworks.com/help/simulink/>

Simulink was used to implement the complete block-based control architecture and perform the simulation of the trajectory-tracking system.

[5] ROS Toolbox Documentation

MathWorks, *ROS Toolbox Documentation*, The MathWorks, Inc.

Available online: <https://www.mathworks.com/help/ros/>

ROS Toolbox was used as the interface between MATLAB/Simulink and ROS 2. The documentation also provides information regarding ROS 2 communication, simulation, data handling, and deployment.

[6] Nonlinear Trajectory-Tracking Control

J. M. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, “A Stable Tracking Control Method for an Autonomous Mobile Robot,” *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 1990.

This reference is relevant to the development of stable trajectory-tracking control laws for mobile robots and the use of tracking errors in the robot coordinate frame.

[7] Differential-Drive Robot Modelling and Trajectory Tracking

“Modelling and Trajectory Tracking of Wheeled Mobile Robots,” *Procedia Technology*, vol. 24, pp. 538–545, 2016.

doi: [10.1016/j.protcy.2016.05.094](https://doi.org/10.1016/j.protcy.2016.05.094).

This work discusses differential-drive mobile-robot modelling, including kinematics, actuator dynamics, rolling resistance, and trajectory-tracking control.

[8] Kinematic and Dynamic Control of Mobile Robots

“A Kinematic and Dynamic Model-Based Motion Controller for Mobile Robots,” *IFAC Proceedings Volumes*, vol. 35, no. 1, pp. 427–432, 2002.

doi: [10.3182/20020721-6-ES-1901.01296](https://doi.org/10.3182/20020721-6-ES-1901.01296).

This reference is relevant to the combination of kinematic and dynamic control for differential-drive mobile robots and the consideration of velocity and acceleration constraints.

[9] Nonlinear Path Generation and Tracking

J. L. Avendaño-Juárez, V. M. Hernández-Guzmán, and R. Silva-Ortigoza, “Nonlinear Path Generation and Tracking Control Design for a Mobile Robot,” *IFAC-PapersOnLine*, vol. 56, no. 2, pp. 9777–9780, 2023.

doi: [10.1016/j.ifacol.2023.10.294](https://doi.org/10.1016/j.ifacol.2023.10.294).

This work addresses trajectory generation and nonlinear tracking control for differential-drive mobile robots.

[10] Differential-Drive Trajectory Tracking

“Trajectory Tracking and Control of Differential Drive Robot for Predefined Regular Geometrical Path,” *Procedia Technology*, vol. 25, pp. 1273–1280, 2016.

doi: [10.1016/j.protcy.2016.08.221](https://doi.org/10.1016/j.protcy.2016.08.221).

This reference discusses trajectory tracking for differential-drive robots and evaluates tracking of predefined geometrical paths through simulation and experimental results.

[11] Nonlinear Dynamic Inversion for Mobile-Robot Tracking

“Trajectory Tracking of Autonomous Mobile Robots Using Nonlinear Dynamic Inversion,” *IFAC-PapersOnLine*, vol. 51, no. 1, pp. 202–207, 2018.

doi: [10.1016/j.ifacol.2018.05.041](https://doi.org/10.1016/j.ifacol.2018.05.041).

This reference discusses nonlinear trajectory tracking for differential-drive mobile robots and includes evaluation using circular and figure-eight reference trajectories.

[12] Robust Trajectory Tracking

“Robust Tracking Control of a Differential Drive Wheeled Mobile Robot Using Fast Nonsingular Terminal Sliding Mode,” *Computer Standards & Interfaces*, 2021.

Available online: <https://www.sciencedirect.com/science/article/pii/S0045790621004420>.

This reference provides additional background on robust trajectory tracking of differential-drive wheeled mobile robots in the presence of nonlinearities, uncertainties, and constraints.

[13] Parametric Identification and Dynamic Control

“Parametric Identification and Controller Design for a Differential-Drive Mobile Robot,” *IFAC-PapersOnLine*, vol. 51, no. 15, pp. 437–442, 2018.

doi: [10.1016/j.ifacol.2018.09.184](https://doi.org/10.1016/j.ifacol.2018.09.184).

This reference discusses dynamic modelling, parameter identification, and inner/outer control loops for differential-drive mobile robots.

- [14] Reference Software and Simulation Resources

ROBOTIS, *TurtleBot3 Simulation Documentation*, ROBOTIS.

Available online: <https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>

The documentation was used as a reference for TurtleBot3 simulation and the use of virtual robot models in simulation environments.

- [15] Reference on ROS 2 Deployment

MathWorks, *ROS 2 Node Generation and Deployment*, The MathWorks, Inc.

Available online: <https://www.mathworks.com/help/ros/ros-and-ros2-node-generation.html>

This reference provides information regarding the generation and deployment of ROS 2 nodes from MATLAB and Simulink.